计算机网络笔记

HNU《计算机网络》课程学习笔记

作者：long6177 | 撰写时间：2026 春季学期

- 计算机网络 introduction
  - 0.1 课程特点
  - 0.2 基本内容(Top to Down 自顶向下介绍)
- 第一章：概论
  - 1.1 何为Internet
  - 1.2 网络边缘
  - 1.3 网络核心
  - 1.4 接入网
  - 1.5 分组交换的时延,丢包
  - 1.6 协议层次和服务模型
- 第二章：应用层(Application Layer)
  - 2.1 应用层协议原理
  - 2.2 Web和HTTP
  - 2.3 FTP
  - 2.4 电子邮件
  - 2.5 DNS: 域名系统
  - 2.6 P2P文件共享
  - 2.7 CDN
  - 2.8 套接字编程
- 第三章：传输层
  - 3.1 传输层概述
  - 3.2 多路复用和解复用
  - 3.3 无连接传输: UDP
  - 3.4 可靠数据传输(rdt)的原理
  - 3.5 面向连接的传输: TCP
  - 3.6 拥塞控制原理
  - 3.7 TCP 拥塞控制
- 第四章：网络层 - 数据平面
  - 4.1 导论 -- 网络层概述
  - 4.2 路由器组成
  - 4.3 IP 协议
  - 4.4 ICMP
- 第五章：网络层 - 控制平面
  - 5.1 路由的概念

    - 5.2 层次路由
- 第六章：链路层
    - 6.1 概述与服务
    - 6.2 差错检测和纠错
    - 6.3 多(址)路访问协议

计算机网络 introduction

- 0.1 课程特点
- 0.2 基本内容(Top to Down 自顶向下介绍)

0.1 课程特点

1. 概念多, 核心概念都要记
2. 要求**逻辑推理能力**强, 文字推理

0.2 基本内容(Top to Down 自顶向下介绍)

> 前半部分介绍功能和服务, 以及相关原理
> 后半部分介绍案例

- 导论: 计算机网络和互联网
- 应用层
  应用层协议(UDP TCP)....
- 传输层
- 网络层: 数据平面
- 网络层: 控制平面
- 数据链路层和局域网(硬件)
- 网络安全
- 无线和移动网络
- 多媒体网络
- 网络管理

第一章：概论

- 1.1 何为Internet
  - 1.1.1 具体构成角度
  - 1.1.2 提供服务角度
  - 1.1.3 什么是协议(protocol)
- 1.2 网络边缘
  - 1.2.1 网络边缘
  - 1.2.2 应用的通信服务
- 1.3 网络核心
  - 1.3.1 电路交换
  - 1.3.2 分组交换
  - 1.3.3 分组交换 vs 电路交换
- 1.4 接入网
  - 1.4.1 接入网的方式
  - 1.4.2 物理介质
  - 1.4.3 广域网, 局域网, 城域网
- 1.5 分组交换的时延,丢包
  - 1.5.1 时延类型
  - 1.5.2 排队时延和丢包
  - 1.5.3 吞吐量
- 1.6 协议层次和服务模型
  - 1.6.1 服务
  - 1.6.2 层次化方式实现复杂网络功能
  - 1.6.3 Internet协议栈
  - 1.6.4 各层次的协议数据单元
  - 1.6.5 封装

主要介绍**框架性内容**, 为后面打下基础

1.1 何为Internet

1.1.1 具体构成角度

计算机网络包含以下设备

1. **主机(host)/端系统(end system)** [节点]
   主机作为数据的源或目的
2. 端系统通过**通信链路**和**分组交换机**(packed switch)连接到一起
3. 分组交换机[节点]有网络层**路由器**(router)和链路层**交换机**(link-layer switch)两种主要类型
4. 链路[边]可以分为**接入网链路**和**主干链路**
   - 接入网链路(access): 主机到互联网的链路;

    - 主干链路(back bone): 路由器之间的链路
5. 以上部件都运行一系列**协议**(protocol)

Internet 就是以TCP/IP作为主要协议的计算机网络.
互联网是"网络的网络", 小型网络通过网络互联设备组成大的网络, 即为Internet

1.1.2 提供服务角度

因特网即为**向应用程序提供(通信)服务的基础设施**

1. 这里的应用程序是指运行在端系统上的**分布式应用程序**, 即他们涉及多个相互交换数据的端系统
2. **套接字接口**(socket interface), 该接口规定了运行在一个端系统上的程序请求Internet向另一个端系统的特定目的地程序交换数据的方式. 是一套发送程序必须遵守的规则集合

1.1.3 什么是协议(protocol)

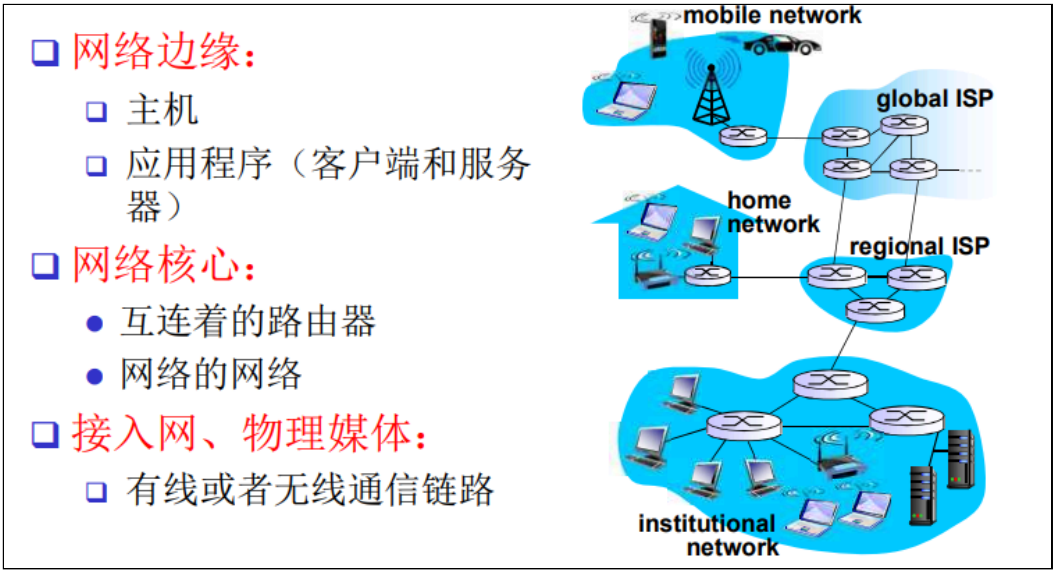
协议定义了在两个或多个通信实体之间交换的**报文的格式(语法和语义)和顺序**, 以及报文发送和/接收一条报文或其他事件**所采取的动作**

网络的不同对等层次有不同的协议, 应用层的协议, 传输层的协议...

协议就是标准, 全球的InterNet标准有**互联网工程任务组IETF**研发, IETF的标准文档称为**请求评论(RFC)**, 他定义了IP, TCP, HTTP等很多协议

1.2 网络边缘

互联网可以分为**网络边缘(edge)**和**网络核心(core)**, 以及**接入网(access)**. 三个子系统构成互联网



1.2.1 网络边缘

需要运行应用程序的端系统即构成网络边缘

端系统(end system)又叫主机(hosst), 可以划分为两类: **客户**(client) 和 **服务器**(server)

1.2.2 应用的通信服务

基础设施为应用提供通信服务的两种方式

- 第一种: 面向连接通信服务
  端系统通信之前要"握手", 先建立连接状态. [TCP-传输控制协议]
    - 使用TCP应用:HTTP(Web), FTP(文件传输), Telnet(远程登录), SMTP(email)

- 第二种: 无连接服务
  不需要握手 [UDP-用户数据报协议]
    - 使用UDP的应用: 流媒体, 远程会议, DNS, Internet电话(实时性)

1.3 网络核心

通过网络链路和交换机移动数据的两种基本方式:

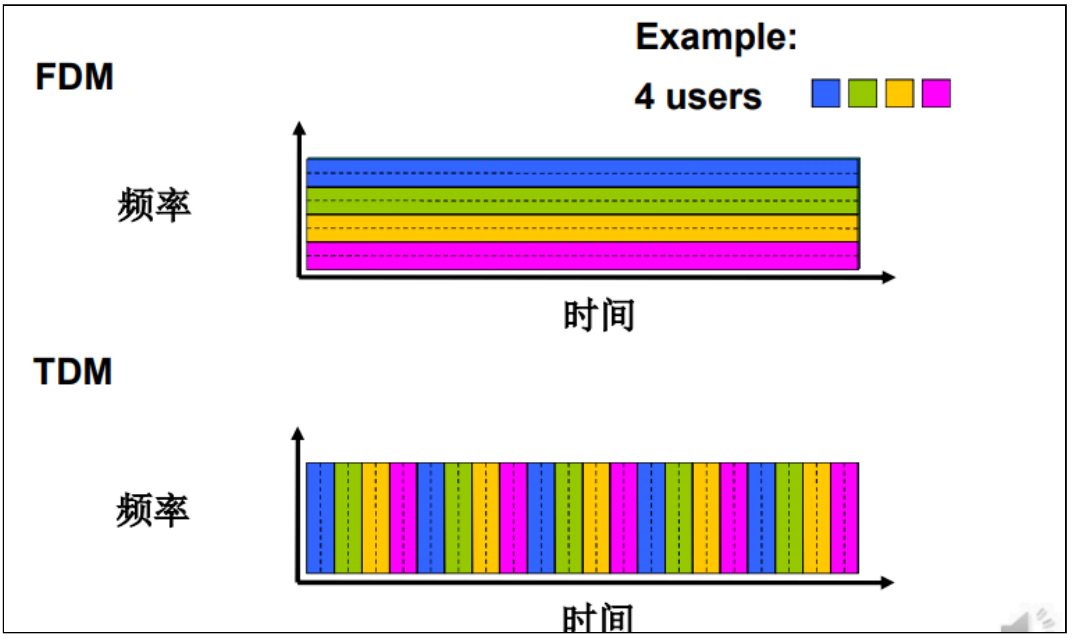
1. **电路交换(circuit switching)**
2. **分组交换(packet switching)**

1.3.1 电路交换

源主机和目的主机之间建立一个独享的线路(电路circuit), 是一种**专门的端到端连接(end to end)**

1.3.1.1 链路的片

电路交换中, 一个较宽的链路能够被分为片用于复用, 分片方式有**频分(FDM)**, **时分(TDM)** 和 **波分(WDM)**



1.3.2 分组交换

源将长报文划分为较小的数据块, 称之为**分组(packet)**

不同于**电路交换**, 分组以等于该**链路最大传输速率的**速度传输通过通信链路

分组交换有如下特征:

1. **存储转发传输**机制
   交换机能够开始向输出链路传输该分组的第一个比特之前, 必须先接收完整个分组
2. **排队时延和分组丢失**
   分组交换机会给每个要输出的链路分配一个**输出缓存(output buffer, 也叫输出队列)**, 当要输出的链路被其他占用时, 先把分组放在**输出缓存**中排队
   另外, 输出缓存满的时候, 分组没有地方排队, 就**分组丢失(丢包)**
3. **转发表和路由选择协议**
   路由器同意**IP地址**和**转发表**来决定到达的分组应该传向哪条链路

分组交换不用建立独享的电路, 使得**共享性更强**, 但会有时延和丢包的代价

1.3.3 分组交换 vs 电路交换

- 分组交换是 "突发数据的胜利者";
- 电路交换是**预先分配**, 分组交换是**按需分配**
- 分组交换, 某时刻活跃用户数较大的概率是比较小的, 因此分组交换能够在用户数更大的情况下实现和电路交换同样的性能

1.4 接入网

接入网的目标是将主机连接到**边缘路由器**
边缘路由器是端系统Host去往任何其他远程端系统的路径上的第一台路由器

1.4.1 接入网的方式

- **有线网络**接入技术: **光纤到户FTTH**, **以太网**, 同轴电缆, 双绞线的DSL, 古老的拨号上网
- **无线网络**接入技术: **WiFi、4G/5G**, 卫星广域覆盖
- 接入场景: 住宅 (家庭) 接入网, 机构 (学校、公司) 接入网, 移动接入网络 (WiFi、4G/5G)

1.4.2 物理介质

物理介质是连接主机和边缘路由器的物理设备, 有:

1. **电缆**
2. **光纤**
3. **无线介质**
4. **其他介质**

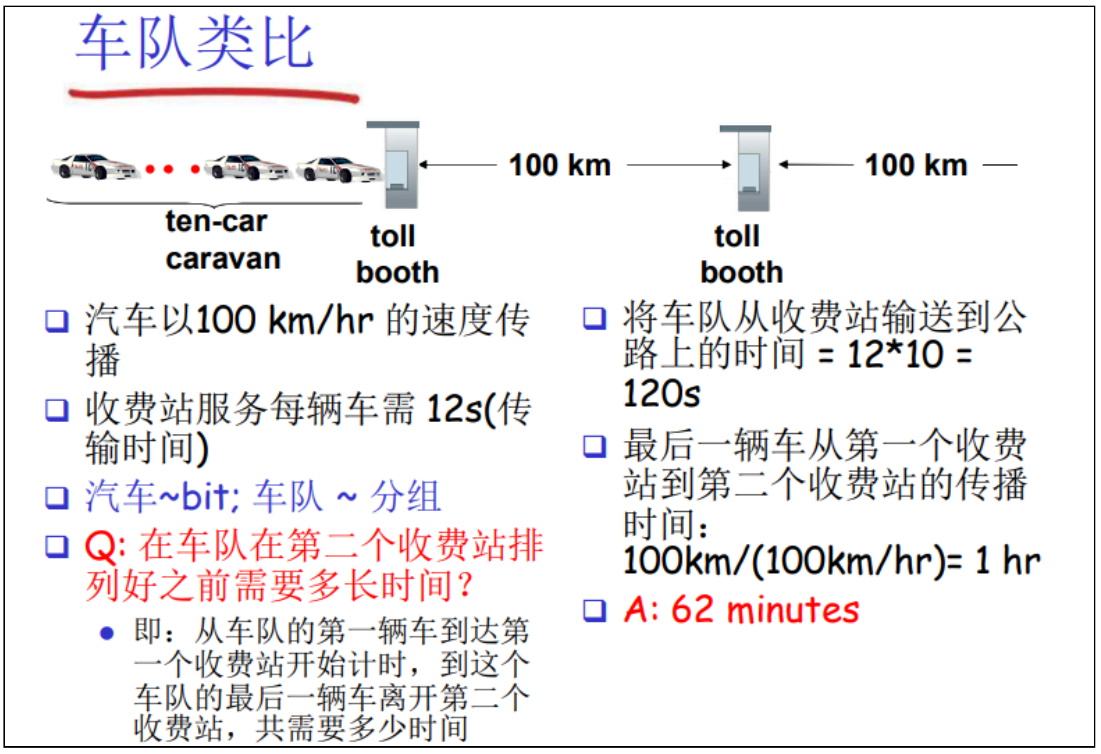
1.4.3 广域网, 局域网, 城域网

1. **广域网(WAN)**
   覆盖范围大, 如一个大城市一个国家或洲际间建立的网络
2. **局域网(LAN)**
   覆盖范围有限, 属于**一个部门或单位组建**的小范围网络
3. **城域网(CAN)**
   包括城市网络, 区域网络等

1.5 分组交换的时延,丢包

1.5.1 时延类型

- 处理时延
  检查分组的首部和决定将分组导向何处所需要的时间是处理时延的一部分
- 排队时延
  在队列中, 分组在目标链路的**缓存**上经受排队时延. 取决于当前链路上排队分组的数量
- **传输时延**
  用L比特表示分组长度, $R\ bps$ 表示路由器A到B的链路传输速率, 传输时延即为L/R, 即为将分词的所有比特推向链路所需要的时间
- **传播时延**
  用d代表物理链路长度, s代表媒体上的传播速度, 传播时延即为d/s, 和两个路由器间的物理距离有关



1.5.2 排队时延和丢包

1.5.2.1 排队时延

假定每个分组到达队列的平均概率为a 分组/秒, 且所有分组都是L比特, 那么定义**流量强度**值的是La/R, 这个代表了到达比特速率和输出比特速率的比值

- $La/R > 1$的话, 比特到达队列的平均速率超过从该队列传输出去的速率, 队列趋于无限大, 因此, 现实中的工程设计不会出现$La/R > 1$的情况
  一个事实是: **当流量强度接近于1时, 平均排队时延会很大**

1.5.2.2 丢包

队列满了的时候, 之后到达的分组就没有地方排队, 就会丢失.

1.5.3 吞吐量

吞吐量: 在远端和目标端(end2end)之间的速率(数据量/单位时间)

- 瞬间吞吐量: 在一个时间点的速率
- 平均吞吐量: 在一个长时间内的平均值

对于多链路的端到端线路, 吞吐量取决于**瓶颈链路**
例如一个在服务器端和客户端之间具有N条链路的网络, R_i是第i条链路的传输速率, 那么端到端的吞吐量为 $min\{R_1, R_2, \ldots, R_N\}$

1.6 协议层次和服务模型

Internet是个非常复杂的系统, 为了组织他的网络体系结构, 我们可以用**分层体系结构进行模块化组织**

1.6.1 服务

服务(Service): 低层实体向上层实体提供他们之间通讯的能力
原语(primitive): 上层使用下层服务的方式, 高层使用低层提供的服务, 以及低层向高层提供服务都是通过服务访问原语来进行交互的
服务访问点SAP(Services Access Point): 上层使用下层提供的服务要通过层间的接口--地点;

1.6.2 层次化方式实现复杂网络功能

- 将网络复杂的功能分层功能明确的层次, 每一层实现了其中一个或一组功能, 功能中有**它的上层**可以使用的功能: **服务**
- 本层协议实体相互交互执行本层的协议动作, 目的是**实现本层功能, 通过接口为上层提供更好的服务**
- 在实现本层协议的时候, **直接利用了下层所提供的服务**
- 本层的服务: 借助下层服务实现的本层协议实体之间交互带来的新功能 (上层可以利用的) + 更下层所提供的服务

1.6.3 Internet协议栈

网络设计这以**分层**的方式组织**协议**以及实现这些协议的网络硬件和软件, 这些协议属于这些层次之一

各层的所有协议被称为**协议栈(protocol stack)**, 因特网的协议栈有五个层次组成, **自顶向下**依次是:

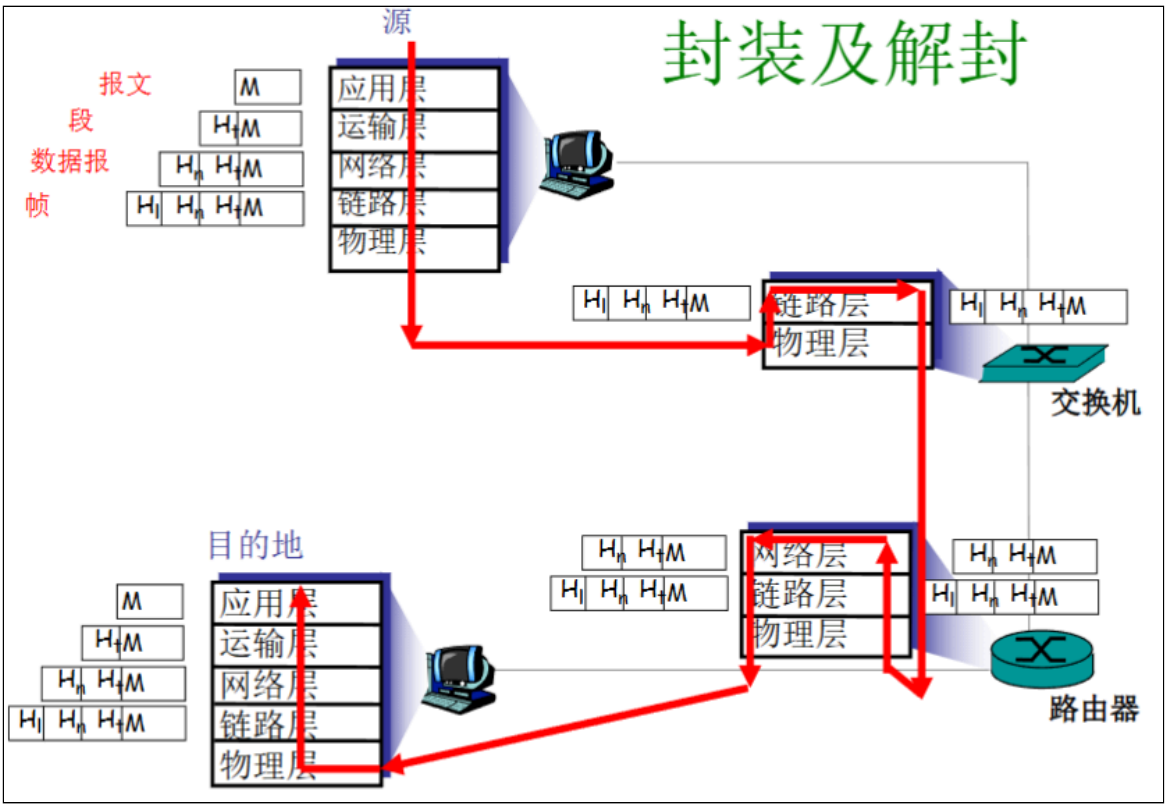
- **应用层**
  网络应用程序和它们的应用层协议存留的地方, 协议有: **FTP, HTTP, DNS**
- **传输层**
  主机到主机数据传输. 协议有: **TCP, UDP**
- **网络层**
  从源到目的地数据报的选路, 协议有**IP, 选路协议**
- **链路层**
  在邻近网元之间传输数据, **PPP, 以太网**
- **物理层**
  将数据转换为物理信号, 并在物理媒体上传输, **比特**

1.6.4 各层次的协议数据单元

1. **应用层**: 报文(message)
2. **传输层**: 报文段(segment)[TCP段, UDP数据报]
3. **网络层**: 分组packet(若无连接方式: 数据报datagram)
4. **链路层**: 帧(frame)
5. **物理层**: 位(bit)

1.6.5 封装

封装: 将数据从上层协议转换为下层协议, 并在下层协议的首部添加必要的信息



应用层报文 → 加运输层首部(运输层向上给应用程序交付报文的信息) → 运输层报文段 → 加网络层首部(源和目的端系统等信息) → 网络层数据报 → 加链路层首部 → 链路层帧

第二章：应用层(Application Layer)

- 2.1 应用层协议原理
  - 2.1.1 应用程序体系结构
  - C/S和P2P混合体
  - 2.1.2 进程通信
  - 2.1.3 运输层服务
  - 2.1.4 因特网提供的运输服务
- 2.2 Web和HTTP
  - 2.2.1 HTTP协议概况
  - 2.2.2 非持续连接和持续连接
  - 2.2.3 HTTP报文格式
  - 2.2.4 用户和服务器的交互--cookie
  - 2.2.5 Web缓存
- 2.3 FTP
  - 2.3.1 FTP 命令
- 2.4 电子邮件
  - 2.4.1 SMTP协议
  - 2.4.3 邮件访问协议
- 2.5 DNS: 域名系统
  - 其他服务 -- 负载分配
  - 2.5.1 DNS名字空间 -- 一颗 "树"
  - 2.5.2 DNS名字服务器
  - 2.5.3 两种查询过程
  - 2.5.4 DNS协议, 报文
  - 2.5.5 DNS攻击
  - 2.5.6 DNS的维护
- 2.6 P2P文件共享
  - 2.6.1 BitTorrent协议
  - 2.6.2 Distributed Hash Table (DHT)
- 2.7 CDN
- 2.8 套接字编程
  - 2.8.1 TCP套接字编程
  - 2.8.2 UDP套接字编程

> 通过应用层的学习, 有助于更好了感受到**协议的方方面面**, 遇到的问题也学习运输层，网络层及数据链路层也会遇到

2.1 应用层协议原理

研发网络应用程序的**核心**是写出能够运行在不同的**端系统**和**通过网络彼此通信**的程序

应用程序软件要为**端系统**编写, 而不是路由器或链路层交换器这些网络核心设备

2.1.1 应用程序体系结构

应用程序体系结构(Application Architecture) 规定了如何如何在各种端系统上组织该应用程序

有**两个主流体系结构**

1. 客户-服务器体系结构(C/S)
2. P2P(peer to peer)体系结构/分布式体系结构(Distributed)

C/S体系结构

C/S体系结构中:

- 服务器(server):
  一直运行(被动接受服务请求);
  有固定的IP地址和周知的端口号(约定)
  会扩展成*服务器场* (google数据中心)
- 客户端(client):
  主动与服务器通信(请求服务);
  与互联网有间歇性的连接;
  可能是动态IP地址;
  客户相互之间不直接通信

具有C/S体系结构的应用程序, 最著名的就是Web引用程序

P2P体系结构(P2P architecture)

P2P 体系结构特点

- 几乎没有一直运行的"服务器"
- 任意端系统(称为对等方peer)之间可以直接进行通信
- 每一个节点既是客户端又是服务器
- 参与的主机间歇性连接并且是可以改变IP地址的

具有P2P体系结构的应用程序, **迅雷**, 每一个用户上线时都能作为一个"服务器"供他人来进行文件下载

P2P的自扩展性
在一个P2P文件共享系统中, 尽管每个对等方都由于请求文件而产生工作负载, 但**每个对等方通过向其他对等方分发文件来为系统增加服务能力**

C/S和P2P混合体

对于一些**即时讯息应用**而言, 服务器被用于跟踪用户的IP地址, 但用户到用户的报文也可以在用户主机之间直接发送接受

2.1.2 进程通信

程序间通信:

1. 对于同一主机上的多个**进程**(即理解成一个程序)
   进程间通信规则由主机上的操作系统来确定
2. 对于不同主机(端系统)上的进程
   通过**跨越计算机网络交换message(报文)来进行相互间的通信**

对于每一对进程通信, 通常将两个进程之一标识为**客户(client)**, 另一个进程表示为服务器(server).
例如: P2P应用中, 下载文件的peer标识为客户, 上载文件的peer标识为服务器

进程和网络之间的接口--套接字

套接字是同一台主机内应用层和运输层之间的接口
由于该套接字是建立网络应用程序的可编程接口, 因此套接字也称为应用程序和网络之间的**应用程序编程接口(Application Programming Interface, API)**

IP地址

在因特网中, 主机由其**IP地址进行唯一标识**
IP地址是一个32位地址(IPv4), 有`.`号分隔的二进制数字组成

进程间通信时, 有**发送进程**向**接受进程**发送分组时, 需要知道接受进程的地址, 为了标识, 需要有两个信息:

1. 进程所在的主机的地址, 即IP;
2. 进程的端口号
   一台主机有多个进程, 进程通信要精确到单个进程到另一个主机的单个进程(光有IP地址不够), 就需要这样的信息, 我们定义成**端口号**, 用来唯一标识主机上的某个进程

需要重点记忆的端口号:

1. Web服务器用**端口号80**来表示
2. 邮件服务器进程(使用SMTP协议)用端口号25来标识

2.1.3 运输层服务

应用层的应用程序通过套接字接触到**运输层协议**
运输层协议有多个(UDP, TCP等), 如何选到恰当的, 需要有以下的考量:

1. **可靠数据传输(可靠性)**
   一些应用要求程序的一段发送的**数据能够正确的, 完整的交付给该应用程序的另一端**, 如果那个协议能提供这样的服务, 那么就认为提供了**可靠数据传输(reliable data transfer)**
   当然也会有**容忍丢失的应用**不需要可靠性
2. **吞吐量(带宽)**
   吞吐量大概就是发送进程向接受进程交付比特的速率
3. **定时(低时延)**
   某些应用, 不如联网多方游戏就非常要求**低时延**
4. **安全性**

2.1.4 因特网提供的运输服务

本小节考察更具体的因特网提供的两种运输服务类型 **UDP**和**TCP**

TCP

- 面向连接服务: 客户机和服务器之间所需的建立("握手")
- 可靠传输: 在发送和接受进程之间, 无差错,按适当次序收发数据
- 流量控制: 发送方不会淹没接收方
- 拥塞控制: 当网络过载时抑制发送方
- 并不提供: 定时, 最小宽带保证(吞吐量)

TCP有可靠传输的特点
电子邮件, Web, 远程终端访问, 文件传输, 流式多媒体大多运行在TCP
微信qq文字消息, 在线充值支付...

UDP

- 在发送进程和接受进程之间的不可靠数据传输
- 不提供: 连接建立; 可靠性; 流控; 拥塞控制; 定时; 带宽保证;

UDP有快速传输的特点
因特网电话应用通常能容忍某些丢失, 因此通常被运行在UDP上
微信语音, 多人在线实时游戏, 视频会议, 抖音直播等

2.2 Web和HTTP

Web页是由**对象**组成, 对象可以是HTML文件, JPEG图片, Java小程序, 音频文件等...
每个对象都存储在服务器中, 可以通过URL寻址, 例如:

```
www.someschool.edu/someDept/pic.gif
```

其中`www.someschool.edu`是存储对象的主机名;
`/someDept/pic.gif`是某对象在主机上的位置

Web的应用层协议就是**超文本传输协议(HyperText Transfer Protocol, HTTP)**

2.2.1 HTTP协议概况

HTTP有两个程序实现, 客户程序和服务器程序, 他们运行在两个不同的端系统上, 通过**交换HTTP报文进行会话**
HTTP定义了这些**报文的结构以及交换的格式规则**

HTTP**使用TCP**作为它的支撑运输协议, 首先用户会发起与服务器的TCP连接请求("三次握手"), 客户和服务器程序都通过套接字接口来访问TCP连接来发送和接收HTTP报文
分层体系结构**最大的优点**:

1. HTTP协议不用担心数据的丢失
2. HTTP协议也不用关注低层的TCP从网络数据丢失和乱序故障中恢复的细节

HTTP是 **"无状态的"**, 每个HTTP报文都是独立的, 与之前的报文没有联系, 为了记住"状态", 后面会引入**cookie**来实现状态保持

2.2.2 非持续连接和持续连接

1. 非持续连接
   至多一个对象经过一个TCP连接发送 (HTTP/1.0使用)
2. 持续连接
   多个对象能够经过客户机和服务器之间的单个TCP连接发送 (HTTP/1.1使用)

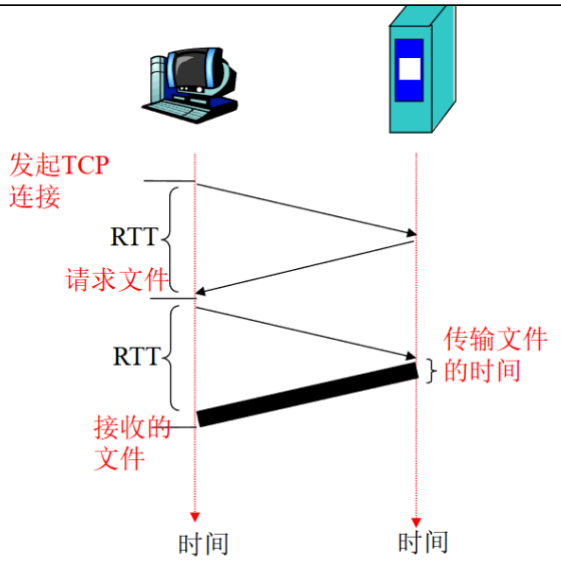
响应时间RTT(road-trip time)

往返时延RRT的定义: 从客户机到服务器发送一个小分组并返回所历经的时间.

响应时间:

- 一个RTT发起TCP连接
- 对HTTP请求和返回的HTTP响应前几个字节的一个RTT
- 文件传输时间

总计 = 2RTT+传输时间



1、整个过程涉及一次"三次握手"，前两次用掉一个RTT

2、第三次+HTTP请求，然后服务器响应，总共用掉第二个RTT

持续连接的TCP运输又有**非流水线**和**流水线**两种方式

无流水线的持续

客户：要买个手机
老板：把手机递给客户
客户：我还要买个电池
老板：把电池递给客户
客户：我再买个充电器
。。。。。。

有流水线的持续:

客户：老板！我要手机、电池、充电器。。。。。。
老板：将东西一个一个递给客户

无流水线的持续:

- 仅当前面的响应已经收到，客户机发出新的请求
- 对每个引用对象一个RTT

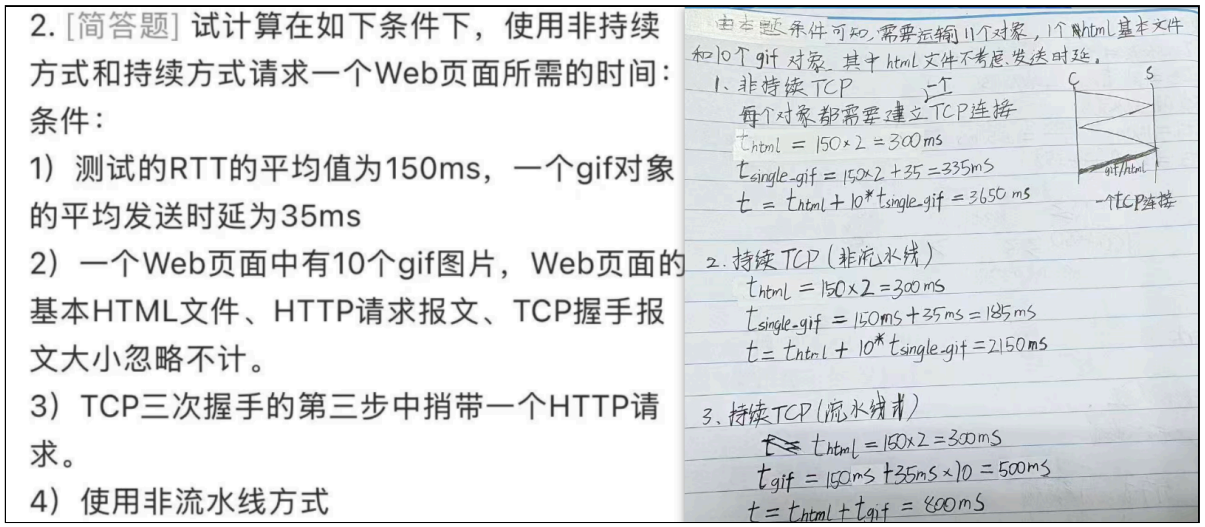
有流水线的持续:

- 在HTTP/1.1 为默认
- 只要客户机遇到一个引用对象，它发送请求
- 对于所有引用的对象花费一个RTT时间

非持续性连接每个对象的传输都要先建立TCP连接, 传输完成后直接关闭, 下一个对象还需要"三次握手"

持续性连接的传输, 建立一个TCP连接后, 后续对象都可以通过该链接有次序的进行运输

RTT 相关例题



2.2.3 HTTP报文格式

HTTP报文有两类:

1. HTTP请求报文;
2. HTTP响应报文

HTTP请求报文

第一行**请求行**有3个字段, `method URL version`:

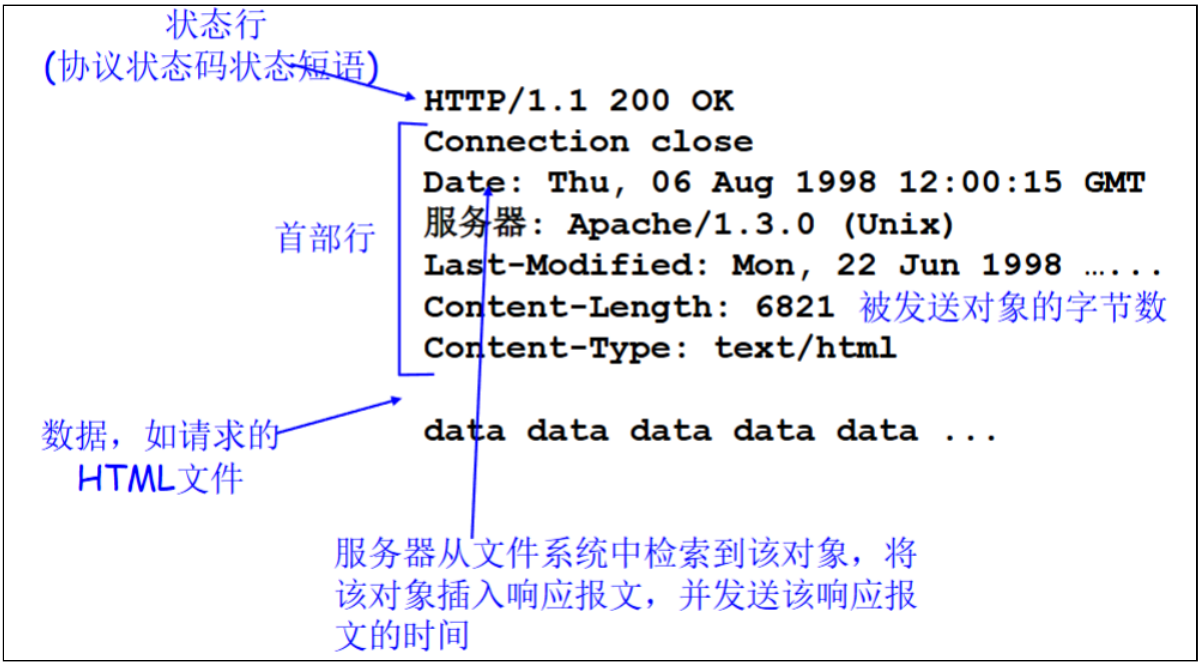
- `method`: 请求方法, 有GET, POST, HEAD, PUT, DELETE, OPTIONS, TRACE, CONNECT等
- `URL`: 资源定位符, 例如`/someDept/pic.gif`, 对象的路径名
- `version`: HTTP版本, 例如`HTTP/1.1`

method方法字段:

- **Get**: **获取资源**, 例如HTML文件, JPEG图片, Java小程序, 音频文件等...
- **Post**: **提交数据**, 例如表单数据(浏览器搜索), 上传文件等..
- **Head**: 服务器收到这样的报文后, 会用一个HTTP报文响应, 但不返回请求对象
- **Put**: **上载文件**, 例如修改HTML文件, 上传文件等...
- **Delete**: **删除资源**, 例如删除HTML文件, 上传文件等...

- ...

HTTP响应报文



状态行有3个字段: `version 状态码 相应状态信息`

- `version`: HTTP版本, 例如`HTTP/1.1`
- `状态码`: 例如200, 404, 500等
- `相应状态信息`: 例如`OK`(200), `Not Found`(404), `HTTP Version Not Supported`(401), `Internal Server Error`(500)等

首部行有一个`Last-Modified`字段, 用于表示资源的最后修改时间, 这一字段对于既可能在本地客户也可能在网络缓存服务器上的对象缓存来说非常重要

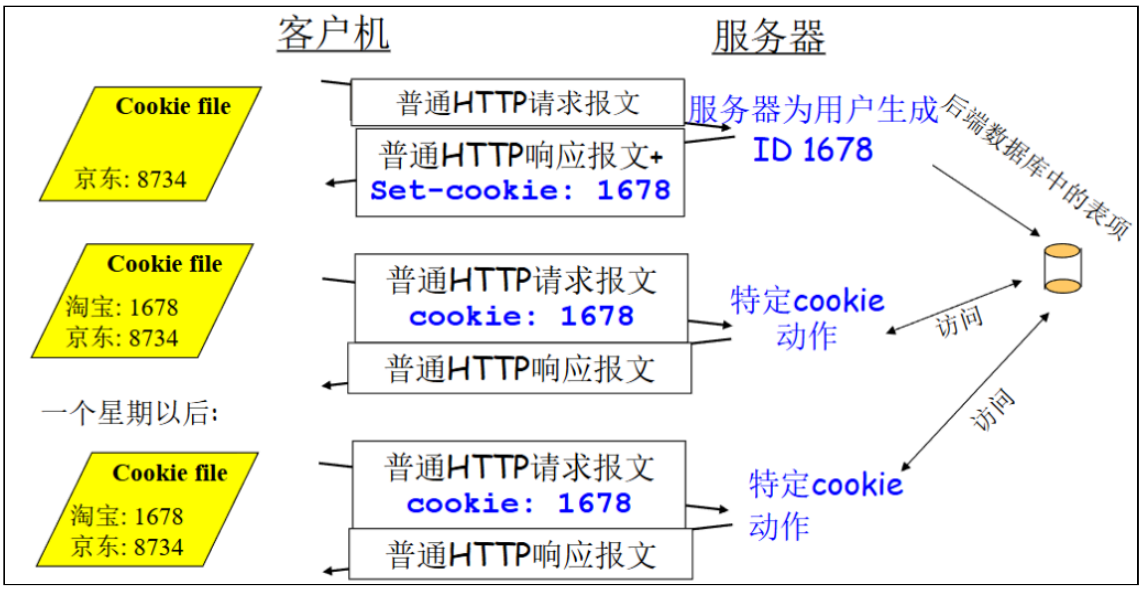
2.2.4 用户和服务器的交互--cookie

HTTP服务器是"无状态"的, 为了保持一些客户信息, 引入了**cookies**技术

cookies技术有四个组件:

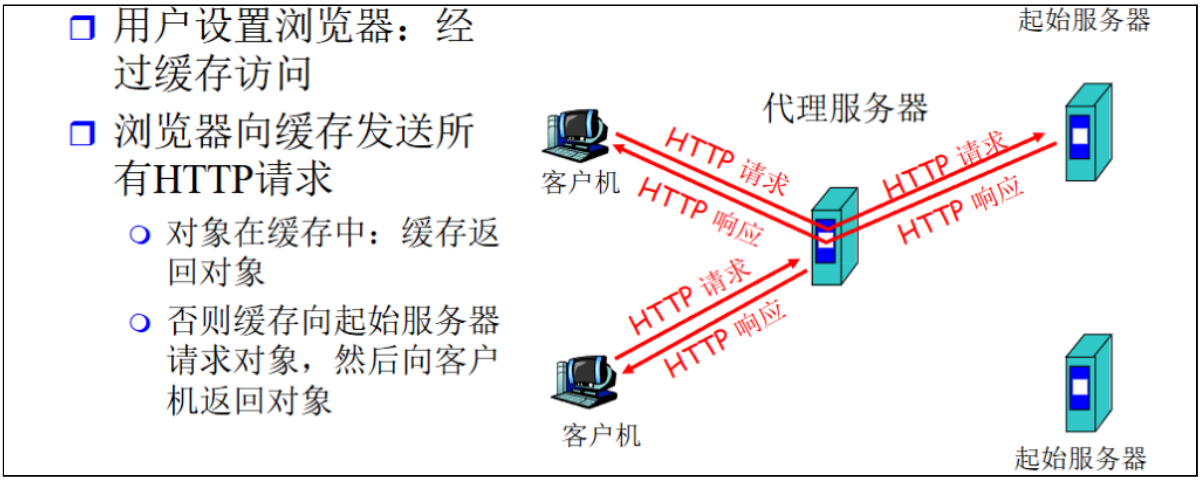
1. HTTP响应报文的cookie首部行
2. 客户端浏览器的cookie存储
3. HTTP请求报文的cookie首部行(客户端浏览器有存储时)
4. web站点的后端数据库

5.



2.2.5 Web缓存

Web缓存(Web Caching)也叫**代理服务器(Proxy Server)**, 他能够代替 *初始服务器* 来满足来自客户的HTTP请求



Web缓存的作用

1. **减少客户机请求的响应时间**
   客户与代理服务器之间的瓶颈带宽一般要远高于客户与初始服务器之间的瓶颈带宽
2. **减小机构访问链路的流量**
   Web缓存中有想要的对象时, 就无需从初始服务器获取, 从而减小机构访问链路的流量

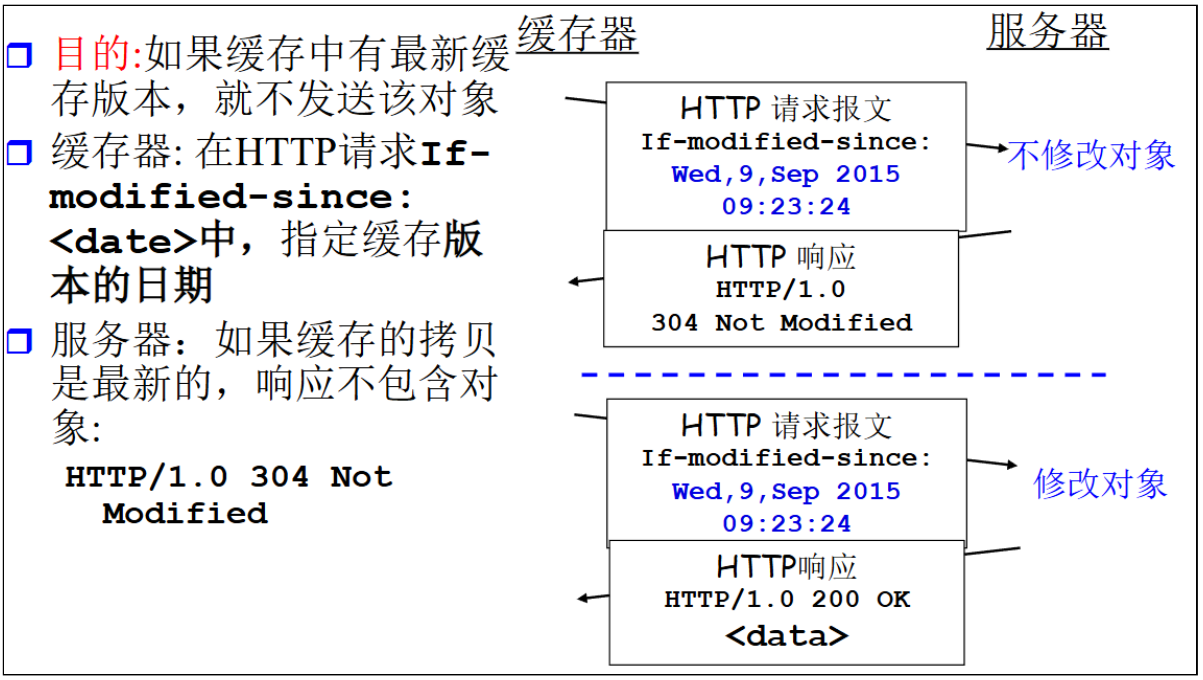
当接入链路流量强度很大时(接近1时, 排队时延会很大), 有两种方法进行改善:

1. 增加接入链路的带宽
   通常这样能降低流量强度, 但带宽升级费用较高
2. 在局域网中部署Web缓存服务器
   这样也能降低流量强度, 更加降低客户机请求的响应时间

条件GET

存放在代理服务器的对象副本可能是陈旧的, 即保存在初始服务器中的这个对象可能已经被改过, 但是代理服务器的副本缓存并不知道
HTTP协议通过**条件GET**这个机制来解决这一问题

一个**条件GET**请求报文的特征就是: 该报文包含**一个** `If-Modified-Since` **首部行**, 该行包含了代理服务器中对象的最后修改时间
这样的请求报文会被发给初始服务器, 用于检查是否需要更新代理服务器中的对象副本



如果条件GET的响应报文的状态行**有** `304 Not Modified` **的字段**, 则说明代理服务器中的对象副本是最新的, 代理服务器就可以直接将该对象交付给客户机

2.3 FTP

FTP(File Transfer Protocol) 是**文件传输协议**, 用于在网络上进行文件的上传和下载
FTP使用**TCP**作为运输协议, FTP服务器监听**TCP端口21**, 等待客户机的连接请求
FTP使用**两个TCP连接**来传输数据:

1. **控制连接(control connection)**: 用于客户机和服务器之间的命令和响应的交换 该连接建立在**端口21**
2. **数据连接(data connection)**: 用于客户机和服务器之间的文件传输
   FTP服务器在**TCP端口20**上监听**数据连接请求**, 当客户机需要传输文件时, 就会建立一个数据连接来进行文件的传输

2.3.1 FTP 命令

命令示例:

- 经控制信道以ASCII 文本发送
- **USER *username***
- **PASS *password***
- **LIST**返回当前目录中的文件列表
- **RETR filename**获取(get)文件
- **STOR filename** 存储 (puts)文件到远程主机

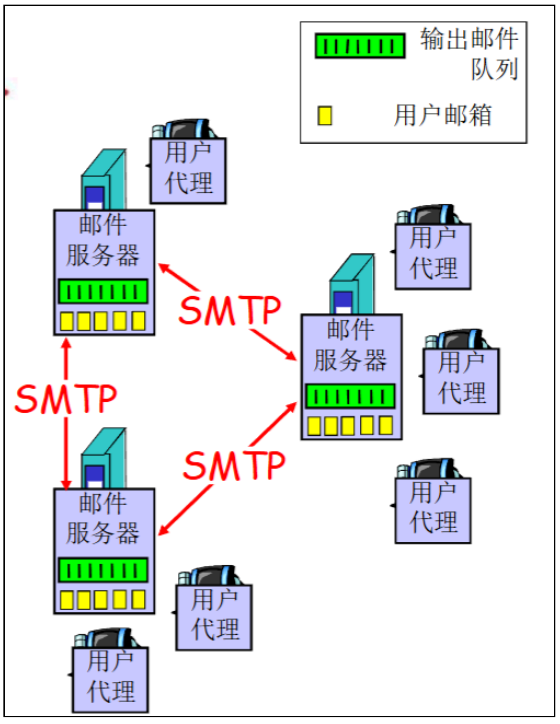
返回码示例:

- 状态码和短语(如在HTTP中的那样)
- **331 Username OK, password required**
- **125 data connection already open; transfer starting**
- **425 Can't open data connection**
- **452 Error writing file**

2.4 电子邮件

因特网电子邮件系统有三个主要组件:

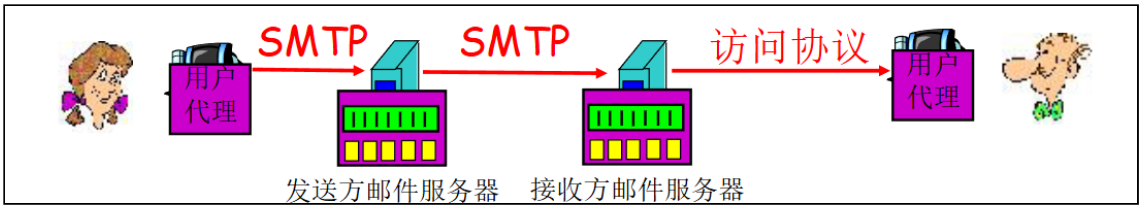
1. **用户代理(user agent)**: 电子邮件客户端程序, 例如Outlook, Gmail等
2. **邮件服务器(mail server)**: 电子邮件服务器程序, 例如SMTP服务器
3. **简单邮件传输协议(Simple Mail Transfer Protocol, SMTP)**: 电子邮件传输协议, 用于在邮件服务器之间传输电子邮件



一个典型的邮件发送过程:

1. 用户**在用户代理中撰写邮件**, **用户代理**将邮件发送给SMTP服务器
2. SMTP服务器将邮件转发给**目的地SMTP服务器**
3. 目的地SMTP服务器将邮件存储在**邮箱中**

4. 接收方的用户代理从SMTP服务器的该用户邮箱(用户名和口令鉴别)中取出邮件, 并显示给用户



2.4.1 SMTP协议

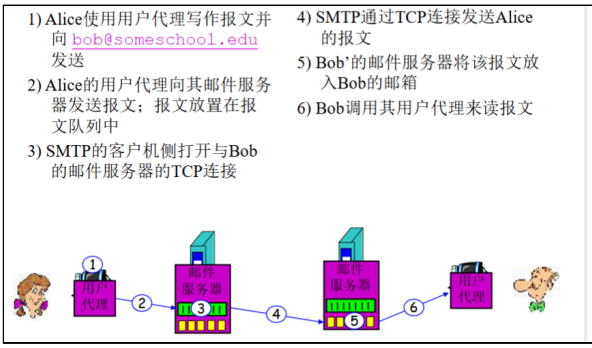
SMTP是电子邮件的主要应用层协议, 使用TCP作为运输协议, SMTP服务器监听**TCP端口25**来等待客户机的连接请求

SMTP传输电子邮件的过程分为三个阶段:

1. 握手(TCP三次握手)
2. 报文的传输(可以传送多个邮件)
3. 关闭(QUIT)

命令/响应的交互:

以下左图为一个邮件传送的场景实例, 右图是SMTP建立TCP连接**之后**, SMTP的发送服务器(C)和接收服务器(S)之间的命令/响应交互:




```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250  Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

基本是C发送规定格式的命令(HELO, MAIL FROM, RCPT TO, DATA, QUIT), S给出相应的响应

邮件报文格式

SMTP是交换邮件报文的协议
邮件报文的格式有**首部**和**主体**两部分:

1. 首部行, 如:
   - `To:`
   - `From:`
   - `Subject:`
2. 主体行:
   报文, 均为ASCII字符
3. 两部分之间用一个空白行间隔

SMTP vs HTTP

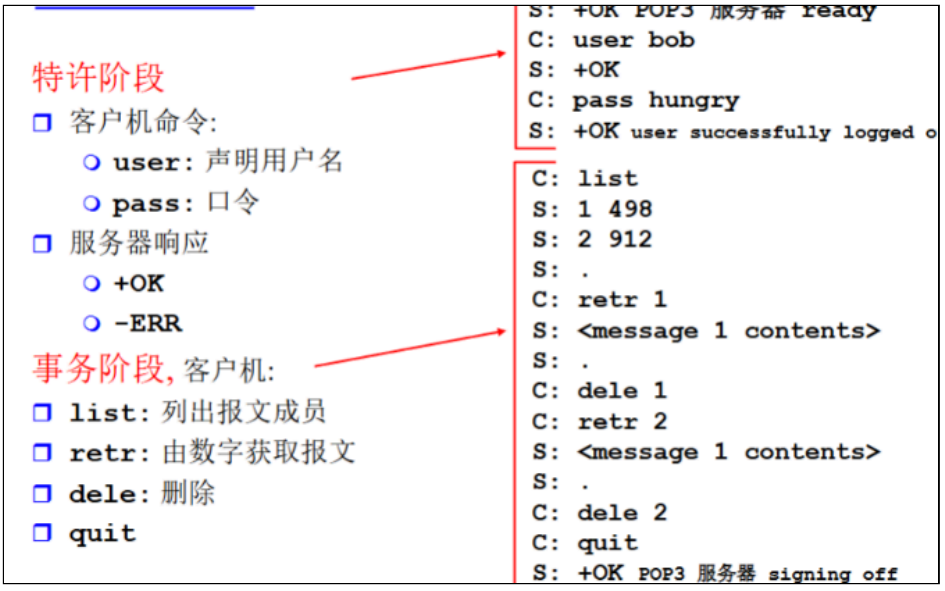
1. HTTP大致是"拉协议", SMTP是"推协议"
2. SMTP要求每个报文采用7比特ASCII吗格式
3. HTTP会把每个对象封装在其自己的响应报文中, SMTP则把所有对象封装在一个邮件报文中

2.4.3 邮件访问协议

接收方用户代理向他的ISP的邮件服务器取邮件是一个"拉报文"操作, 但是SMTP是一个"推报文"协议, 这个过程需要引入一些特殊的**邮件访问协议**

1. **POP3(邮局协议3)**
2. **IMAP(互联网邮件访问协议)**

POP3协议



2.5 DNS: 域名系统

DNS(Domain Name System) 是**域名系统**:

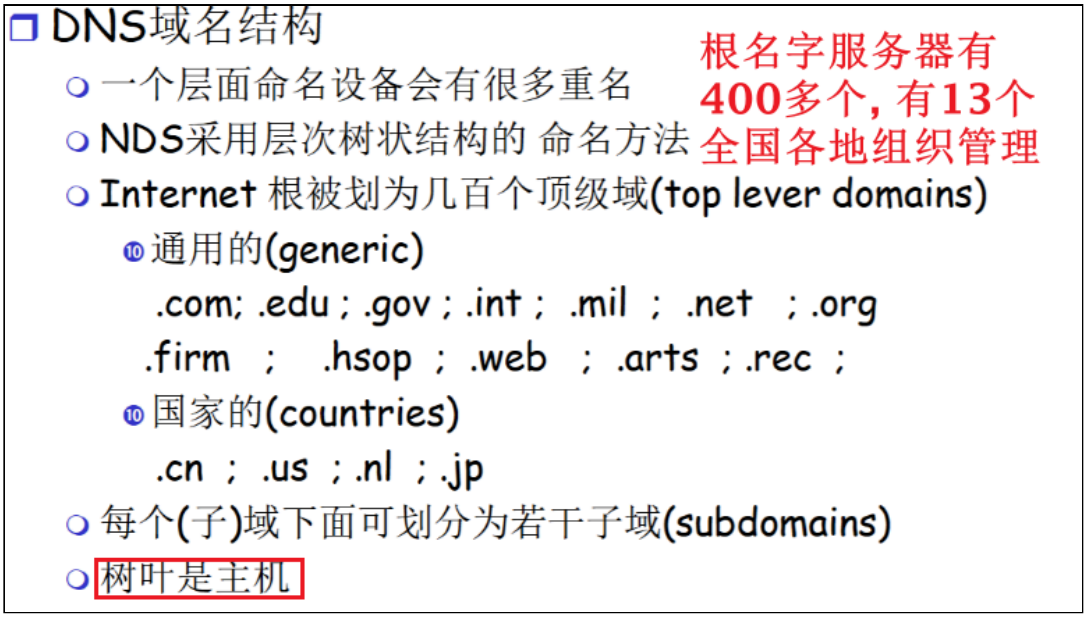
1. 一个有由分层的DNS服务器实现的**分布式数据库**
2. 一个**应用层协议**, 使得主机能够查询该分布式数据库来获取主机名和IP地址之间的映射关系

为了有意义, 好记, 主机通常使用**主机名(hostname)**而不是IP地址来标识, 并且这样的命名要是**层次化**的, 例如`www.someschool.edu.cn`, 另外网络传输需要IP地址, 因此就有了DNS, 其**主要服务就是将主机名转换为IP地址**, 并且DNS服务器是分布式的, 以层次化的方式组织成一个树形结构, 每个DNS服务器负责管理一个域, 例如`edu.cn`域, `someschool.edu.cn`域等

其他服务 -- 负载分配

一些繁忙的站点要被冗余分布在多台服务器上, 一个域名对应的是一个**IP的值的集合**, 客户端发出这样的DNS请求时, DNS服务器就会返回一个IP地址, 该IP地址是从这个集合中**轮询**(每次请求**出队**一个, 出队的这一个重新排到**队尾**)选出的, 这样就实现了负载分配

2.5.1 DNS名字空间 -- 一颗 "树"



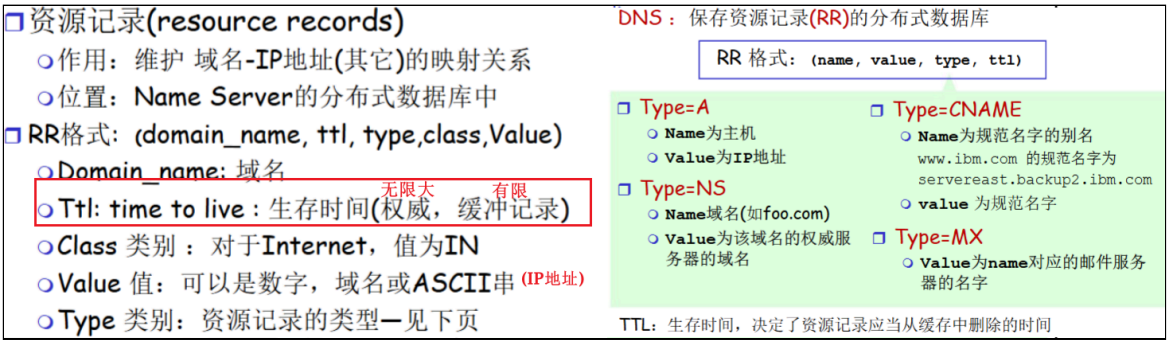
主机的域名是从叶子往上直到根
域的域名是从非叶子节点往上走到根

权威DNS服务器

名字空间会被分为不同的区域(zone), 每个区域由一个**权威DNS服务器**来管理, 权威DNS服务器记录的主机名和IP的映射关系是**权威的**, 对于没有缓存的主机名查询路径, **最终都会到达权威DNS服务器**来获取主机名和IP的映射关系

DNS记录: 资源记录(RR, resource record)

共同实现DNS分布式数据库的所有DNS服务器存储了**资源记录**:



比如一个RR的内容是:

```
www.someschool.edu.cn. 3600 IN A 128.119.245.
```

这里的`www.someschool.edu.cn.`是主机名, `3600`是TTL, `IN`表示Internet, `A`表示该记录是一个主机名和IP地址的映射关系(type), `128.119.245.`是对应的IP地址(value)

2.5.2 DNS名字服务器

除了**权威DNS服务器**之外, 还有**本地DNS服务器(local DNS server)**, **根DNS服务器(root DNS server)**, **顶级域DNS服务器(top-level domain DNS server, TLD)** 等

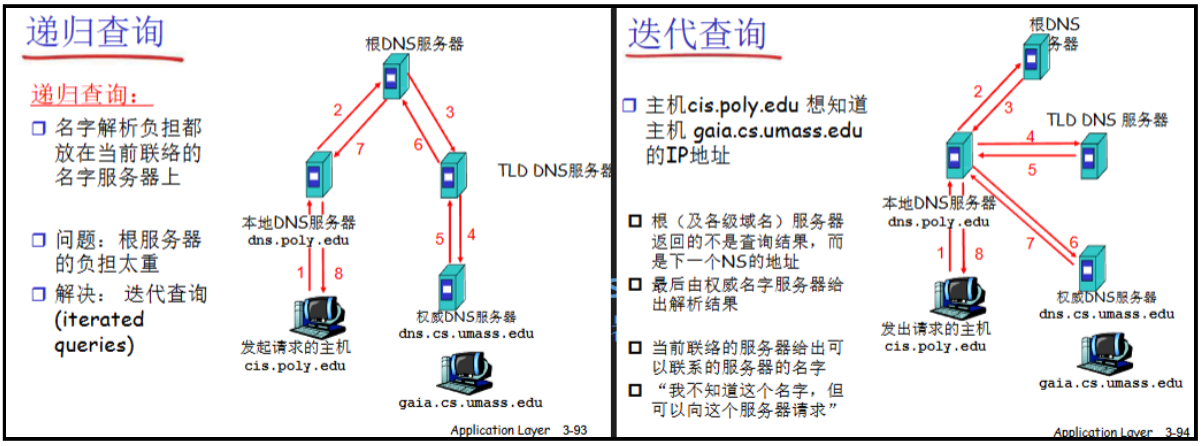
本地DNS服务器

当一个主机发起一个DNS查询时, 该查询会先送到其本地DNS服务器

1. 当查询名字位于本地DNS服务器的**缓存(caching)**中, 本地DNS服务器就直接返回查询结果
2. 当本地DNS不能解析该查询名字时, **就会开始沿着根DNS服务器 -> 顶级域DNS服务器 -> 权威DNS服务器**的查询路径来进行查询

第二种查询有两种方式:

2.5.3 两种查询过程



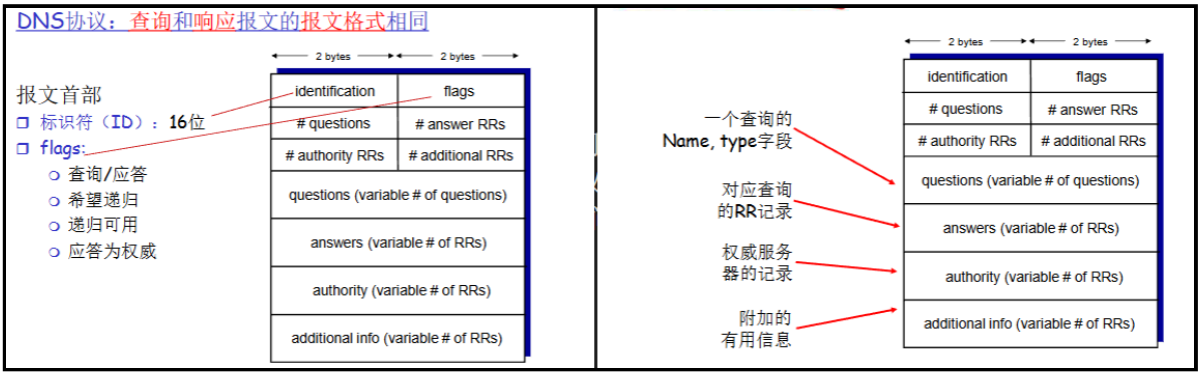
在**DNS**中, 为了提高性能, 一旦名字服务器学到了一个名字和IP地址的映射关系, 就会把该映射关系保存在本地DNS服务器的**缓存**中

- 根服务器通常都在本地服务器中缓存着根服务器的IP地址, 因此根服务器不会经常被访问

缓存是为了提高效率, 但是缓存不能一直存在, 要有一个**生存时间(TTL, time to live)**, 当TTL过期(默认为两天)时, 该缓存就会被删除, 需要重新查询

2.5.4 DNS协议, 报文

DNS协议使用UDP作为运输协议(不需要握手), DNS服务器监听**UDP端口53**来等待客户机的连接请求
DNS的报文格式如下:



2.5.5 DNS攻击

2.5.6 DNS的维护

2.6 P2P文件共享

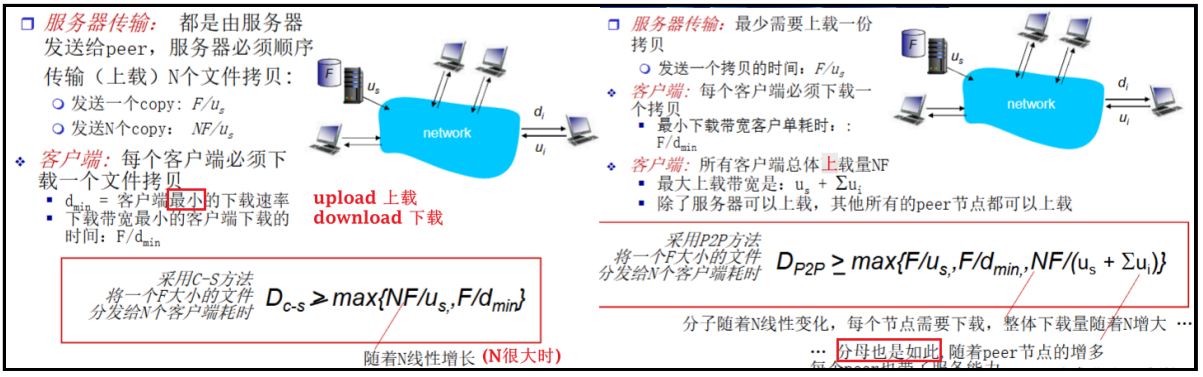
关于P2P的文件共享分发, 大致可以分为两种, 一种是**非结构化P2P文件共享**, 另一种是**DHT结构化P2P文件共享**

1. **非结构化P2P**中, peer节点之间的连接(yverlay)是任意的, 有大致三种形式
    - 集中化目录: Napster, 用一个服务器维持Peer List, 需要是进行查询, 然后建立P2P文件传输. 会有单点故障的问题

- 完全分布式: Gnutella, 没有服务器, 通过**洪泛查询**来进行查询, 但是效率较低
- 混合型: KaZaA, 可以理解成Napster和Gnutella的混合体, 分为很多小组, 每个小组的组长之间采用Naspter, 一个小组内的peer之间采用Gnutella

2. **DHT结构化P2P**中, peer节点之间的连接是有结构的, 不如构成一个环形的或者是树形的overlay, 并且通过**哈希函数**来进行查询, 例如Chord协议, 通过哈希函数把文件名映射成一个ID, 通过哈希函数把peer节点的IP地址映射成一个ID, 通过这些ID来进行查询, 这样效率较高

P2P vs. CS 文件传输



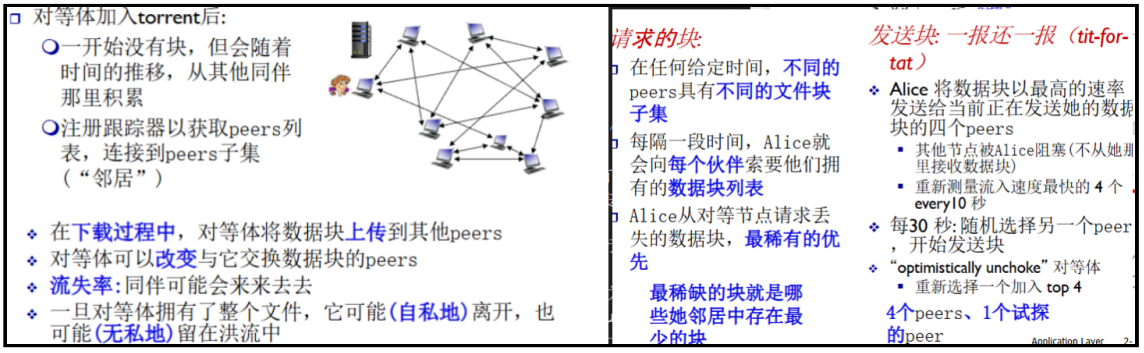
2.6.1 BitTorrent协议

接下来重点讲一下一个非结构化P2P文件共享的实例, **BitTorrent**协议
在**BitTorrent**协议中, 有三个重要的机制:

1. **稀缺性优先(Rarest First)**, 即优先从对等节点请求那些稀缺的文件块, 最稀缺的块就是那些他邻居中存在最少的块, 这样能够维持洪流的健康, 不会受peer节点的加入和离开而受到太大的影响.
2. **一报换一报(tit-for-tat)**, 即并不是雨露均沾, 而是优先选择对他传输速率最高的四个peers来发送块.
3. **乐观疏通(optimistic unchoking)**, 即定期选择一些之前没有与之交换数据的peers进行数据交换, 以发现更好的下载源, 更新最好的四人名单; 这样就有可能给一些刚加入洪流的小白提供第一桶金, 是他们后面有能力加入到Torrent的集体利益提升中去.

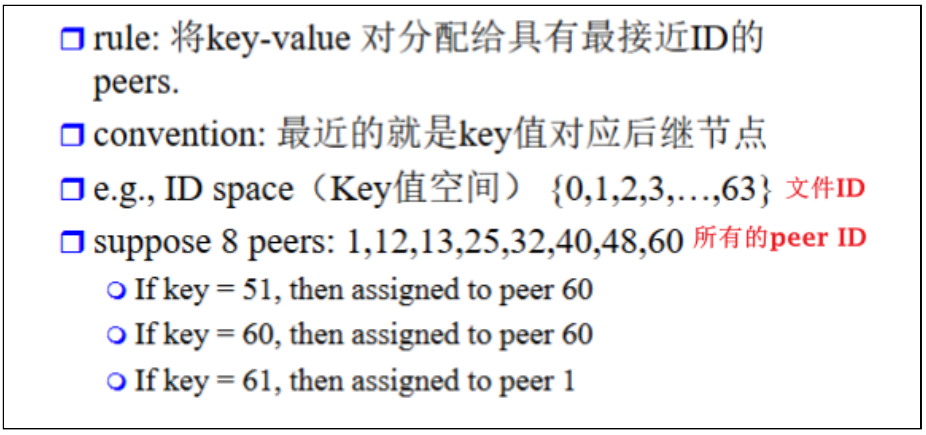
BitTorrent的基本前提:

1. 追踪器Tracker: 记录着维持某个文件的peer列表
2. 洪流Torrent: 参与一个特定的文件分发的所有peers集合
3. 新手加入torrent是, 会从tracker获得参与的对等方节点信息
4. 洪流里的每个节点都有一个**bit map**, 用于映射自己对文件的每个块的拥有情况.

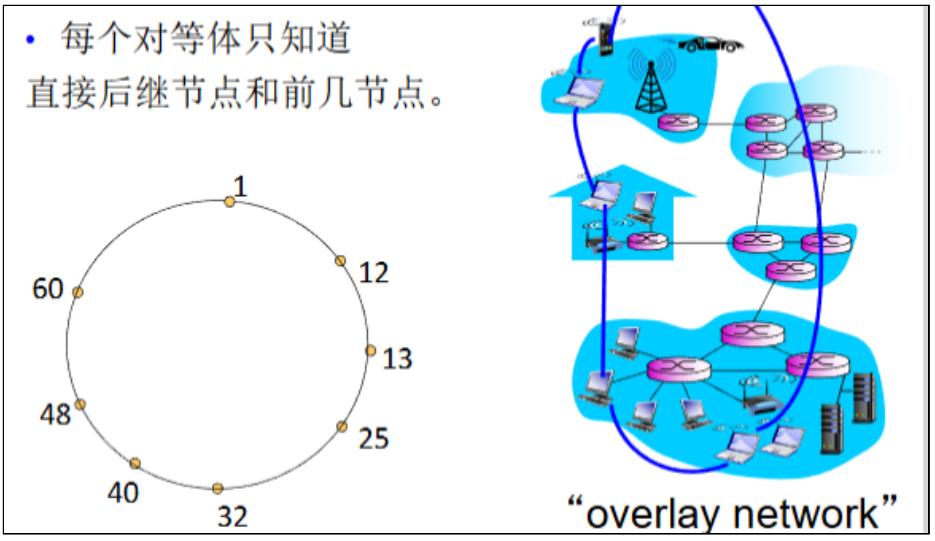


2.6.2 Distributed Hash Table (DHT)

无论是peer还是文件块, 都会**通过Hash算法变成所谓的ID**
DHT的世界里, 没有什么中心服务器, 整个DHT就是一个巨大的查询数据库, 每个节点负责存储某个文件他的ID和文件所在的位置, 即存的是key-value
在DHT的结构中, key-value的分配规则是: **将key-value对分配给具有最接近ID的peers.**



每次一个peer想要获得某个文件, 他就会根据改文件的ID去找到那个具有相应key-value的peer, 向他获得文件位置(可能是某个peer的IP), 然后再去文件位置获得文件内容



最经典的就是chord环形DHT结构

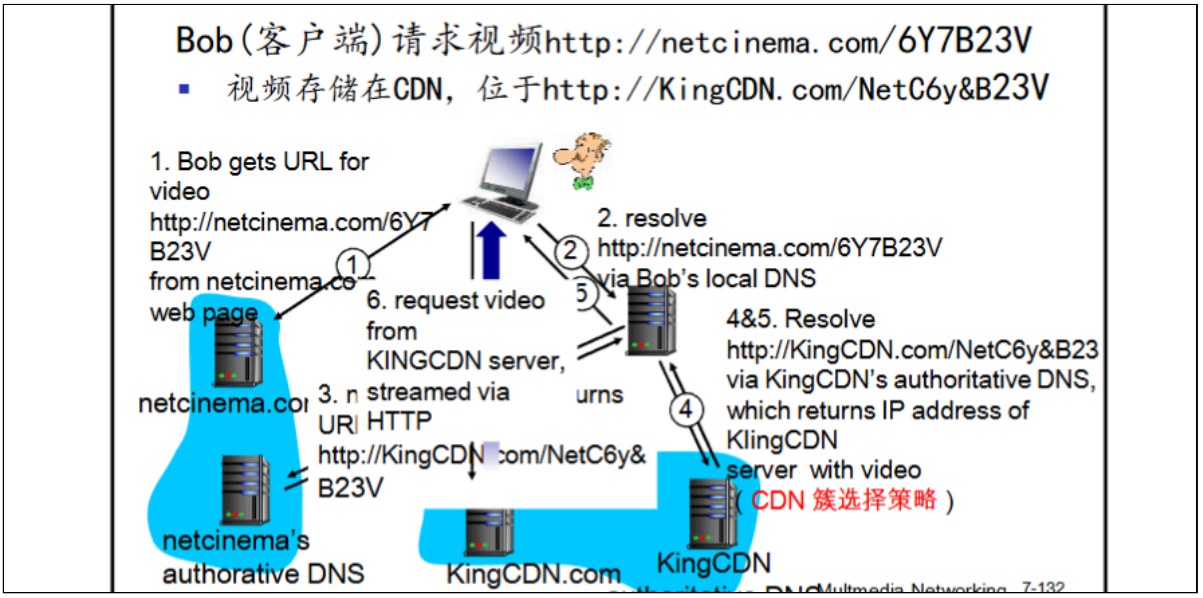
2.7 CDN

最常见的网络视频点播场景是: **流化服务**
即, 用户即点即看, 直接播放, 传多少数据, 播放多少, 并不需要完整下来之后才能开始看.
常见的流化服务是**DASH**

在这种流化服务下, 会有这样的挑战: **服务器如何通过网络向上百万用户同时流化视频内容?**
直接的想法是不断升级服务器, 制造一个超级服务器, 但是会有如下问题:

1. 服务器到客户端的路径上跳数较多, 瓶颈链路的带宽小导致停顿
2. 网络中同时充斥着同一视频的多个拷贝, 资源浪费
3. 单点故障

因此就有了**内容分发网络(Content Distribution Network, CDN)**的概念, 其核心思想就是**将内容分发到离用户更近的地方**, 这样就能提高效率, 减少停顿, 避免单点故障
CDN运营商会在很多地方部署**缓存节点**, 即缓存服务器, ISP向CDN服务商购买这些节点后, 这些缓存服务器会从原始服务器(ISP运营商的源服务器)上获取内容的副本, 当用户请求内容时, 就会从离用户最近的缓存服务器上获取内容, 这样就是能够加速ISP的流化服务
用户在进行点播时, 客户端会先按DASH那样获得告示文件, 通过告示文件获悉最近的最快的缓存服务器(也可以通过域名解析重定向), 然后从该服务器上获取内容, 这样就能加速流化服务
以下是一个简单的CDN的示例:

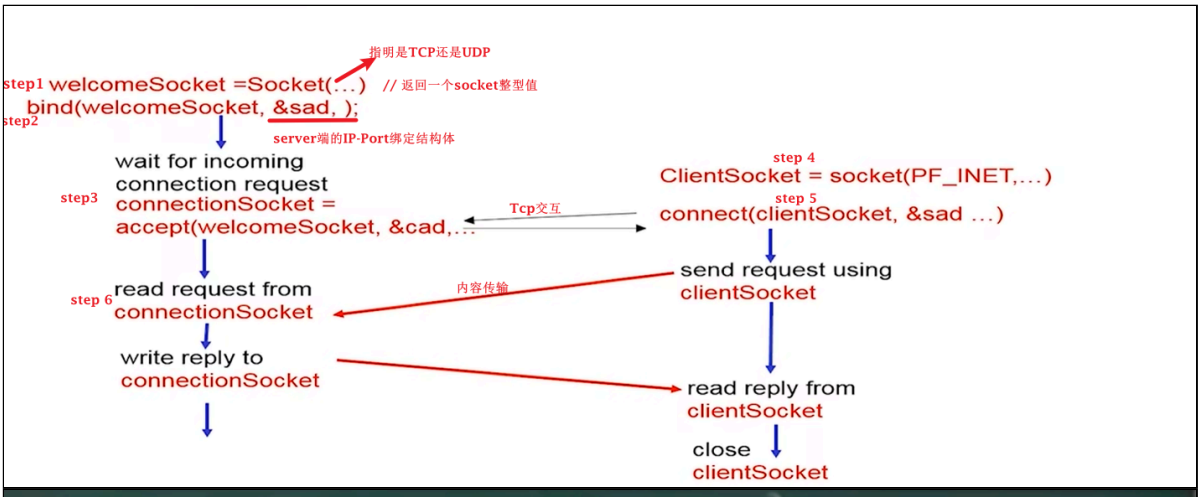


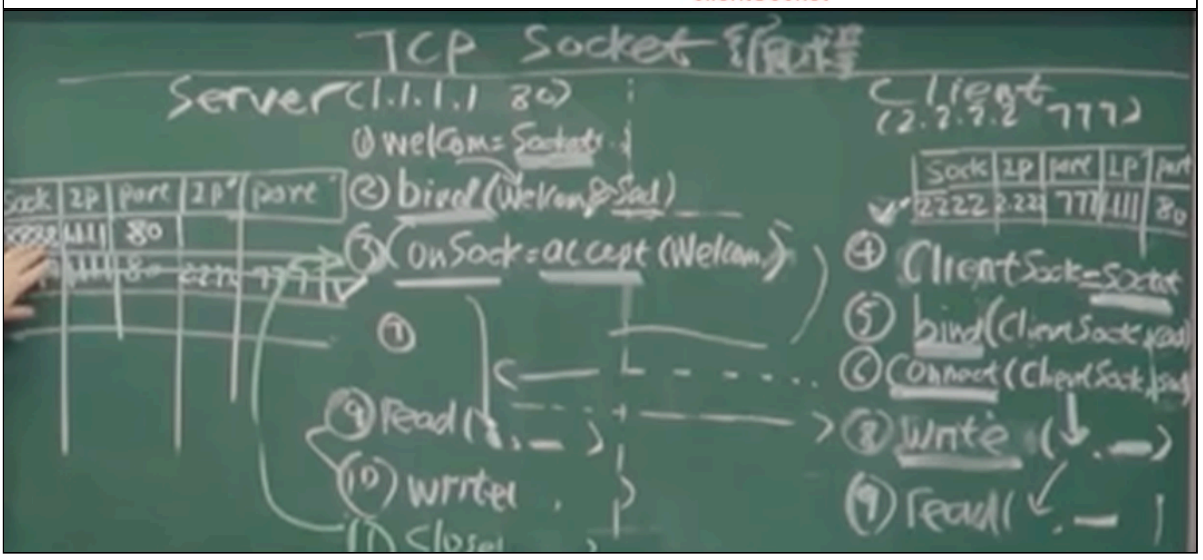
2.8 套接字编程

应用进程使用传输层提供的服务才能交换报文, 实现这一功能的接口就是**套接字(socket)**, 套接字是应用层和运输层之间的接口, 也是应用程序编程接口(API)
socket可以理解成是分布式应用进程之间的门，传输层协议提供的端到端的服务接口.

2.8.1 TCP套接字编程

TCP传输的过程可以用一下伪代码来表示.





用文字表述TCP连接的过程:

服务器首先运行，等待连接建立

1：服务器进程必须先处于运行状态

- 创建欢迎socket welcome socket=...
- 和本地端口捆绑 bind(...)
- 在欢迎socket上阻塞式等待接收用户的连接 connectsocket=connect(...)

客户端主动和服务器建立连接：

2：创建客户端本地套接字（隐式捆绑到本地port）clientsocket = socket(...)

- 指定服务器进程的IP地址和端口号，与服务器进程连接

4：连接API调用有效时，客户端P与服务器建立了TCP连接

3：当与客户端连接请求到来时

- 服务器接受来自用户端的请求，解除阻塞式等待，返回一个新的socket（与欢迎socket不一样），与客户端通信
  - 允许服务器与多个客户端通信
  - 使用源IP和源端口来区分不同的客户端

从应用程序的角度

TCP在客户端和服务器进程之间提供了可靠的、字节流（管道）服务

2.8.2 UDP套接字编程

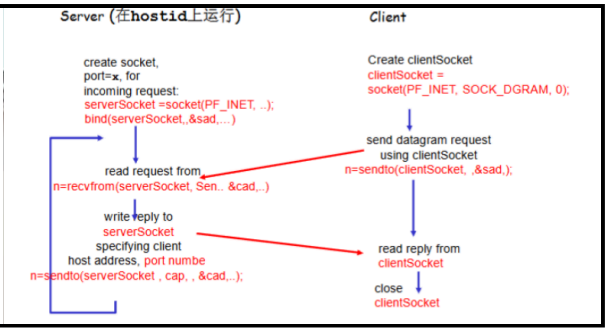
UDP: 在客户端和服务器之间没有连接

- 没有握手
- 发送端在每一个报文中明确地指定目标的IP地址和端口号
- 服务器必须从收到的分组中提取出发送端的IP地址和端口号

UDP: 传送的数据可能乱序，也可能丢失

进程视角看UDP服务

UDP 为客户端和服务器提供不可靠的字节组的传送服务



第三章：传输层

- 3.1 传输层概述
  - 3.1.1 传输层 vs. 网络层
- 3.2 多路复用和解复用
  - 3.2.1 UDP的多路复用和解复用
  - 3.2.2 TCP的多路复用和解复用
  - 3.2.3 UDP vs TCP
- 3.3 无连接传输: UDP
  - 3.3.1 UDP校验和
- 3.4 可靠数据传输(rdt)的原理
  - 3.4.1 rdt 1.0
  - 3.4.2 rdt 2.0
  - 3.4.3 rdt 3.0
  - 3.4.4 流水线协议
- 3.5 面向连接的传输: TCP
  - 3.5.1 段结构
  - 3.5.2 可靠数据传递
  - 3.5.3 流量控制
  - 3.5.4 TCP连接管理
- 3.6 拥塞控制原理
  - 3.6.1 拥塞的原因
  - 3.6.2 拥塞的代价
  - 3.6.3 拥塞控制的方法
- 3.7 TCP 拥塞控制
  - 3.7.1 端到端拥塞感知
  - 3.7.3 TCP 拥塞控制 速率控制
  - 3.7.4 TCP 拥塞控制算法
  - 3.7.5 TCP 吞吐量
  - 3.7.6 TCP 具有公平性

3.1 传输层概述

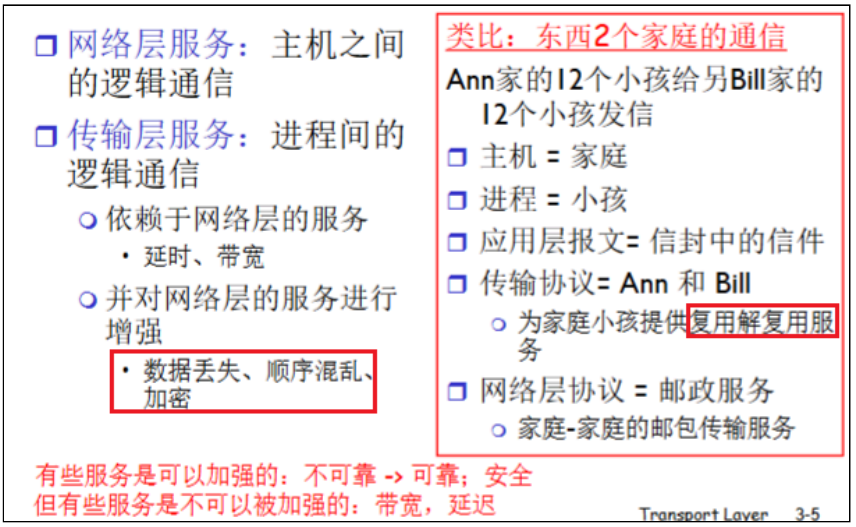
传输层的定位是为**运行在不同主机上的应用进程**提供**逻辑通信**服务

传输协议(TCP, UDP等)运行在端系统:

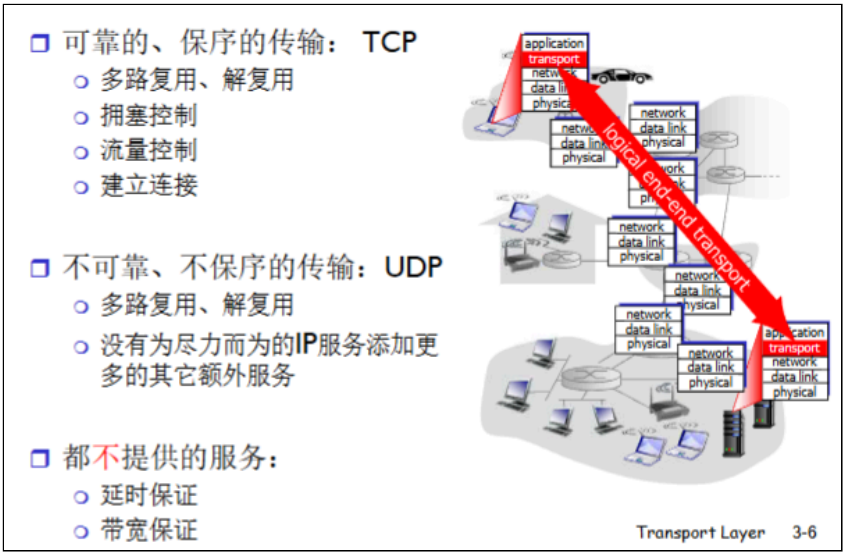
1. **发送方**: 蒋应用层的报文分成**报文段**, 并为每个报文段添加一个**传输层首部**(header), 形成**传输层数据报**(segment), 然后将数据报交给网络层(网络层是IP协议)
2. **接收方**: 从网络层接收数据报, 去掉传输层首部, 将报文段重组成应用层报文, 交给应用层

3.1.1 传输层 vs. 网络层

网络层提供的服务是**主机到主机的服务**, 传输层把网络层服务**细分为进程到进程的服务**
传输层的服务属于是**更加的优化网络层的服务品质**, 然后传给应用层



这里传输层带来的"加强服务"如下:



3.2 多路复用和解复用

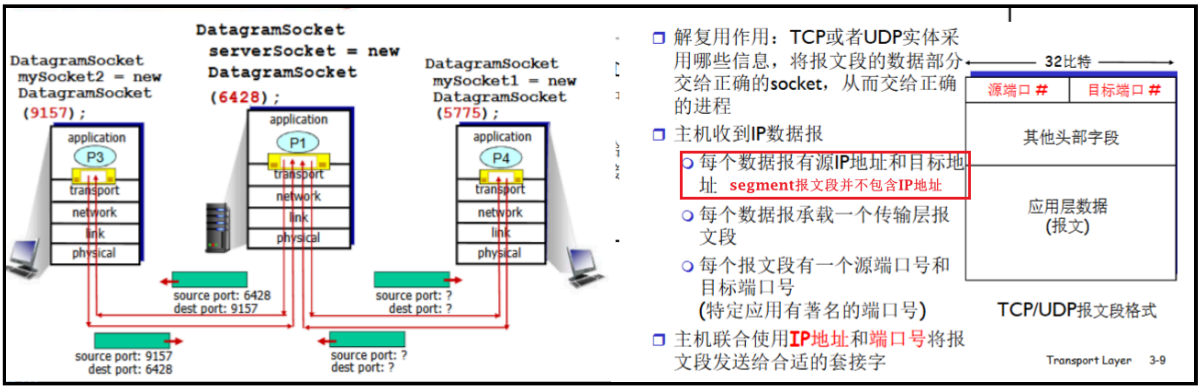
为了实现**进程到进程的通信**, 传输层需要使用**端口号(port number)**来标识不同的应用进程, 端口号是一个16位的整数, 范围是0~65535

1. **多路复用(multiplexing)**: **发送方**从多个套接字接收来自多个进程的报文, 根据套接字对应的IP地址和端口号等信息对每个端口对应的报文段**用头部加以封装**, 这些头部信息会用于以后的解复用(TCP和UDP的头部信息不太一样)
2. **解复用(demultiplexing)**: **接收方**根据报文段的**头部信息中的IP地址和端口号**将接收到的报文段发给正确的套接字和对应的应用进程

$$IP\text{数据报}(\text{网路层}) = IP\text{首部} + \text{传输层报文段}(\text{传输层}) = IP\text{首部} + \text{传输层首部}(\text{源端口目标端口}) + \text{应用层报文}$$

3.2.1 UDP的多路复用和解复用

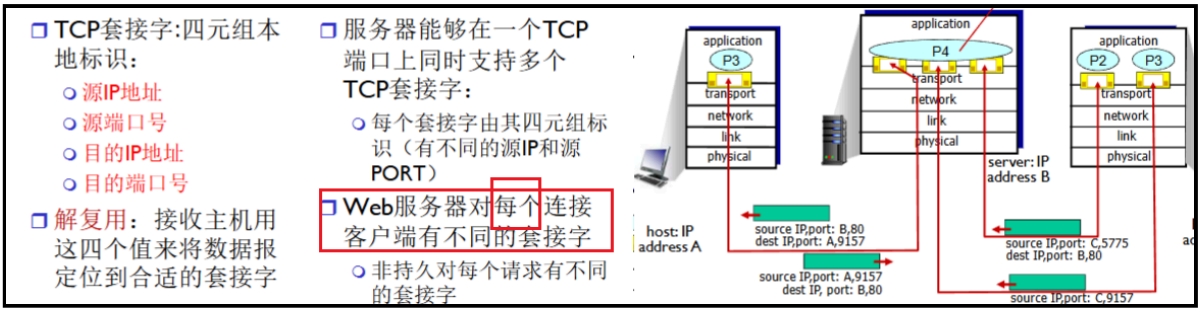
UDP的数据段首部包含**源端口号**和**目的端口号**;
UDP使用**二元组**来标识一个套接字, 二元组由**目的IP地址**和**目的端口号**组成;



当主机接收到UDP的段时, 会检查UDP点钟的目标端口号, 将**UDP段交给具备那个端口号的套接字**
即便UDP的报文段的源端口号不同(即来自不同的客户端进程), **只要目的端口号相同, 他们就会被交给同一个套接字**

3.2.2 TCP的多路复用和解复用

TCP使用**四元组**来标识一个套接字, 四元组由**源IP地址, 源端口号, 目的IP地址, 目的端口号**组成



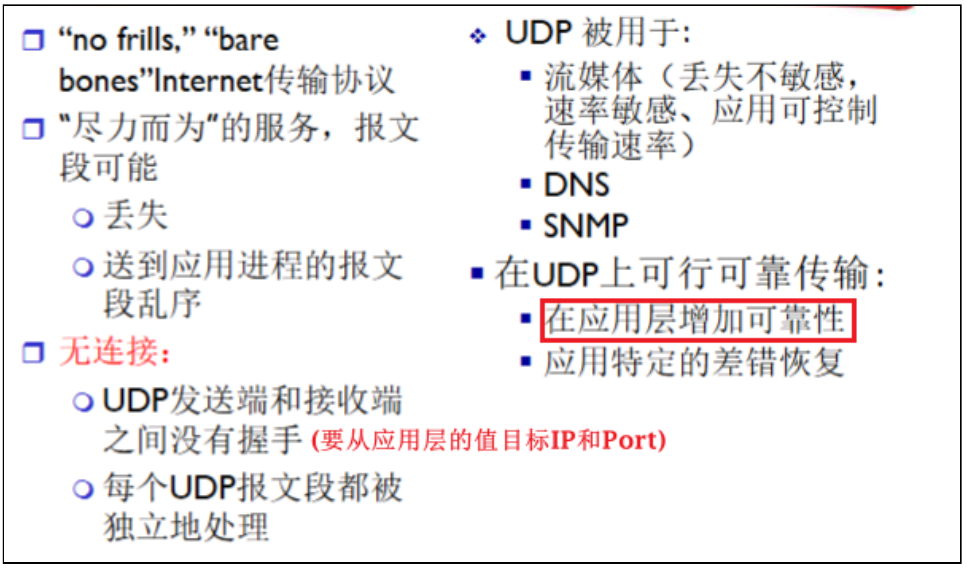
接收主机用这四个值来**将数据报定位到合适的套接字**, 服务器能够在一个TCP端口上同时支持多个TCP套接字, 每个套接字的区别是**源IP地址和源端口号不同**

3.2.3 UDP vs TCP

一个使用二元组一个使用四元组, TCP为了能够实现可靠的连接, 必须使用四元组来标识一个套接字, 以区分不同的连接; UDP不需要连接, 只需要使用二元组来标识一个套接字

3.3 无连接传输: UDP

UDP(User Datagram Protocol) 被称为"尽力而为"的传输协议, 因为他在网络层提供的IP服务的基础上, 并没有给出太多的服务质量提升.

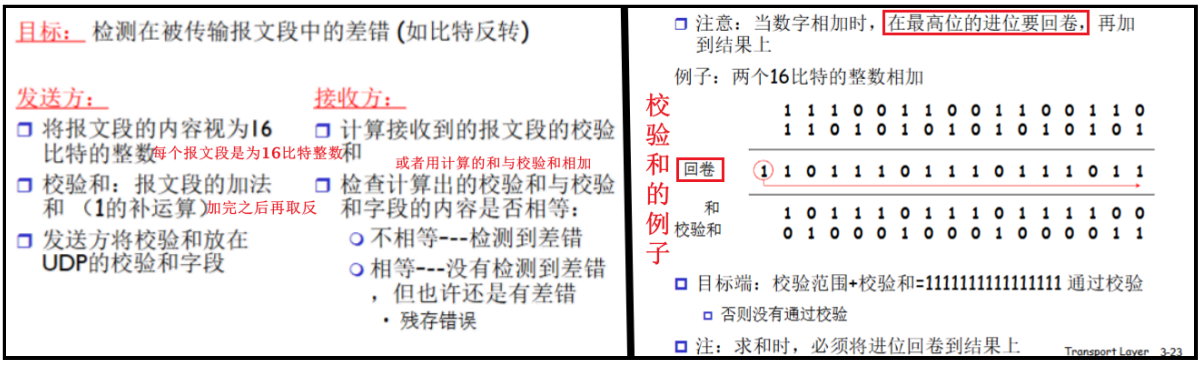


UDP报文段的首部包含**源端口号, 目的端口号, 长度, 校验和(差错检测码, EDC)**四个字段; 首部之后就是应用层的报文数据;

为什么要有UDP ?

1. **不建立连接**: 减少一部分时延
2. 简单: 在发送端和接收端没有连接状态
3. **报文段的头部很小**: 开销小
4. **无拥塞控制和流量控制**: UDP可以尽可能快的发送报文段, 不受网络拥塞和接收端处理能力的限制; TCP则需要进行拥塞控制和流量控制, 以避免网络拥塞和接收端过载

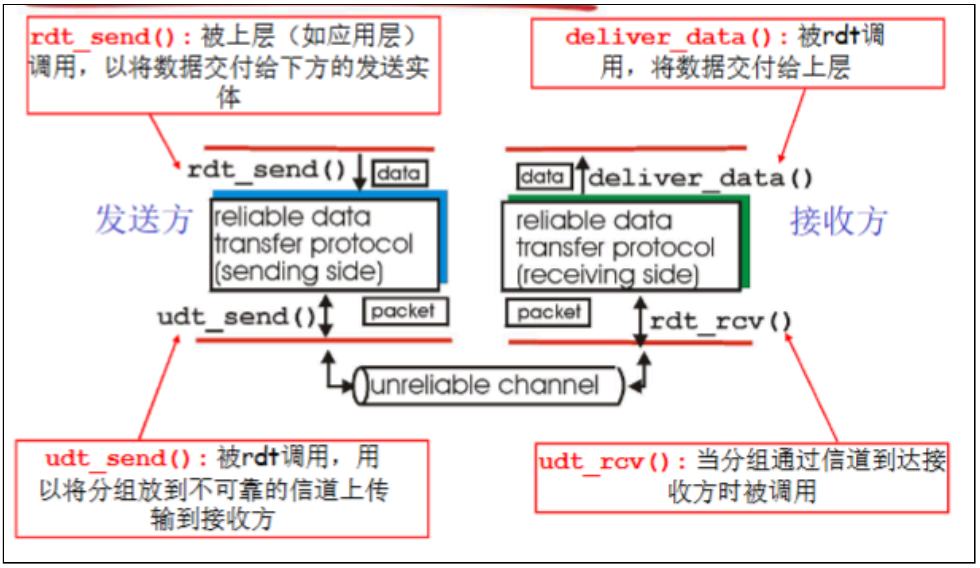
3.3.1 UDP校验和



3.4 可靠数据传输(rdt)的原理

> **可靠数据传输(reliable data transfer, rdt)**在应用层, 传输层和数据链路层都很**重要**

rdt协议的设计目标是**在不可靠的信道(比如比特差错, 分组丢失) 上实现可靠的数据传输**, 来自下层提供的服务是不可靠的, 但是rdt要向上层提供可靠的服务.



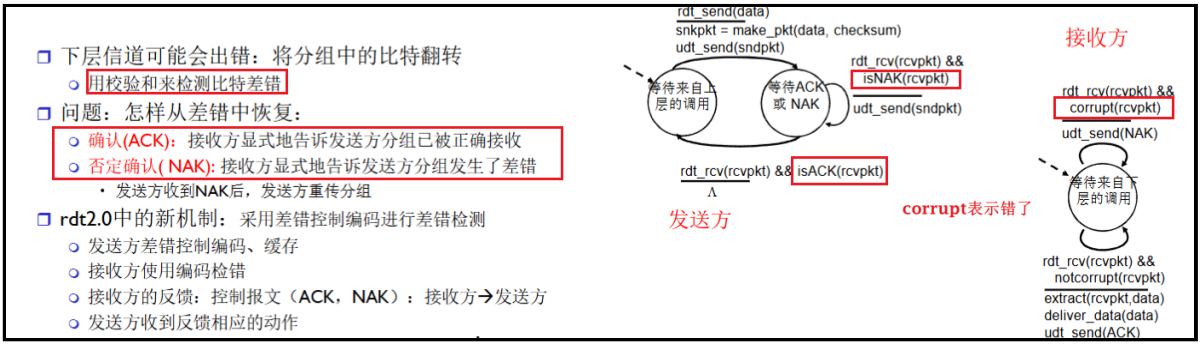
接下来我们将通过一个rdt逐步进化的过程来理解rdt协议的设计原理, 下层是不可靠的信道, 我们可以先假设他是可靠的, 然后逐步来理解rdt在信道变得越来越不可靠时要怎么去升级以维持对上层的可靠服务.
每一步升级我们都用有限状态机(FSM)来描述rdt协议的行为

3.4.1 rdt 1.0

信道可靠, 没有比特差错和分组丢失

3.4.2 rdt 2.0

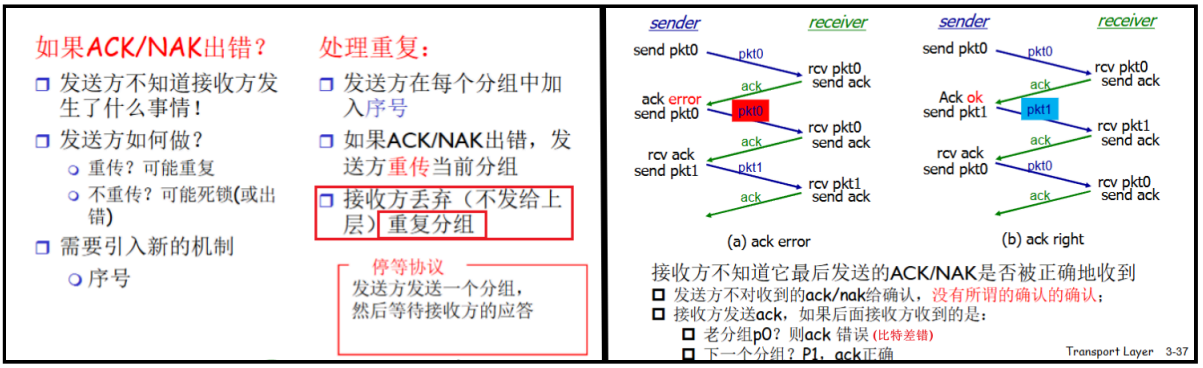
信道不可靠, 有比特差错



要注意, 传送的分组数据可能会有比特差错, 那么**NAK, ACK分组也可能会有比特差错**, 这就是rdt2.0的致命缺陷.

3.4.2.1 rdt 2.1

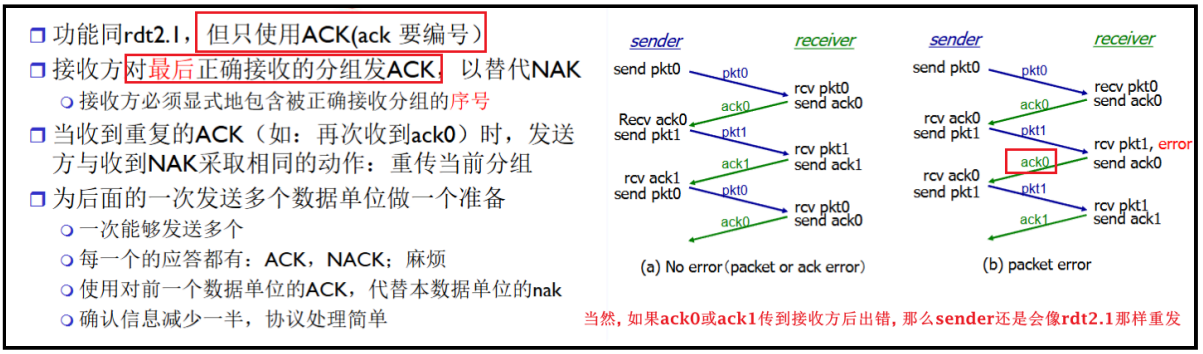
解决2.0的NAK, ACK分组可能会有比特差错的问题, 通过**添加序列号**来解决这个问题, 这样就可以区分出ACK和NAK分组的比特差错了.



实际上的序列号只用0和1就够了, 因为我们只需要区分出当前分组和上一个分组就可以了, 这样就可以解决ACK和NAK分组的比特差错问题了

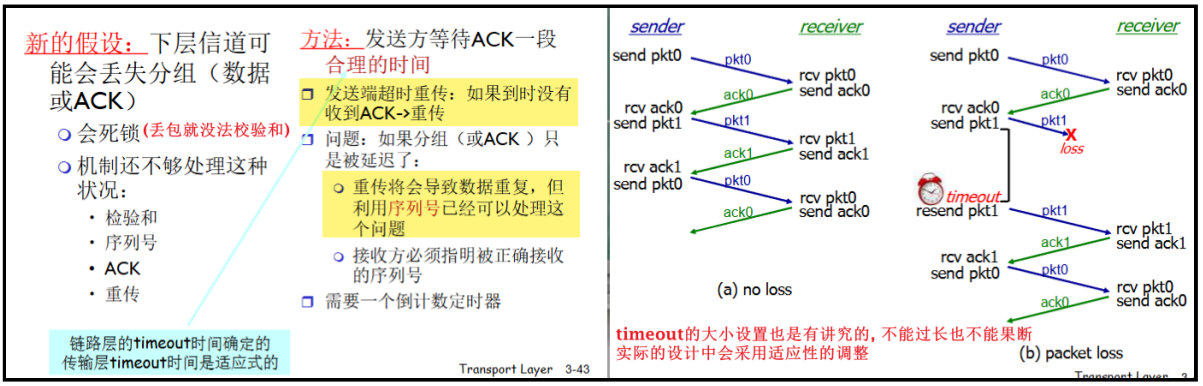
3.4.2.2 rdt 2.2

取消掉NAK分组, 只使用ACK分组来确认分组的正确接收, 这样就可以简化协议了, 但是需要在ACK分组中添加一个序列号来区分当前分组和上一个分组

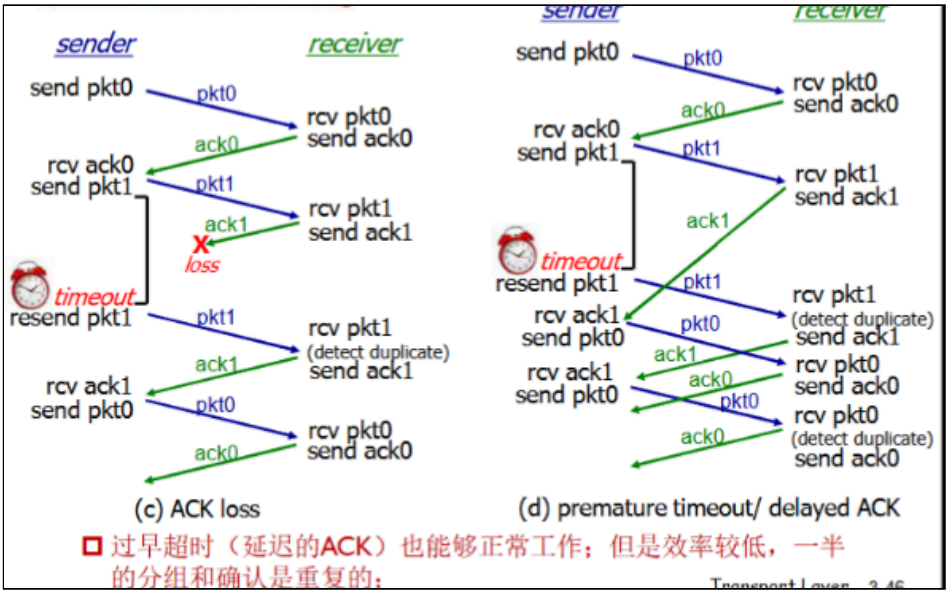


3.4.3 rdt 3.0

信道不可靠, 有比特差错, 有分组丢失(路由器缓存队列满了)



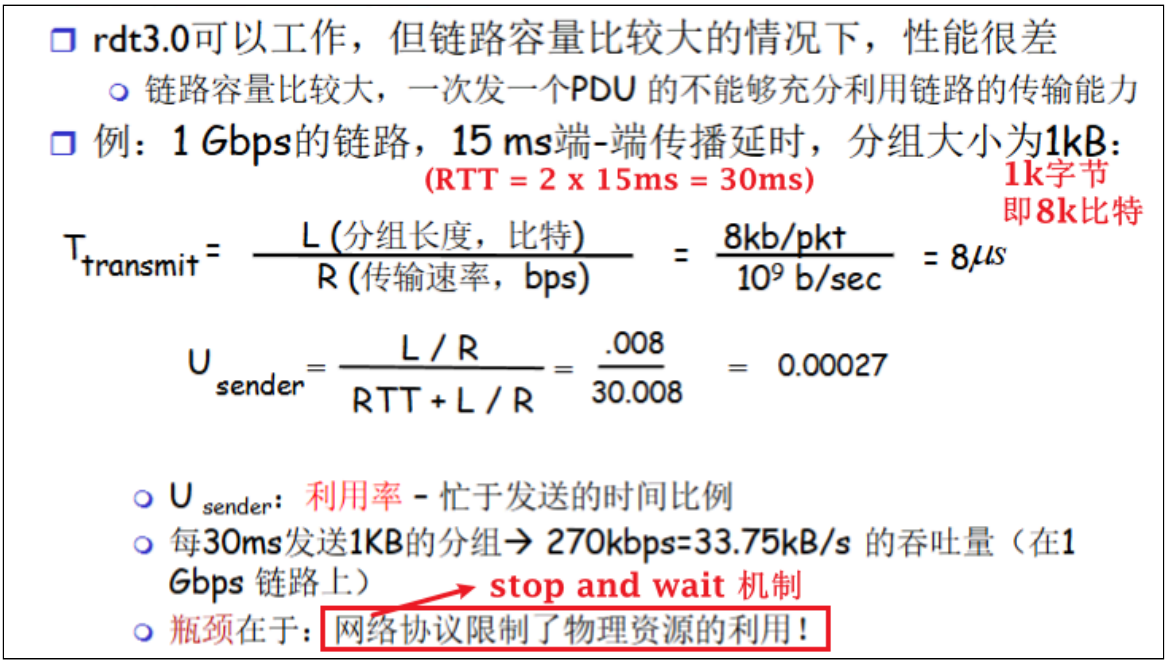
当超时时间设置非常不合理, 比如过短时, 就会出下列的(d)情况:



虽然也能正常工作, 但是会导致**效率很低**

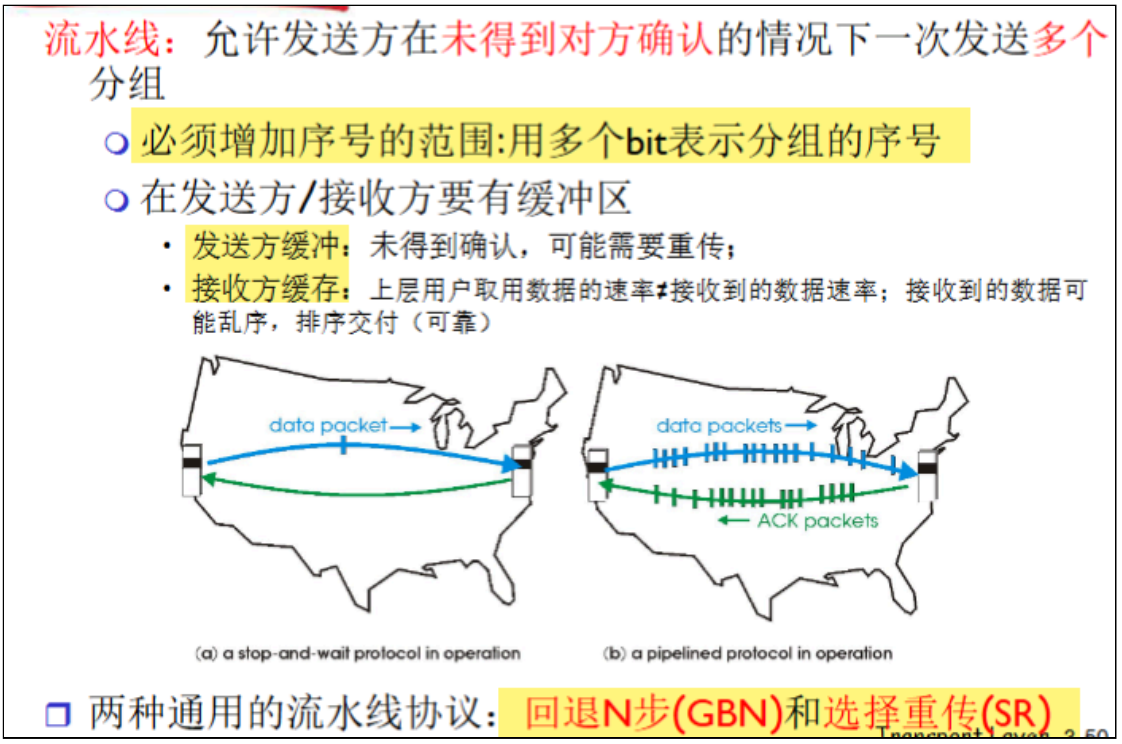
到此为止, rdt3.0已经看起来**基本完备**了

但是rdt3.0的效率很低, 因为每发送一个分组都要等待一个ACK分组的到来, 这就导致了**停等协议(stop-and-wait protocol)**, 也就是每发送一个分组都要停下来等待ACK分组的到来, 这就导致了**信道利用率非常低**



3.4.4 流水线协议

因此, 现在我们需要把**停等协议**升级为**流水线协议(pipelined protocol)**



流水线协议主要有两种, **回退N步(GBN)和**选择重传(SR)**, 他们的区别在于接收方的滑动窗口/接收缓冲区的大小是等于1还是大于1.**
这两个流水线协议, 都要基于一个通用的滑动窗口协议(sliding window protocol)来实现

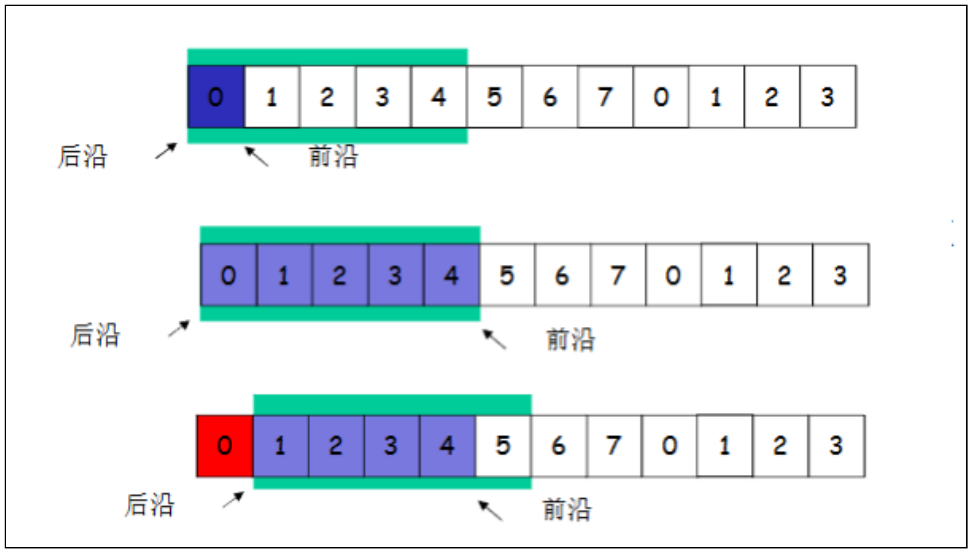
3.4.4.1 滑动窗口协议

该协议引入了**缓冲区和滑动窗口的概念**

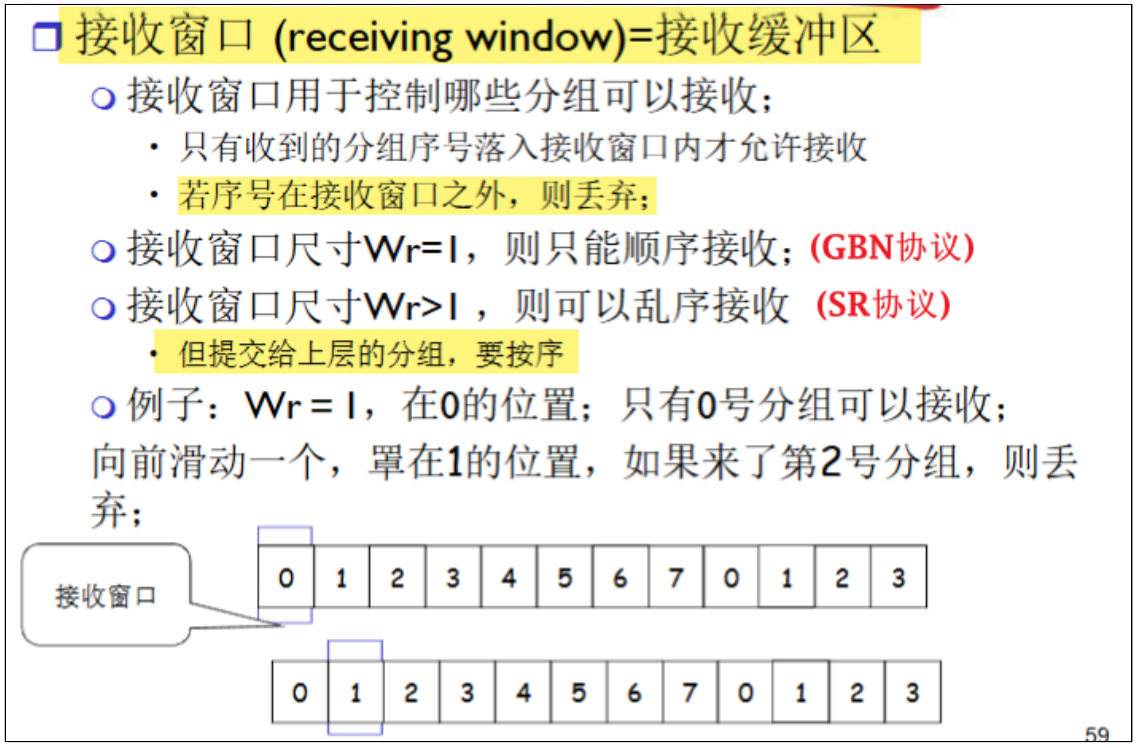
1. **发送缓冲区**: 用于存放可以允许进行发送的分组
2. **发送窗口**: 用于存放已经发送但还没有收到ACK的分组, 发送窗口是发送缓冲区的一个子集, 用于存放已经发送但还没有收到ACK的分组
3. **发送缓冲区的大小**: 对于停等协议, 发送缓冲区的大小为1; 对于**流水线协议**, 发送缓冲区的大小大于1, 但是也有上限, 太大的话链路利用率达到饱和了, 就没有意义了
4. **接收缓冲区**: 用于存放已经收到但还没有被应用层读取的分组
5. **接收缓冲区的大小**: 对于GBN协议, 接收缓冲区/接收窗口的大小为1, 表示允许接收的分组. 对于SR协议, 接收缓冲区/接收窗口的大小大于1, 表示可以乱序接收的分组范围
6. **接收窗口**: 接收窗口等于接收缓冲区的大小

发送窗口的后沿和发送缓冲区的后沿始终是同一个位置, 当从当后沿位置的分组已收到ACK确认后, 发送缓冲区就整体前移, 发送窗口的后沿也就跟着发送缓冲区的后后沿移动一下

每发送一个缓冲区中的分组, 发送窗口的前沿就往前移动一格

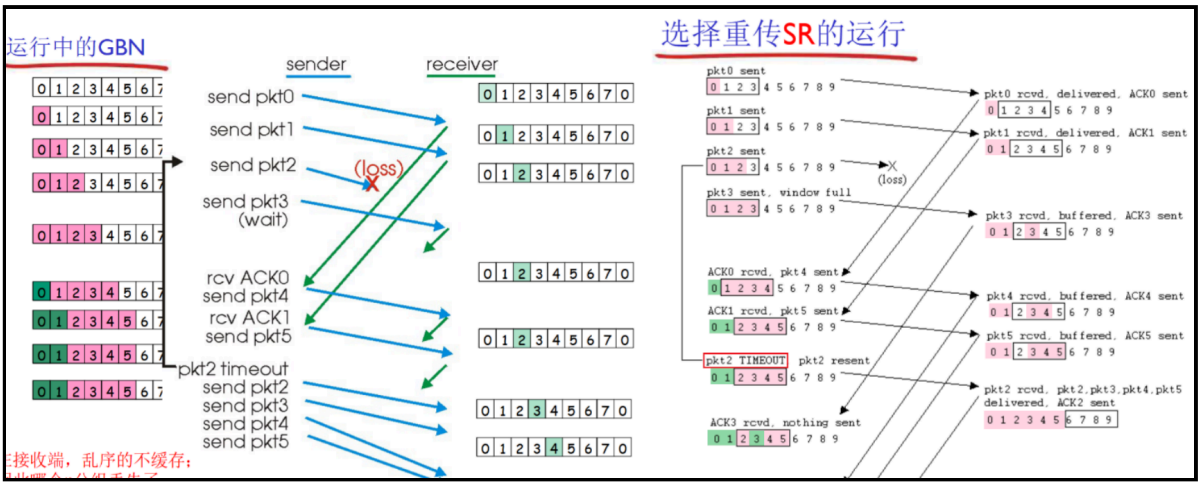


对于接收窗口/接收缓冲区, 规则如下:



GBN vs SR协议

GBN和SR图示例子



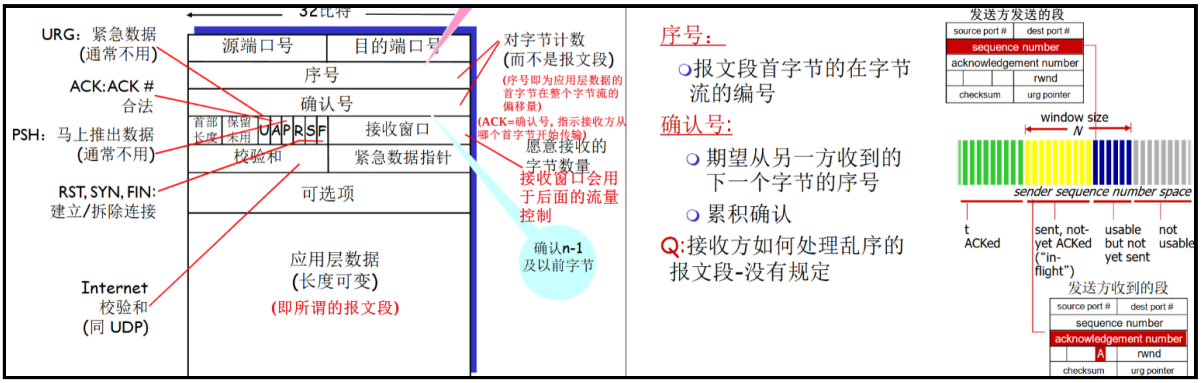
3.5 面向连接的传输: TCP

TCP(transmission control protocol)概述

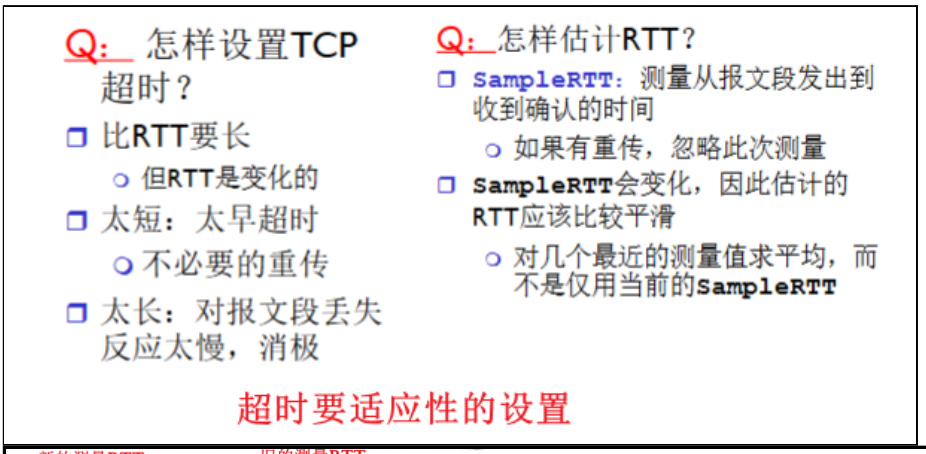
1. **点对点**: 一个接收方, 一个发送方
2. **可靠, 按顺序的字节流**: 没有报文段边界
3. **具有发送与接收缓冲**
4. **管道化(流水线)**: TCP拥塞控制和流量控制窗口大小
5. **全双工**: 数据可以同时在两个方向传输
   数据报文被分成一个个的报文段, 每个把文段都给他加上一个TCP首部, 形成TCP数据报
   MSS表示最大的报文段长度, MSS的值是由TCP首部和IP首部的长度决定的....
6. **面向连接**: 数据交换之前, 通过握手初始化发送方和接收方的状态变量
7. **流量控制**: 发送方不会发送超过接收方缓冲区大小的数据

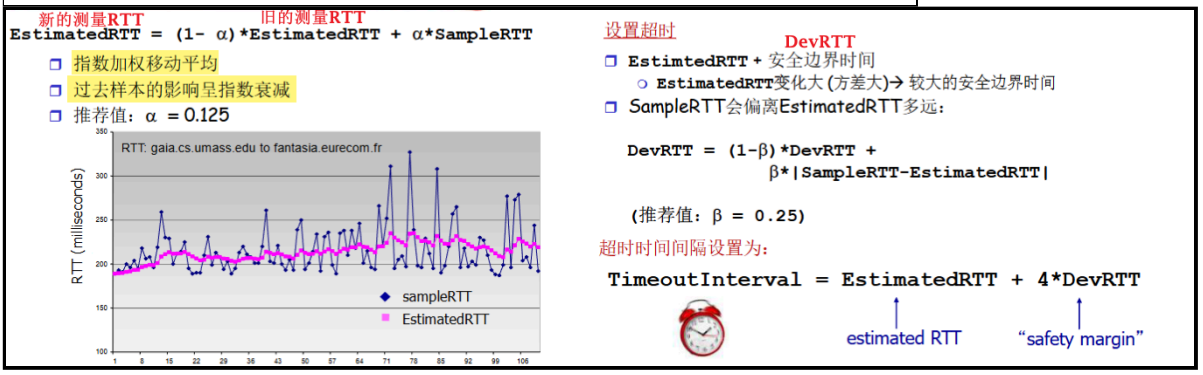
3.5.1 段结构

TCP的报文段结构



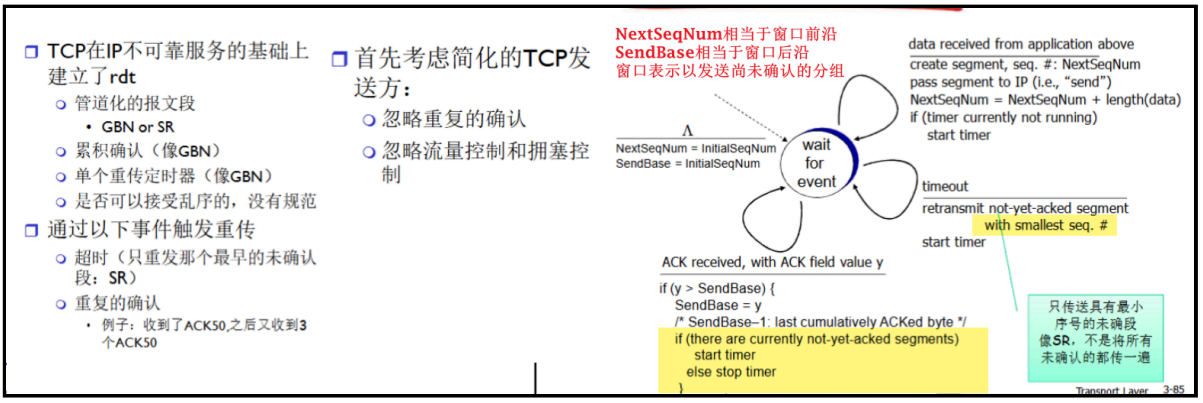
TCP超时定时器的设置





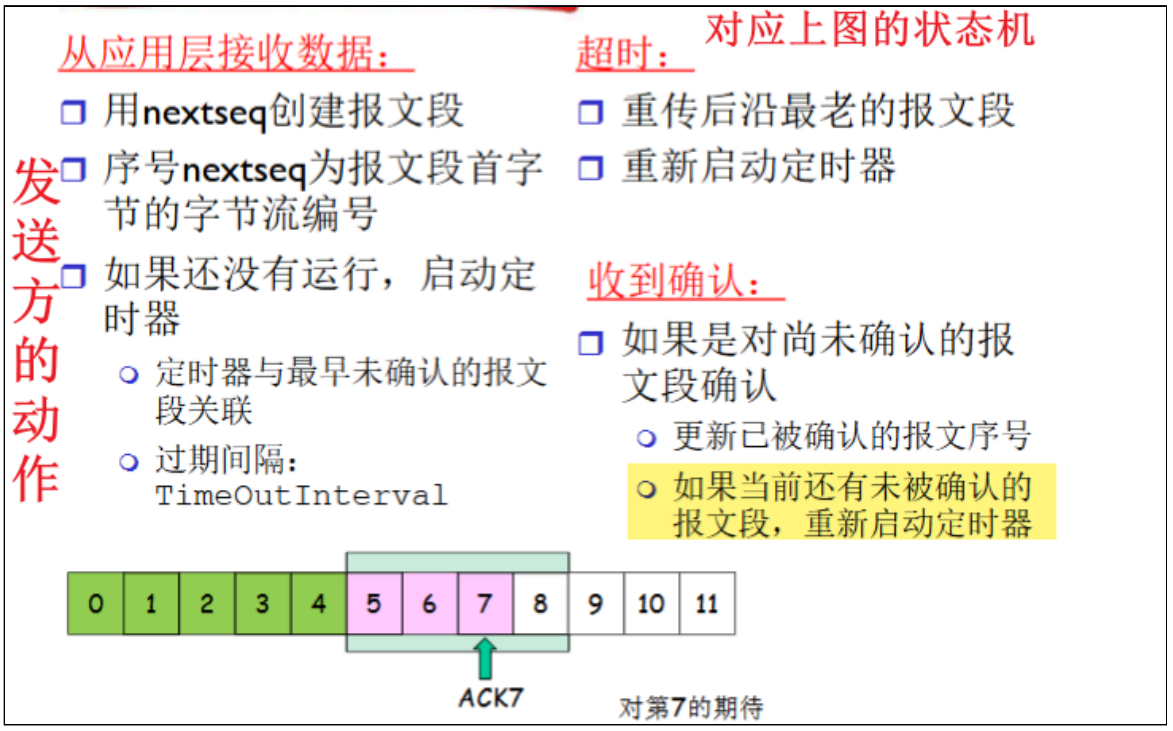
3.5.2 可靠数据传递

前面有将GBN和SR协议, 而TCP的可靠数据传输可以是两者的结合体, 在某些方面像GBN, 在某些方面像SR

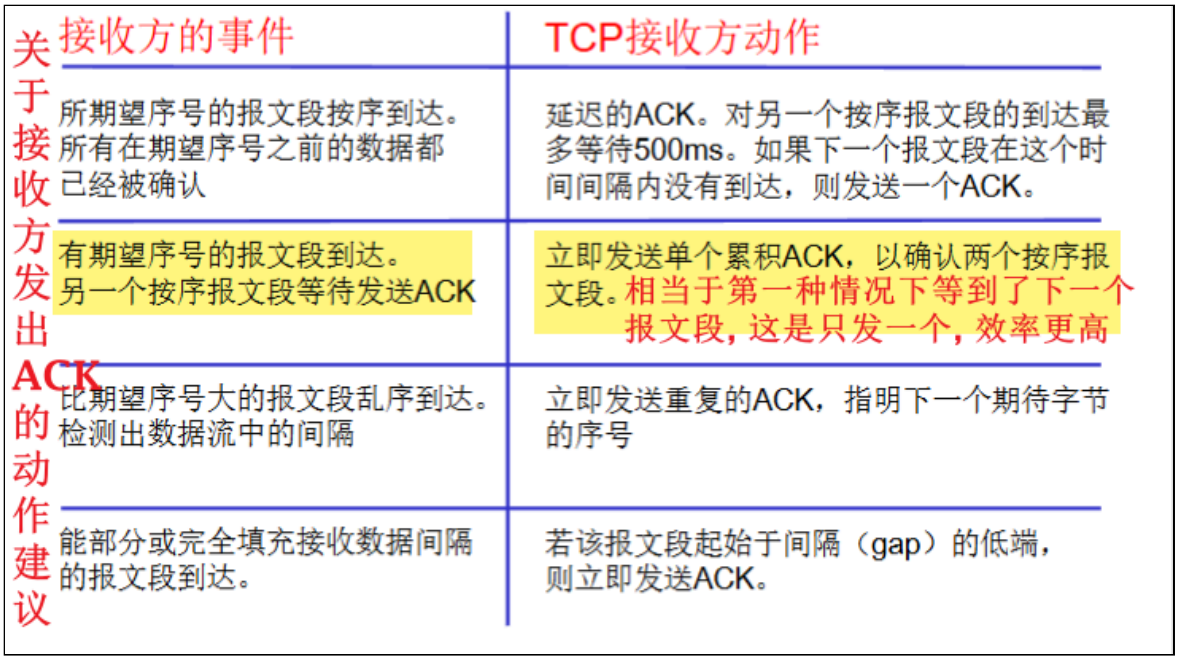


TCP的rdt规定的是**累计确认**, 即ACK(n)表示已经正确接收了所有序列号小于n的分组 (但是不保证序列号为n的分组已经被正确接收了)

tcp发送方应有动作

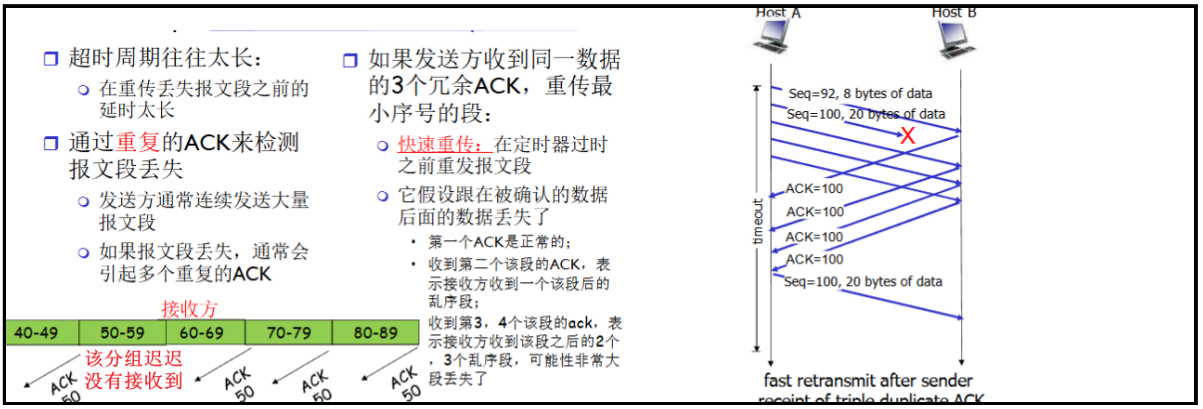


tcp接收方应有动作



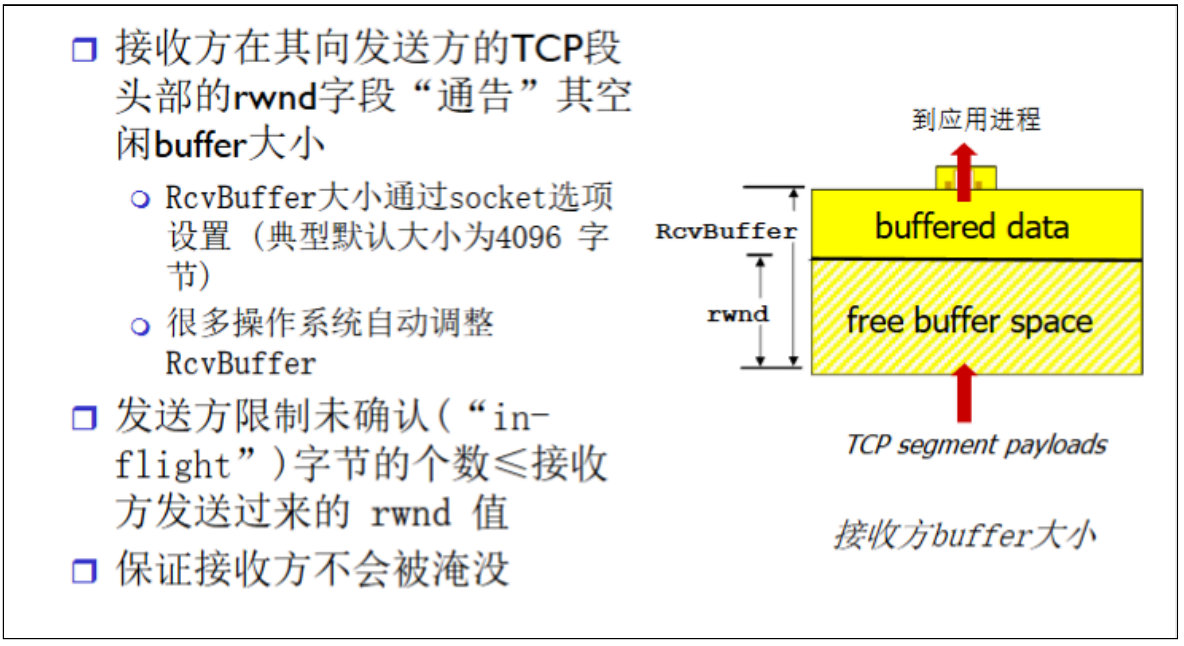
快速重传(fast retransmit)

前面说到, tcp接收方发出的ACK确认是**累计确认**, 因此就会有前面所说的没有超时, 但是发送方连续收到了多个同一个ACK分组确认的情况, 这个时候, 发送方需要做的是立即重传, 无需再等待超时了, 这个机制就叫做**快速重传**(fast retransmit), 这样做的目的就是要尽快的重传丢失的分组, 以减少重传的时延



3.5.3 流量控制

流量控制的主要目的是: 防止发送方发送太快, 太多以至于让接收方的缓冲区溢出, 造成分组丢失
前面的TCP报文的头部信息里有**接收窗口大小**的字段(接受方空闲的缓冲区大小, rwnd), TCP的发送方就是根据这个来进行流量控制

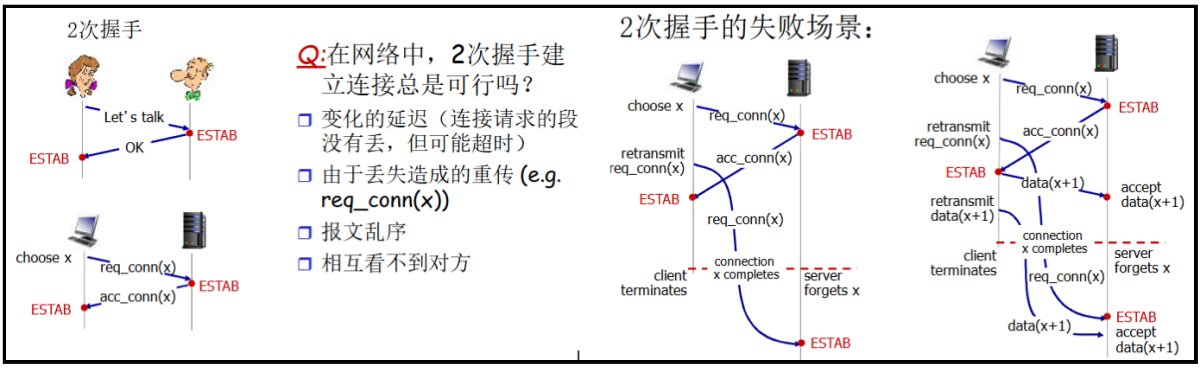


3.5.4 TCP连接管理

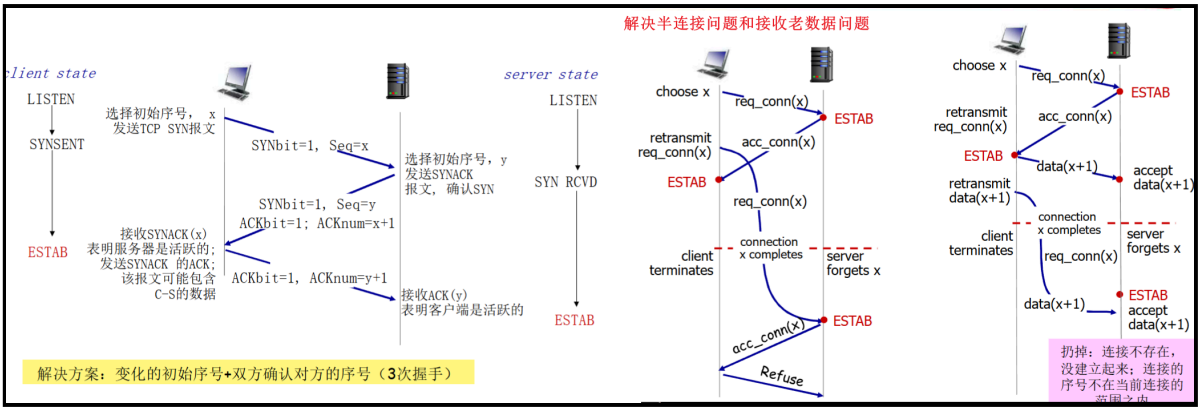
TCP建立连接

在正式交换数据之前, 发送方和接收方需要握手建立通信关系, 一方面需要双方都同意建立连接, 另一方面需要交换一些初始的状态变量(比如序列号等), 以便后续的数据传输能够顺利进行

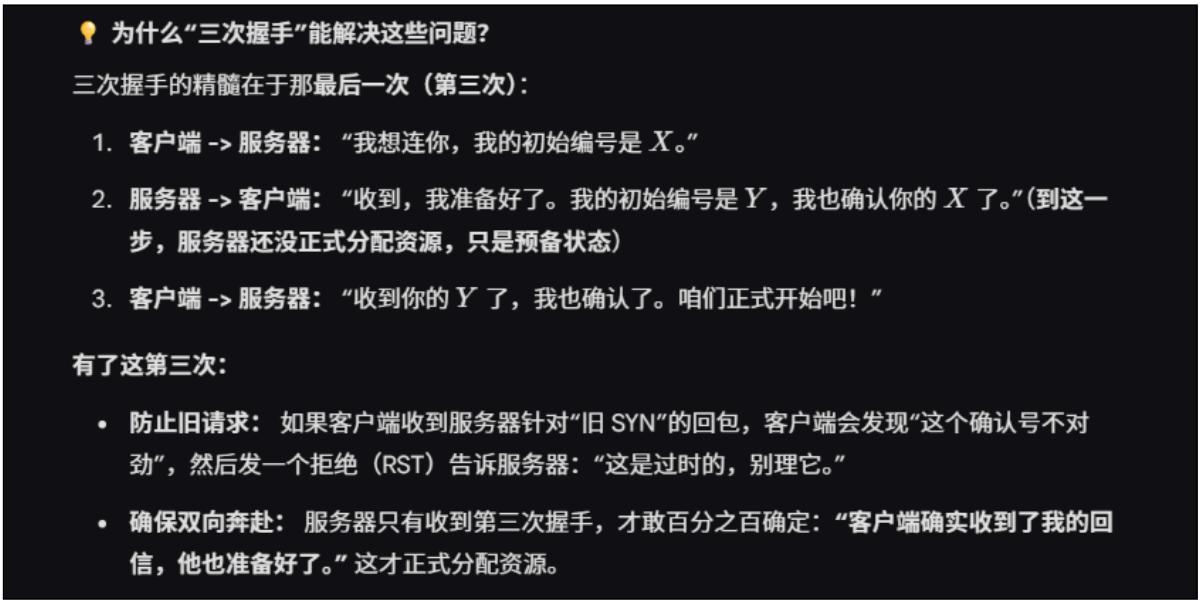
正常的逻辑来讲, 建立连接似乎用**两次握手**就足够了, 但是由于不可靠的信道有丢失等问题, 可能导致两次握手建立连接出现很多问题, 下图左侧失败场景是**服务器半连接问题**, 右侧失败失败场景是**服务器把旧数据当做新数据接收**



因此, 通过三次握手建立连接是必要的, TCP报文包含一个**SYN**的标志位, 用于表示这是一个连接请求报文
为了解决两次握手的半连接问题和接收老数据问题, TCP的三次握手过程如下:

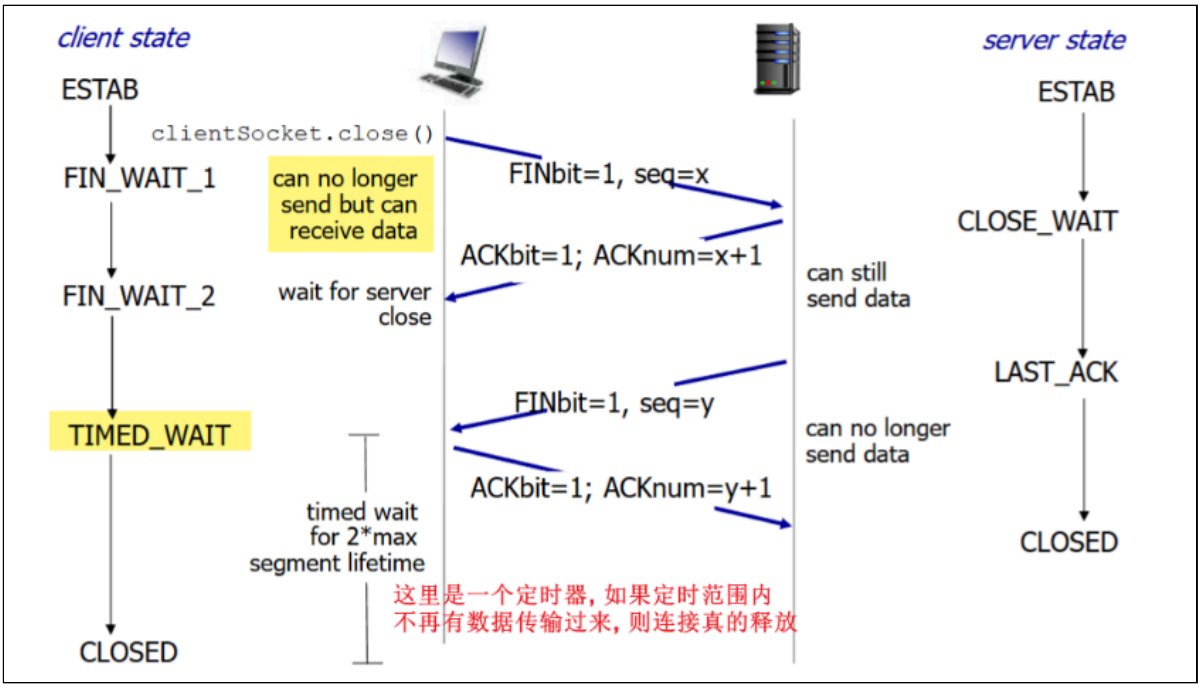


通俗的解释TCP三次握手的过程如下:



TCP关闭连接

TCP设计的关闭链接**不算是一个很完备的机制**
TCP在关闭连接是需要服务器端和客户端分子**关闭他们自己那一侧的连接**, 也就是说, 服务器端和客户端都需要发送一个**FIN**报文来关闭连接.
一旦一端接收到FIN, 就会哦那个ACK回应, 同时可以把自己需要发出的FIN段连同这个ACK段一起发出.



3.6 拥塞控制原理

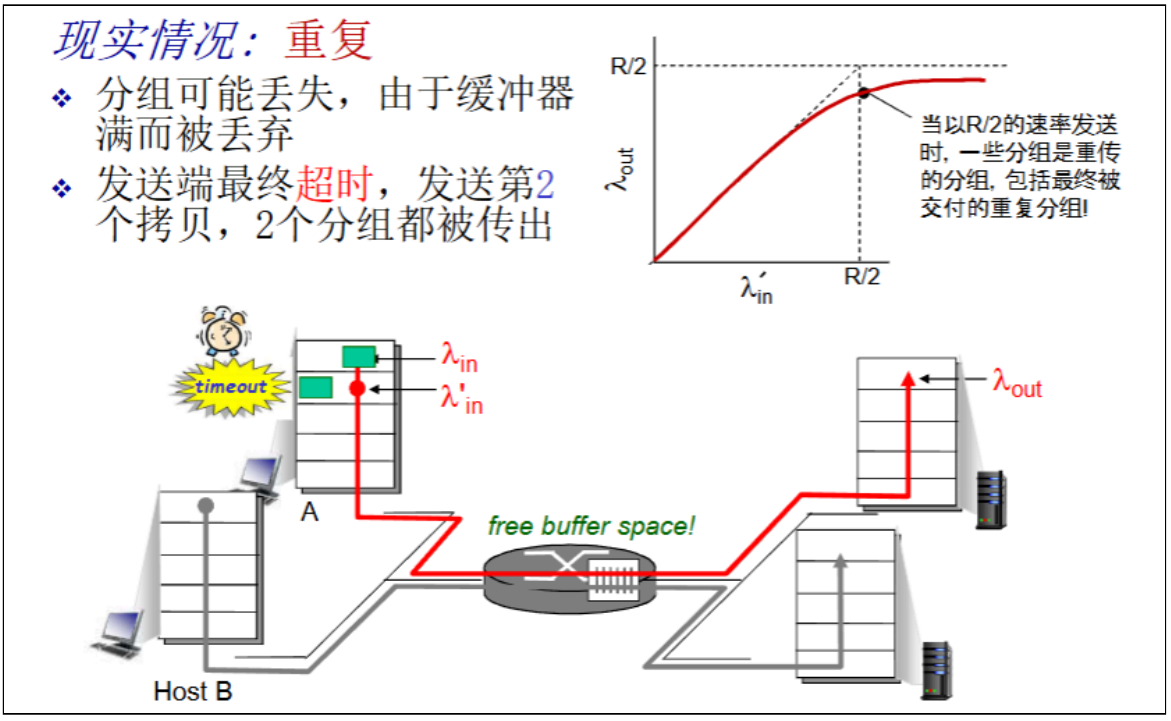
流量控制是**端系统之间的控制**, 主要是为了防止发送方发送过快, 以至于让接收方的缓冲区溢出

拥塞控制是为了应对**网络的堵塞(网络核心)**, 而不是端系统的过载, 网络堵塞的表现主要是:

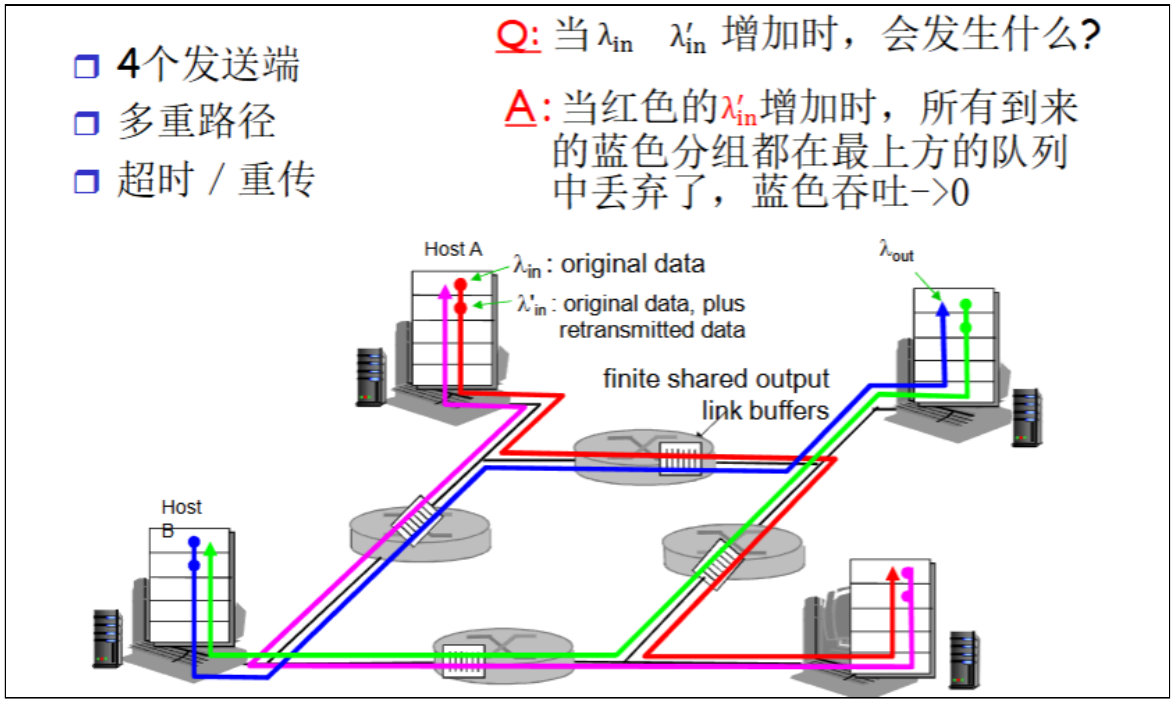
1. **分组丢失**: 路由器的缓冲区溢出, 丢失率要比不堵塞时高很多
2. **时延增加**: 要在路由器的队列中排队, 时延要比不堵塞时高很多

3.6.1 拥塞的原因

由于路由器的**缓冲区是有限的**, 当网络中的数据流量过大时, 路由器的缓冲区就会溢出, 导致分组丢失; 同时, 当网络中的数据流量过大时, 路由器的队列就会变长, 导致时延增加
对于那些丢失的分组, **为了保证有效的输出**, 发送端会利用**超时**来进行重传, 这样的重传是必要的; 但是由于发送端并不知道路由器的缓冲大小以及是否缓冲区已满, 此时即便某个分组不丢失, 只是滞留在了路由去的缓冲队列里, 那么发送端仍会发送重传分组, 这些属于是**不必要的重传**.



另外, 但一个路由器负责多重路径路径时, 各个路径可能会争抢路由器的缓冲区, 往往位于上游的路径会占用更多的缓冲区, 导致位于下游的路径的分组更容易被丢弃, 这就是**公平性问题**.



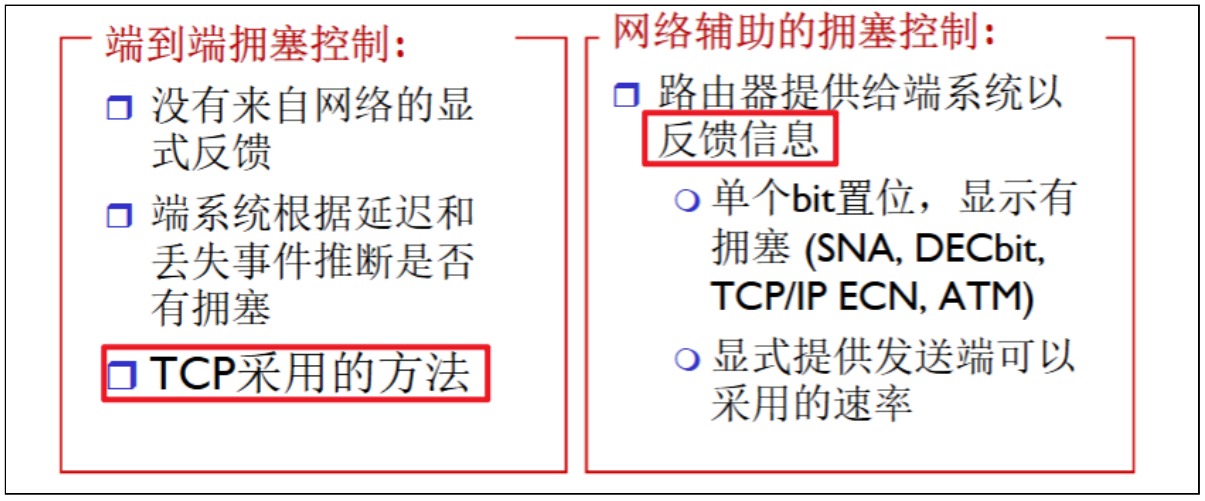
3.6.2 拥塞的代价

综上, 根据他们的原因, 可以知道拥塞带来的危害有

1. 为了达到一个有效输出, 网络需要做更多的工作---**重传**
2. 没有必要的重传使得链路中包括了多个分组的拷贝, 即那些没有丢失, 但是处于拥塞状态, 传播时间很长导致超时的分组.
3. 当分组丢失时, **任何关于这个分组的上游传输能力**都被浪费了

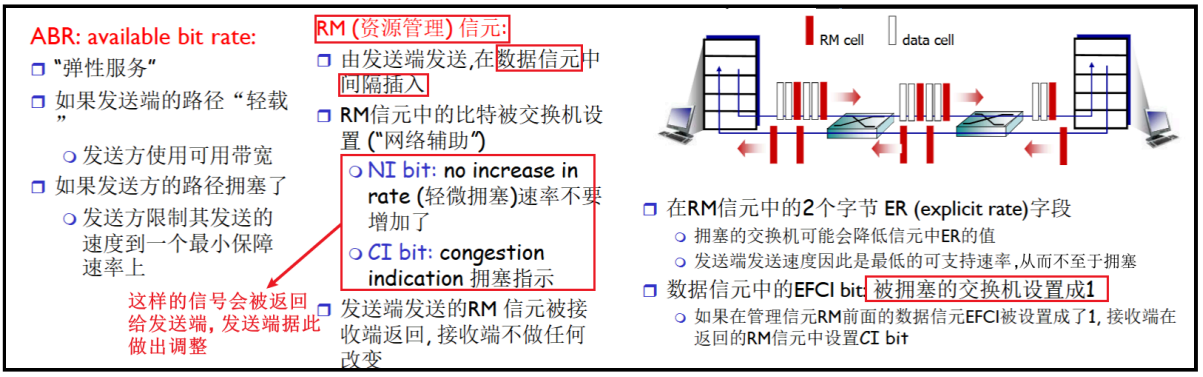
3.6.3 拥塞控制的方法

拥塞控制的主要方法有两种, **端到端的拥塞控制**和**网络辅助的拥塞控制**



案例: ATM ABR拥塞控制

ATM网络的拥塞控制就是典型的**利用反馈信息**来进行控制的案例, 缺点就是会给网络本身带来额外负担, 复杂性提升.



3.7 TCP 拥塞控制

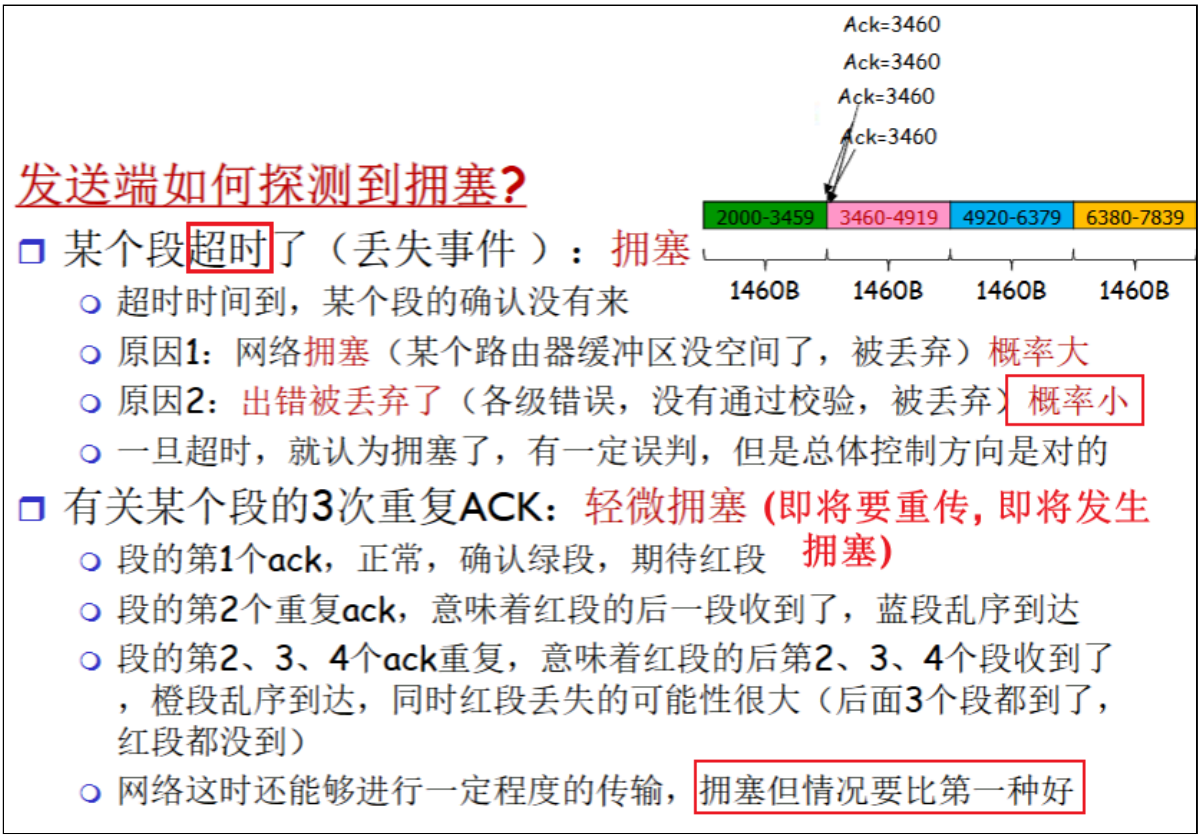
TCP 采用**端到端的拥塞控制**, 路由器不向主机提供有关拥塞的反馈信息, 端系统只根据自身的得到的信息判断是否发生拥塞, 并采取相应的动作
这样的机制下, **路由器的负担较轻**, 符合**网络核心简单的TCP/IP架构原则**

设计TCP的拥塞控制需要考虑以下问题:

1. **如何检测拥塞?**: 是轻微拥塞还是拥塞
2. **控制拥塞的策略**: 在拥塞发送时如何动作来降低发送速率(轻微拥塞如何做, 拥塞如何做?), 在拥塞缓解时如何动作来增加发送速率?

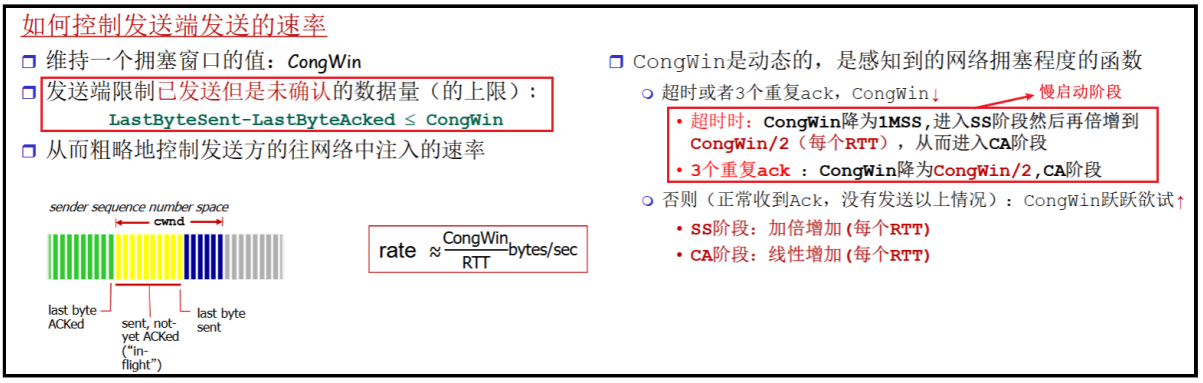
3.7.1 端到端拥塞感知

TCP的拥塞感知主要是通过**超时**和**重复ACK**来实现的, 这两者都可以用来检测拥塞, 但是他们的检测能力不同, 超时可以检测到严重拥塞, 而重复ACK能检测到轻微拥塞



3.7.3 TCP 拥塞控制 速率控制

TCP通过维持一个**拥塞窗口(congestion window, cwnd)** 来控制发送速率, 这个窗口的大小是动态变化的, 根据网络的拥塞程度来调整



另外, 为了即照顾到流量控制, 又照顾到拥塞控制, 通常将TCP的发送窗口(SendWin)设置为拥塞窗口(CongWind)和接收窗口(RecvWin)的最小值
即**SendWin = min(CongWind, RecvWin)**
这样的联合控制操作既能满足拥塞控制, 又能满足流量控制, 以保证TCP的可靠数据传输

3.7.4 TCP 拥塞控制算法

前面图片中提到的**CongWin控制调整策略**可以分为三个:

- **慢启动(SS)**
- **AIMD(加性增, 乘性减)**
- **超时事件后的保守策略**

整个TCP控制过程大致就是:一开始ConWin设置成1 MSS, 进行慢启动(SS阶段), 这时候CongWin增长很快, 也就是发送速率增长很快, 后面由于增长太快大概率发生**超时**, 此时的动作是会**把一个阈值Threshold设置成当前CongWin的一半(这里的阈值用于后续实现从SS状态到CA拥塞避免状态)**, 然后把CongWin设置成1 MSS, 重新开始慢启动SS阶段, 当CongWin增长到Threshold时, 就进入CA拥塞避免阶段, 这时候CongWin的增长就变慢了, 以线性增长的方式增加, 以避免再次发生丢失事件;
如果CA阶段发生了**超时**, 那么就会把Threshold设置成当前CongWin的一半, 然后把CongWin设置成1 MSS, 重新开始慢启动SS阶段, 以此类推
如果在CA阶段发生了丢失事件(**连续收到三个重复的ACK**), 那么就会把Threshold设置成当前CongWin的一半, 然后把CongWin设置成1 MSS, 重新开始慢启动SS阶段, 以此类推然后**并不把CongWin设置成1 MSS, 而是把CongWin设置成Threshold+3, 直接进入拥塞避免阶段, 以此类推**

总结所有动作, 大致如下表格:

事件	状态	TCP 发送端行为	解释
以前没有收到ACK的data被ACKed	慢启动(SS)	CongWin = CongWin + MSS If (CongWin > Threshold) 状态变成 “CA”	每一个RTT CongWin 加倍
以前没有收到ACK的data被ACKed	拥塞避免 (CA)	CongWin = CongWin+MSS * (MSS/CongWin)	加性增加, 每一个RTT对CongWin 加一个 1 MSS
通过收到3个重复的ACK，发现丢失的事件	SS or CA	Threshold = CongWin/2, CongWin = Threshold+3, 状态变成 “CA”	快速重传, 实现乘性的减. CongWin 没有变成1 MSS.
超时	SS or CA	Threshold = CongWin/2, CongWin = 1 MSS, 状态变成 “SS”	进入slow start
重复的 ACK	SS or CA	对被ACKed 的segment， 增加重复ACK的计数	CongWin and Threshold 不变

Transport Layer 3-134

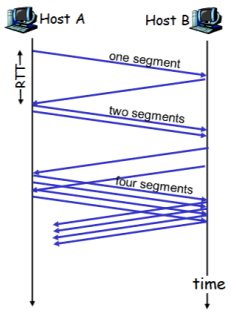
慢启动(SS)

慢启动概述

- 连接刚建立, **CongWin = 1 MSS**
  - 如: MSS = 1460bytes & RTT = 200 msec
  - 初始速率 = 58.4kbps
- 可用带宽可能 大于 MSS/RTT
  - 应该尽快加速，到达希望的速率
- 当连接开始时，指数性增加发送速率，直到发生丢失的事件
  - 启动初值很低
  - 但是速度很快
- 当连接开始时，指数性增加（每个RTT）发送速率直到发生丢失事件
  - 每一个RTT， CongWin加倍
  - 每收到一个ACK时，CongWin加1（why）
  - 慢启动阶段：只要不超时或3个重复ack，一个RTT，CongWin加倍

总结: 初始速率很慢，但是加速却是指数性的

- 指数增加，SS时间很短，长期来看可以忽略



AIMD(加性增, 乘性减)

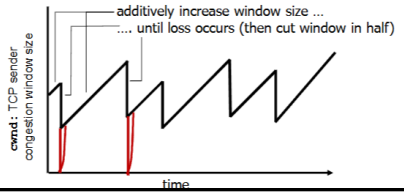
乘性减:

丢失事件后将**CongWin**降为**1**，将**CongWin/2**作为阈值，进入慢启动阶段（倍增直到**CongWin/2**）

加性增：

当CongWin>阈值时，一个RTT如没有发生丢失事件,将**CongWin**加1MSS: *探测*

AIMD saw tooth behavior: probing for bandwidth



- 当收到3个重复的*ACKs*:
  - **CongWin** 减半
  - 窗口（缓冲区大小）之后线性增长
- 当超时事件发生时:
  - **CongWin**被设置成 1 MSS，进入SS阶段
  - 之后窗口指数增长
  - 增长到一个阈值（上次发生拥塞的窗口的一半）时，再线性增加

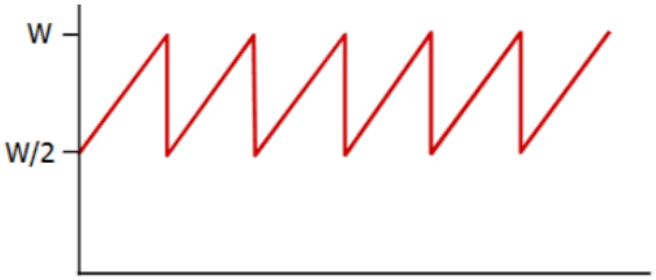
思路

- 3个重复的ACK表示网络还有一定的段传输能力
- 超时之前的3个重复的ACK表示“警报”

3.7.5 TCP 吞吐量

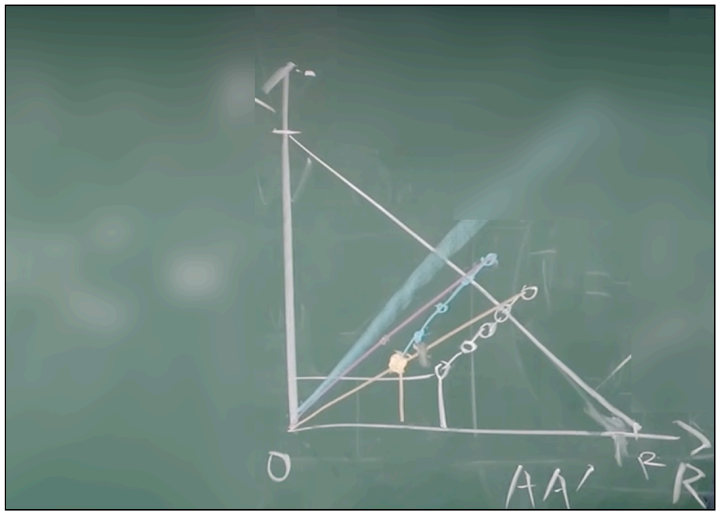
- TCP的平均吞吐量是多少，使用窗口window尺寸W和RTT来描述?
  - 忽略慢启动阶段，假设发送端总有数据传输 慢启动阶段很快就过去了
- W：发生丢失事件时的窗口尺寸（单位：字节）
  - 平均窗口尺寸（#in-flight字节）： 3/4W = (w/2 + w) / 2
  - 平均吞吐量：RTT时间吞吐3/4W

$$\text{avg TCP thruput} = \frac{3}{4}\frac{W}{RTT} \text{ bytes/sec}$$



3.7.6 TCP 具有公平性

对于一条被多个连接通道通用的链路, TCP的拥塞控制能够防止某个链路侵略性的独占带宽, 当某个过大时, 他就说主动缩小, 这样的话, 最后能够大致维持一个每个连接通道都能获得一个公平的带宽分配的状态, 这就是TCP的公平性



如图, 横纵坐标分别代表两条链路的各自的带宽占用率, 图上的小圈点代表经历不同个数的RTT后, 两条链路的带宽占用率的状态, 可以看到, 不管初始状态是什么样的, 最后都会趋于一个公平的状态(也就是图上的45度线), 这就是TCP的公平性

第四章：网络层 - 数据平面

- 4.1 导论 -- 网络层概述
  - 4.1.1 网络层服务
  - 4.1.2 网络层的两个平面
- 4.2 路由器组成
  - 4.2.1 输入端口功能
  - 4.2.2 交换结构
  - 4.2.3 输出端口功能
- 4.3 IP 协议
  - 4.3.1 IP 数据报格式(IPv4)
  - 4.3.2 IP 数据报分片和重组
  - 4.3.3 IP 编址
  - 4.3.4 转发表和转发算法
  - 4.3.5 IP 地址的获取
  - 4.3.6 NAT -- 网络地址转换
  - 4.3.7 层次编址下的路由
- 4.4 ICMP
  - Traceroute

4.1 导论 -- 网络层概述

4.1.1 网络层服务

- 在发送主机和接收主机对之间传送段（segment）
- 在发送端将段封装到数据报中
- 在接收端，将段上交给传输层实体
- 网络层协议存在于每一个主机和路由器
- 路由器检查每一个经过它的IP数据报的头部

网络层功能：

- *转发*: 将分组从路由器的输入接口转发到合适的输出接口
- *路由*: 使用路由算法来决定分组从发送主机到目标接收主机的路径
  - 路由选择算法
  - 路由选择协议

旅行的类比：

- 转发: 通过单个路口的过程
- 路由: 从源到目的的路由路径规划过程

转发 关注的是局部
路由 关注的是端到端的路径

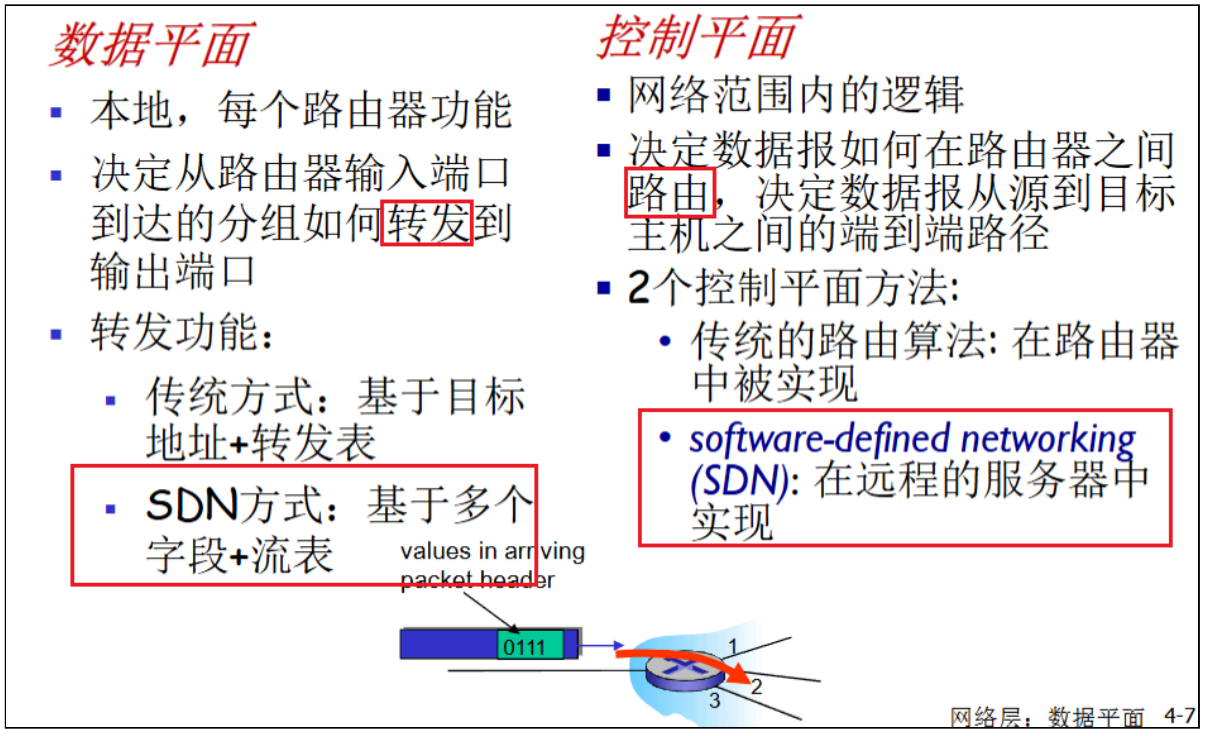
网络层接收来自上层的**报文段**, 封装成**数据报**

发送端和接收端所做的工作大致如下

位置	主要工作
发送端（如主机或路由器）	1. **封装**：将传输层传来的报文段（segment）加上 **IP 头部**（包含源IP、目的IP、TTL等）。 2. **分片**（若需要）：如果数据报长度超过出站链路的 **MTU**（最大传输单元），则拆分为多个较小的 IP 分片。 3. **路由（查表）**：根据目的IP查找 **路由表**，决定下一跳的IP和出接口。 4. **交付给数据链路层**：将构造好的IP数据报（或分片）传递下去，封装成帧。
接收端（最终目标主机）	1. **接收与提取**：从数据链路层接收帧，取出其中的IP数据报。 2. **分片重组**：如果收到的是一组分片，则缓存并等待所有分片到达，重组为完整的原始IP数据报。 3. **交付给上层**：检查目的IP是否为本机（或本机配置的地址），确认无误后去掉IP头部，将负载交给传输层（如TCP/UDP）。 4. （若设备是路由器且启用转发）**转发**：如果目的IP不是本机，则可能根据转发表继续转发，此时行为类似于发送端。

> 简单记忆：发送端 **封装 + 分片 + 查表**；接收端 **重组 + 解封装 + 上交**。

4.1.2 网络层的两个平面



两个平面的区分**描述的是路由器/交换机的内部功能划分**

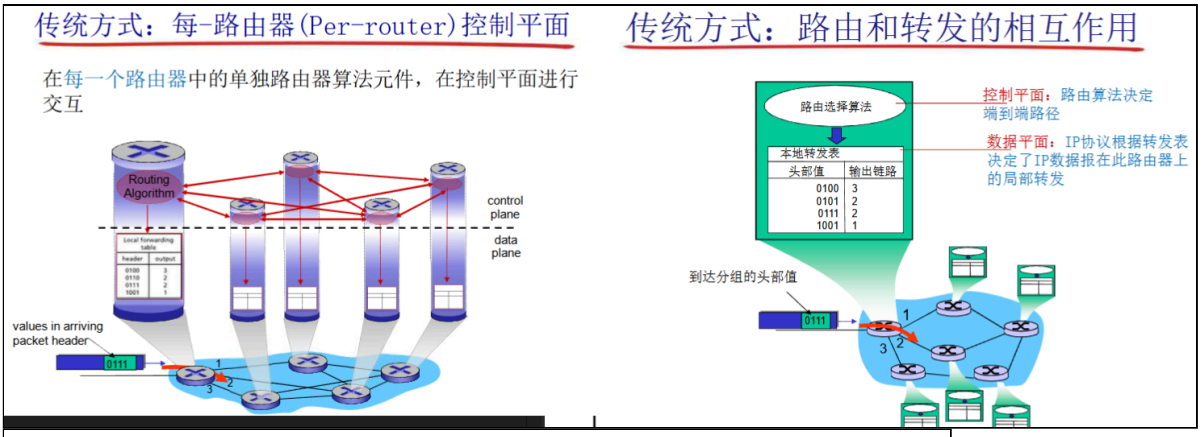
平面	核心任务	关键特点	实现位置
数据平面	**转发（Forwarding）** - 对每一个到达的IP分组，查找 **转发表（Forwarding Table）**，决定输出端口。 - 修改TTL，重新计算校验和，将分组从正确端口送走。	- **逐分组、高速** - 操作简单、重复 - 时间敏感（每分组纳秒级）	通常由 **硬件**（如ASIC、NP）实现，以保证线速转发。
控制平面	**路由（Routing）** - 运行路由协议（OSPF、BGP、RIP等）。 - 与相邻路由器交换路由信息，计算 **路由表（Routing Table）**。 - 将路由表中最优条目 **下发** 到数据平面的转发表。	- **全局、慢速** - 逻辑复杂 - 时间不敏感（秒/分钟级）	通常由 **软件**（CPU、路由进程）实现。

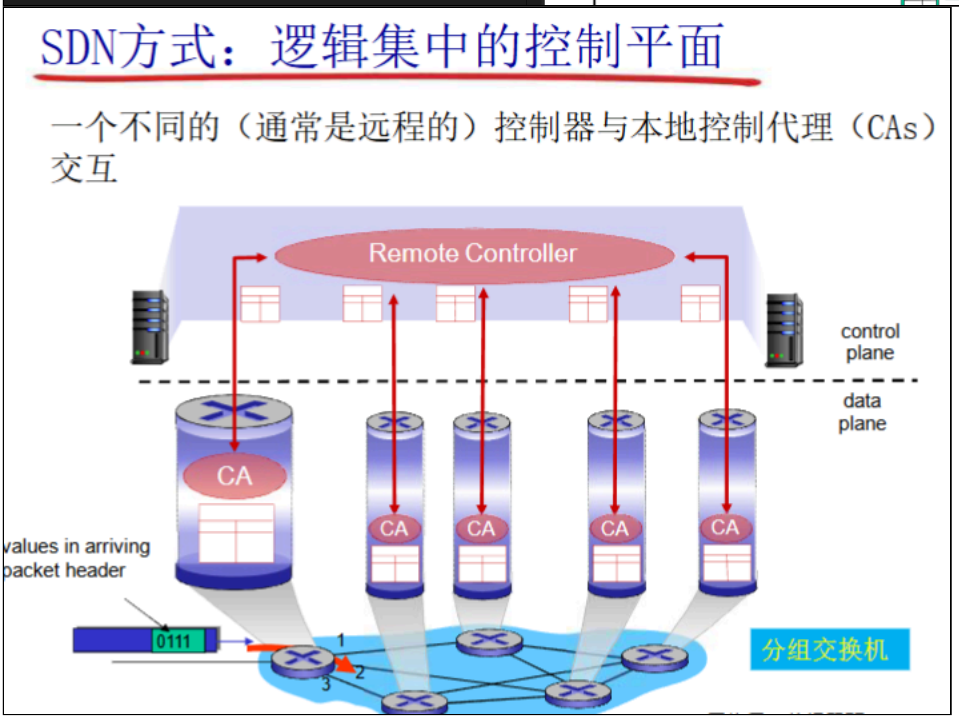
直观理解就是:

- **数据平面**: 像个交换机矩阵 -- 每个包来了, 看一眼表就扔出去
- **控制平面**: 像个大脑 -- 统一对话了解全网络拓扑, 然后告诉数据平面 "该往哪儿扔"

传统方式 vs. SDN

SDN（Software-Defined Networking，软件定义网络） 是一种网络架构思想，核心是**将网络设备的控制平面与数据平面分离**，并**将控制逻辑集中到一个逻辑上统一的控制器**中，从而实现网络的可编程和集中管理。





SDN 把控制平面集中在一个 Remote Controller 上, 数据平面则分布在各个交换机上.
控制平面是软件, 把控制平面集中式, 这样就可以更灵活地控制定义改变网络, 也更容易实现一些高级功能

网络服务模型

“服务模型”这个术语，在计算机网络中指的是：网络层向它的上层（通常是传输层）承诺提供什么样的数据传输服务。它定义了上层可以期待什么、不能期待什么——比如数据能否一定送到、会不会有顺序错乱、延迟有多大、带宽能否保证等。

网络层向上层提供的服务, 可以从 可靠性, 延迟 等指标来看待

Q: 从发送方主机到接收方主机传输数据报的“通道”，网络提供什么样的服务模型？

对于单个数据报的服务:
- 可靠传送
- 延迟保证，如：少于40ms的延迟

对于数据报流的服务:
- 保序数据报传送
- 保证流的最小带宽
- 分组之间的延迟差

对于有些网络, 除了以上的传输服务, 网络层还要提供连接建立服务

不同于传输层的"连接"只关注端系统之间的"连接", 网络层的连接建立服务还要关注**路由器之间的连接**.

- 在某些网络架构中是第三个重要的功能
  - ATM, frame relay, X.25
- 在分组传输之前，在两个主机之间，在通过一些路由器所构成的路径上建立一个网络层连接
  - 涉及到路由器
- 网络层和传输层连接服务区别:
  - 网络层: 在2个主机之间，涉及到路径上的一些路由器
  - 传输层: 在2个进程之间，很可能只体现在端系统上(TCP连接)

IP 的服务模型被称为“**尽力而为**”（Best-Effort），它的核心意思是：网络会“尽量”把分组送到目的地，但不作任何承诺。

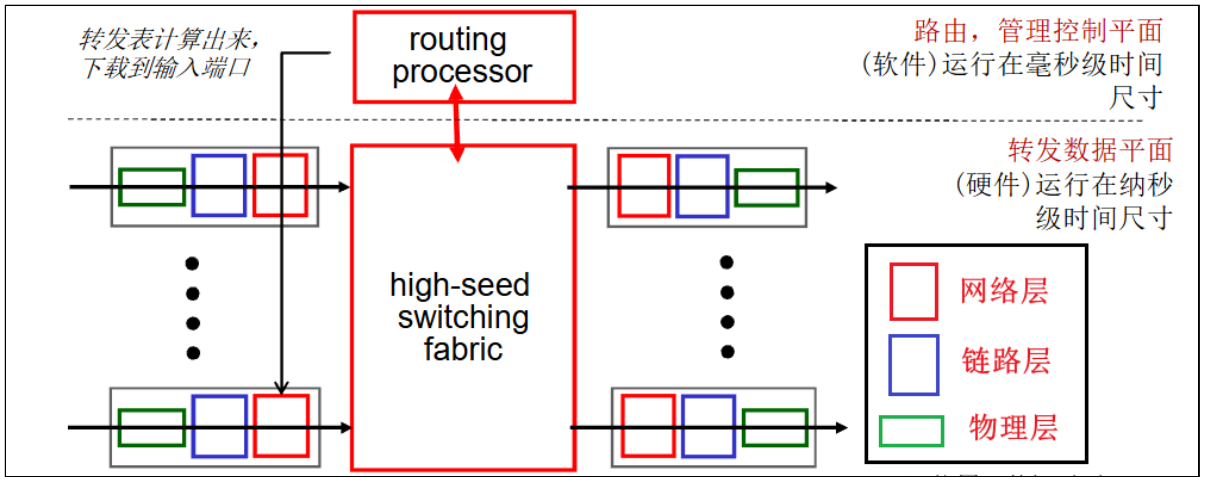
服务模型的定义可以从下面的维度去考量

网络架构	服务模型	保证？				拥塞反馈
		带宽	丢失	保序	延迟	
Internet	best effort	none	no	no	no	no (inferred via loss)
ATM	CBR 恒定速率	constant rate	yes	yes	yes	no congestion
ATM	VBR 变化速率	guaranteed rate	yes	yes	yes	no congestion
ATM	ABR 可用比特率	guaranteed minimum	no	yes	no	yes
ATM	UBR 不指名比特率	none	no	yes	no	no

把任务推到端系统

4.2 路由器组成

高层面(非常简化的)通用路由器体系架构



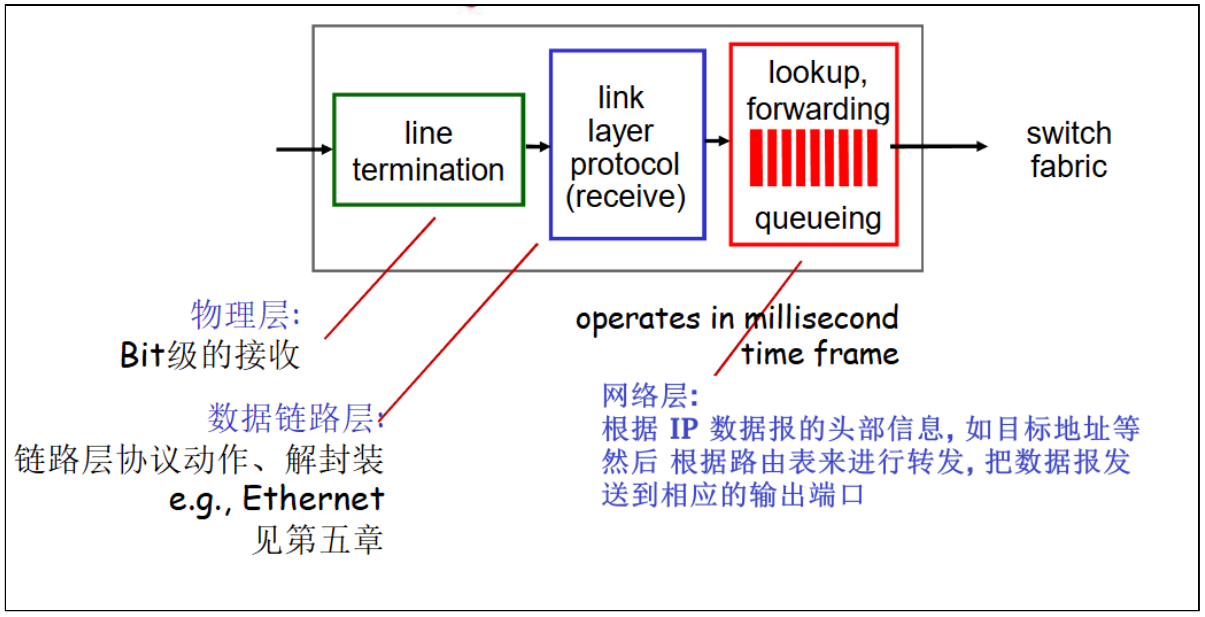
SDN 流表 和 **传统模式转发表**

- **传统转发表：** 像一个纸质电话本。你只能根据"姓"（目的 IP）查找一个"固定电话号码"（出端口）。电话本是由分散的邮递员（路由协议）各自手写的，更新很慢。
- **SDN 流表：** 像一个可编程的智能规则引擎。你可以根据"姓、名、地址、家庭人数、通话时间"等 **多个条件查找，** 执行"转接、修改语音、同时录音、转语音信箱"等 **多种动作。** 所有规则由一个中央大脑（控制器）实时编写并推送到每个电话机。

特性	传统转发表	SDN 流表
匹配字段	仅目的 IP	2~4 层任意字段组合
动作	转发或丢弃，极少修改	任意修改、多端口、封装、送控制器
表项来源	本地路由协议（分布式）	远程控制器（集中式）
更新速度	慢（秒~分钟）	快（毫秒级）
典型场景	IP 骨干网、企业网络	数据中心、SD-WAN、校园实验

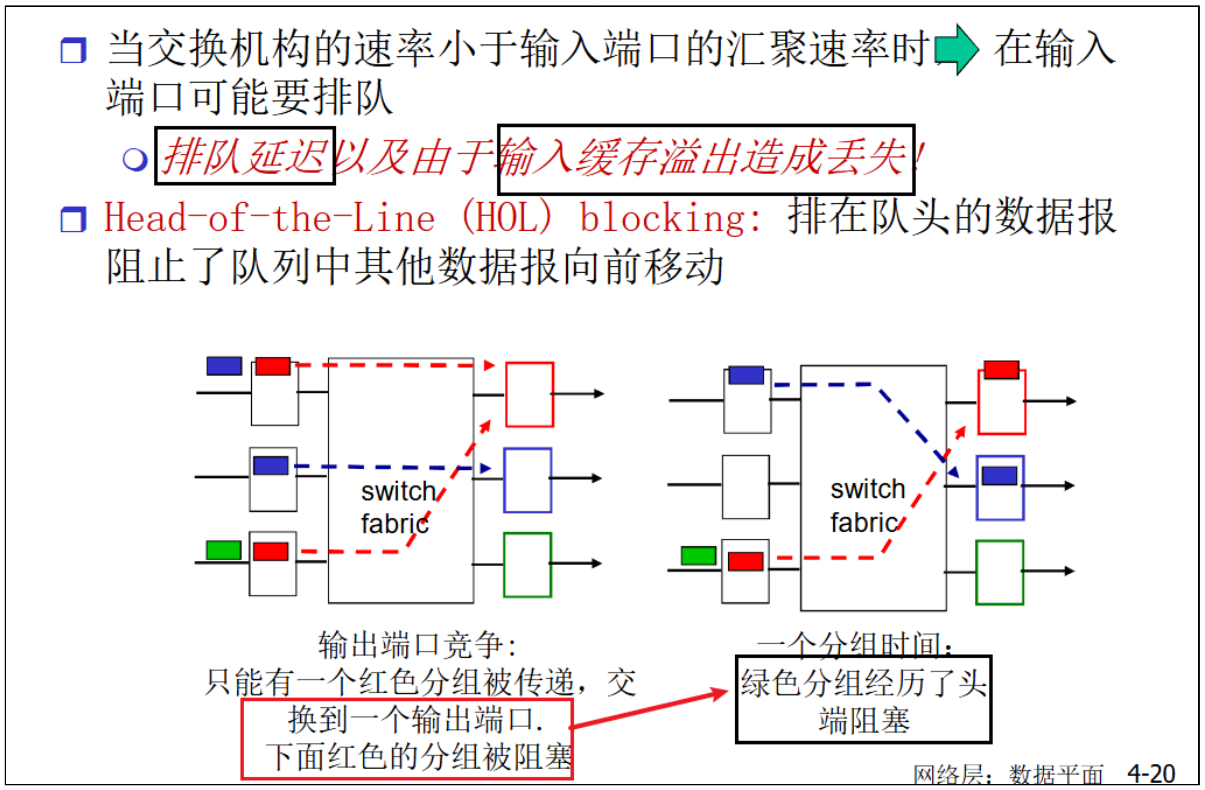
接下来, 我们会讲路由器的端口按照功能划分成两类: **输入端口** 和 **输出端口**
分别描述输入端口和输出端口提供了怎么样功能/服务

4.2.1 输入端口功能

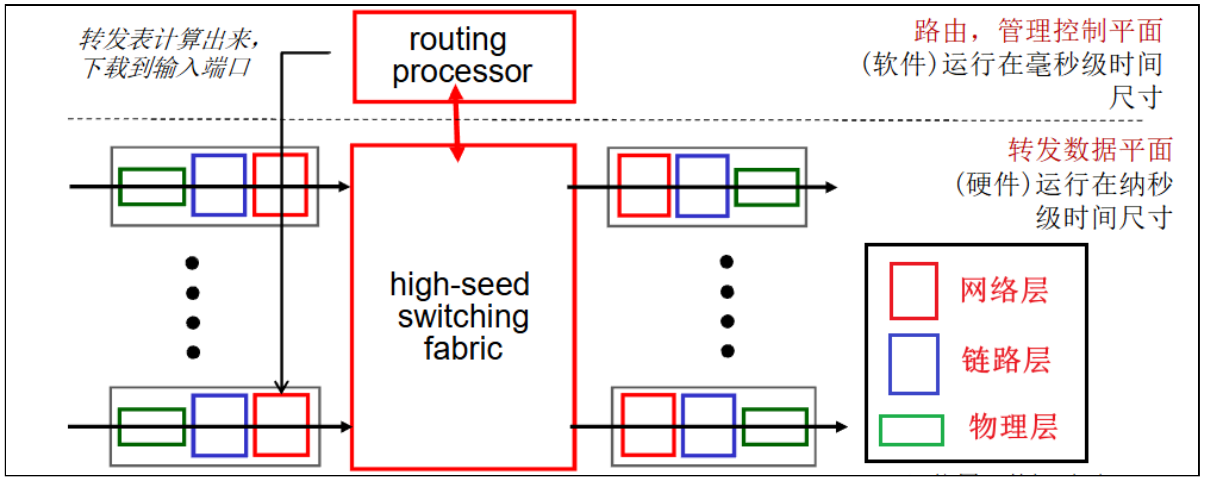


接下来根据上图, 详细介绍输入端口的功能 -- **输入端口缓存**

由于输入端口后出现 HOL（Head-of-Line）blocking, 输入端口需要一个缓存来存储数据报



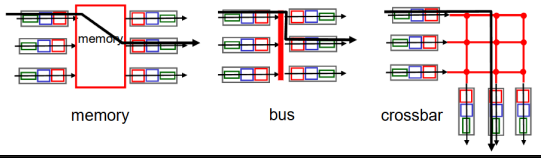
4.2.2 交换结构



中间的交换，连接输入端口和输出端口，**他的结构是怎样的**

交换结构

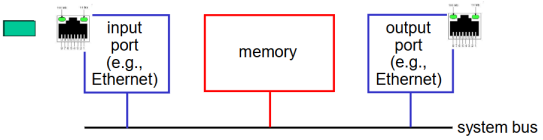
- 将分组从输入缓冲区传输到合适的输出端口
- 交换速率：分组可以按照该速率从输入传输到输出
  - 运行速度经常是输入/输出链路速率的若干倍
  - N 个输入端口：交换机构的交换速度是输入线路速度的N倍比较理想，才不会成为瓶颈
- 3种典型的交换机构



(1) 通过内存交换

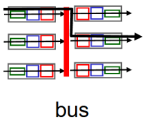
第一代路由器：

- 在CPU直接控制下的交换，采用传统的计算机
- 分组被拷贝到系统内存，CPU从分组的头部提取出目标地址，查找转发表，找到对应的输出端口，拷贝到输出端口
- 转发速率被内存的带宽限制(数据报通过BUS两遍)
- 一次只能转发一个分组



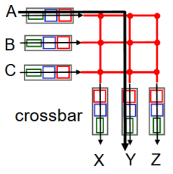
(2) 通过总线交换 已经比内存交换快了很多

- 数据报通过共享总线，从输入端口转发到输出端口
- 总线竞争：交换速度受限于总线带宽
- 1次处理一个分组
- 1 Gbps bus, Cisco 1900； 32 Gbps bus, Cisco 5600；对于接入或企业级路由器，速度足够（但不适合区域或骨干网络）

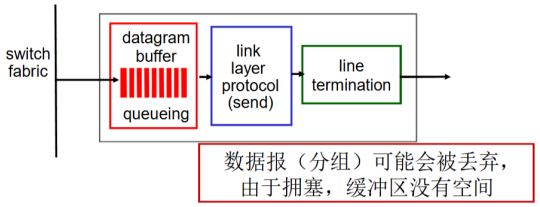


(3) 通过互联网络(crossbar等)的交换

- 同时并发转发多个分组，克服总线带宽限制
- Banyan（榕树）网络，crossbar(纵横)和其它的互联网络被开发，将多个处理器连接成多处理器
- 当分组从端口A到达，转给端口Y；控制器短接相应的两个总线
- 高级设计：将数据报分片为固定长度的信元，通过交换网络交换
- Cisco12000：以60Gbps的交换速率通过互联网络



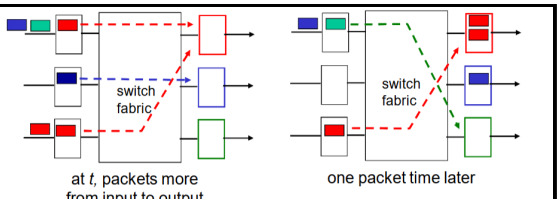
4.2.3 输出端口功能



数据报（分组）可能会被丢弃，由于拥塞，缓冲区没有空间

- 当数据报从交换机构的到达速度比传输速率快就需要输出端口缓存
- 由调度规则选择排队的数据报进行传输

优先权调度–谁会获得最优性能，网络中立？



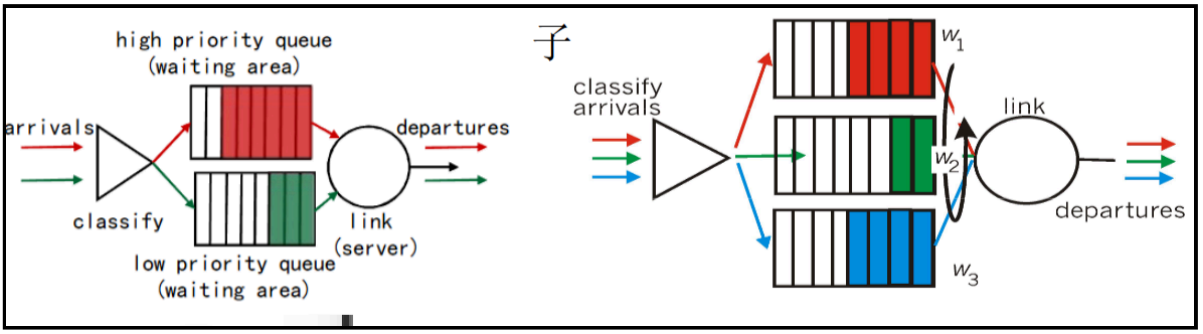
- 假设交换速率R_{switch}是R_{line}的N倍（N：输入端口的数量）
- 当多个输入端口同时向输出端口发送时，缓冲该分组（当通过交换网络到达的速率超过输出速率则缓存）
- 排队带来延迟，由于输出端口缓存溢出则丢弃数据报！

同样由于汇聚和流出速度的不一致性, 输出端口也需要一个 **缓存/排队** 来存储数据报
但是输出端口的数据报排队并不是遵循 FIFO 的, 而是根据一些**调度规则**来进行排队的

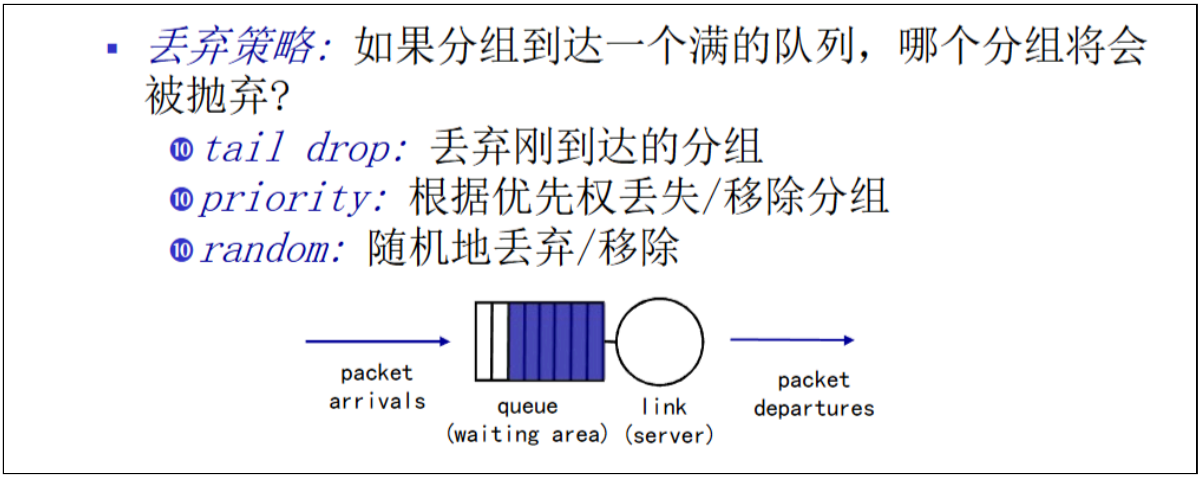
调度规则

调度 就是指 **选择哪一个分组作为下一个被转发的分组**, 调度规则有很多种:

1. **优先权调度**: 每个队列分配一个优先级（如高、中、低）。只要高优先级队列非空，就总是先发送它的分组；高优先级队列为空时才服务次高优先级，依此类推。
2. **轮询调度(Round Robin)**: 多个队列轮流服务。每个队列每次发送一个分组（或固定字节数），然后切换到下一个队列。若某队列为空则跳过。
3. **加权公平队列(Weighted Fair Queuing, WFQ)**: 类似轮询调度, 但每个队列分配一个权重。权重越高的队列在轮询中获得更多的服务机会。例如，权重为2的队列每轮发送两个分组，而权重为1的队列每轮发送一个分组 (给不同的队列分配不同的带宽)。

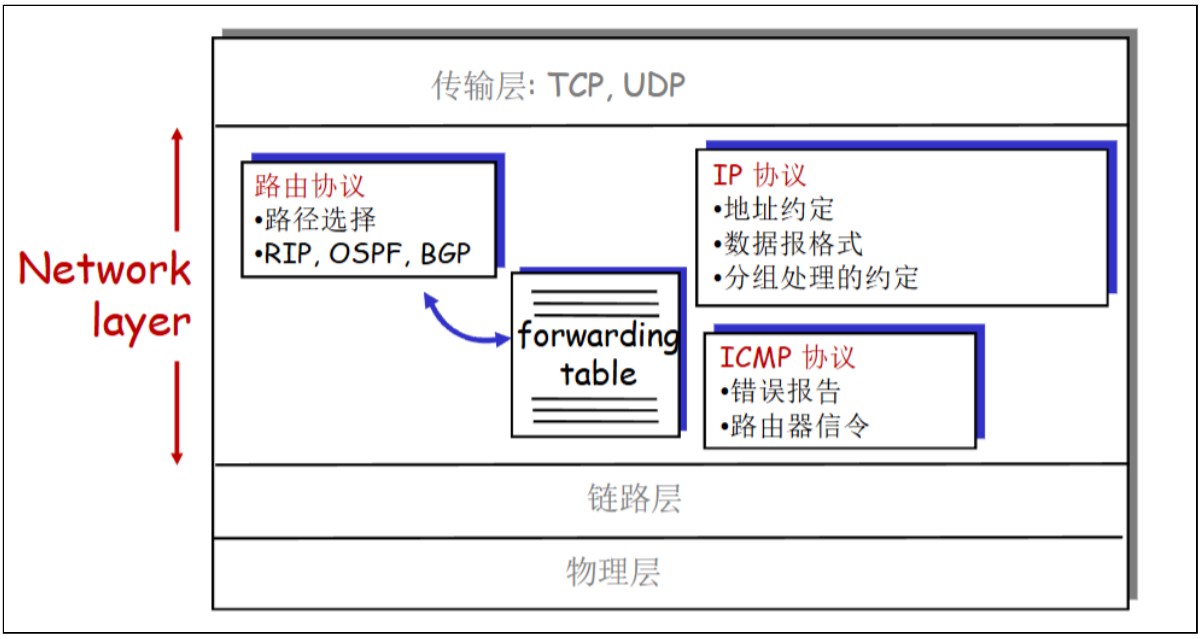


丢失策略



4.3 IP 协议

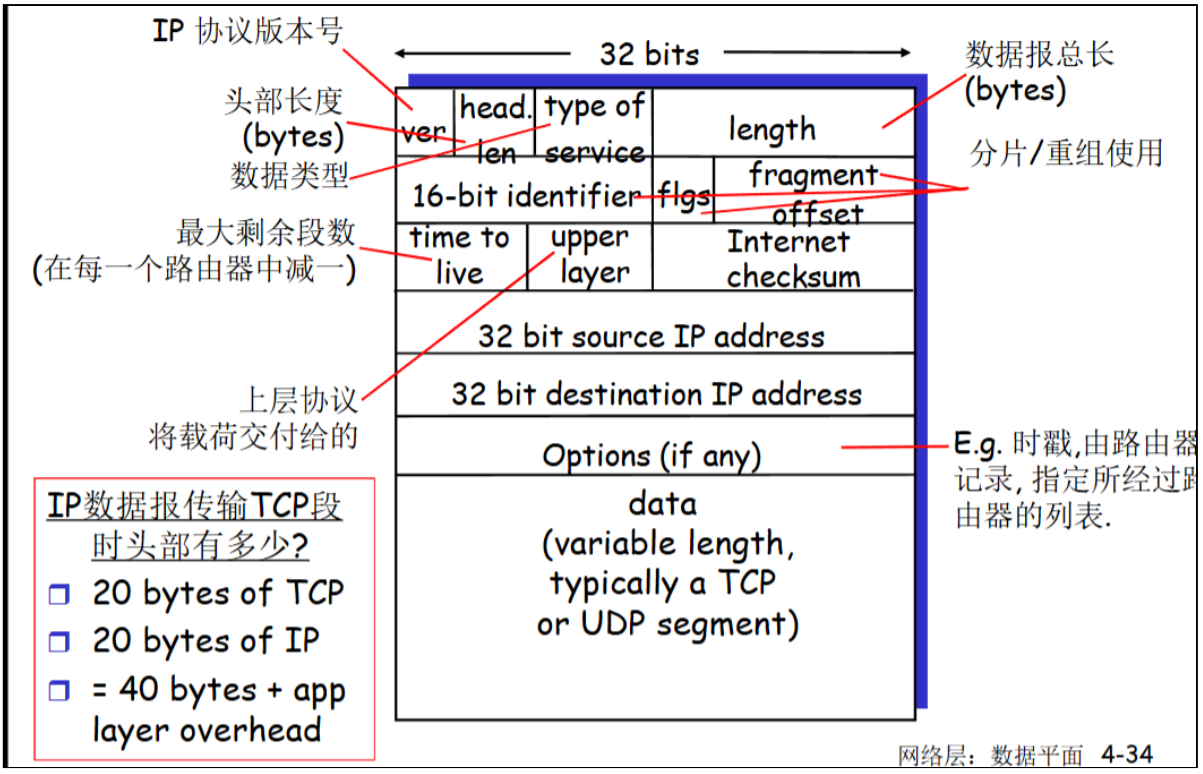
主机, 路由器中的网络层的功能分布如下:



其中, IP 协议的主要功能是 **实现数据平面的转发功能**, 包括

1. **地址约定**
2. **数据报格式**
3. **分组处理的约定**

4.3.1 IP 数据报格式(IPv4)

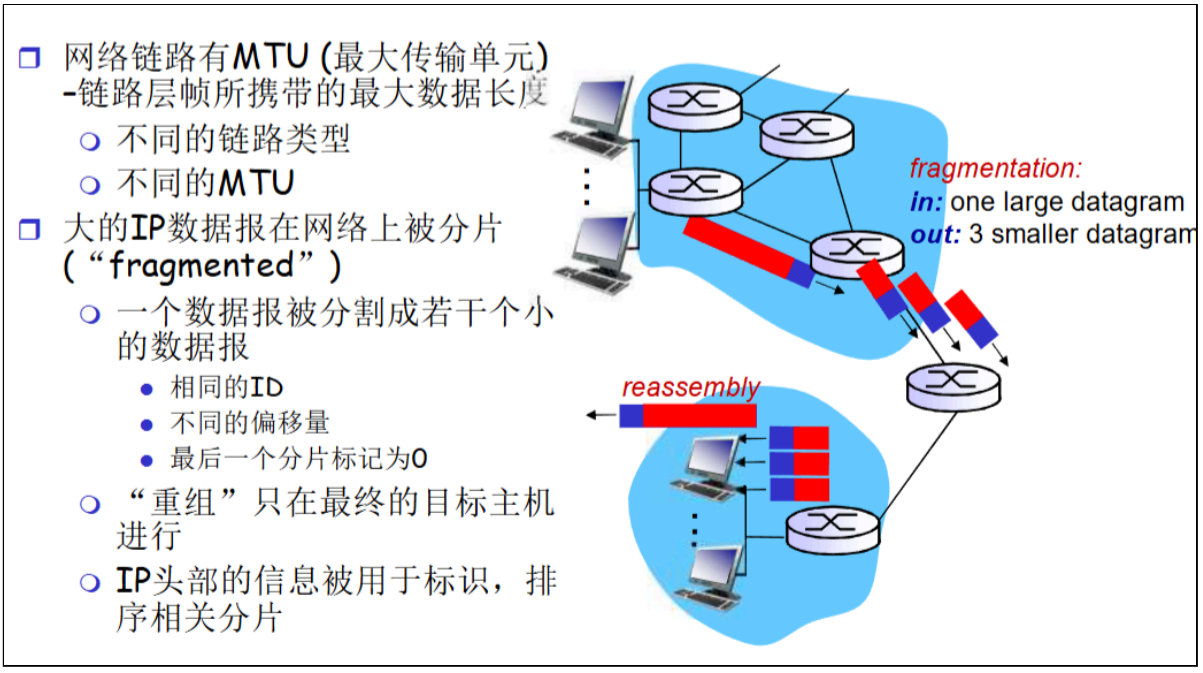


1. `head len` **字段(IHL)**: IP 数据报头部有固定的 20 字节, 以及可选的变长部分, 可选的头部长度可以通过`head len`减去 20 字节
   注意: `head len`字段的单位是 32-bit word, 因此它的值必须乘以 4 才能得到字节数, 意思就是可选的变长部分以 每 4 字节(一行)进行增加 (5 <= head len <= 15)

2. **第二行的字段**: 包括 `16-bits identification`, `3-bits flags`, `13-bits fragment offset` 这三个字段, 主要是为了实现 **IP 数据报的分片和重组功能**
3. `TTL` **字段**: 代表数据报在网络中可以经过的最大路由器数量, 每经过一个路由器就减 1, 当 TTL 减到 0 时, 数据报就会被丢弃
4. `Internet Header Checksum` **字段**: 代表 IP 数据报头部的校验和, 主要是为了检测数据报头部在传输过程中是否发生了错误

4.3.2 IP 数据报分片和重组

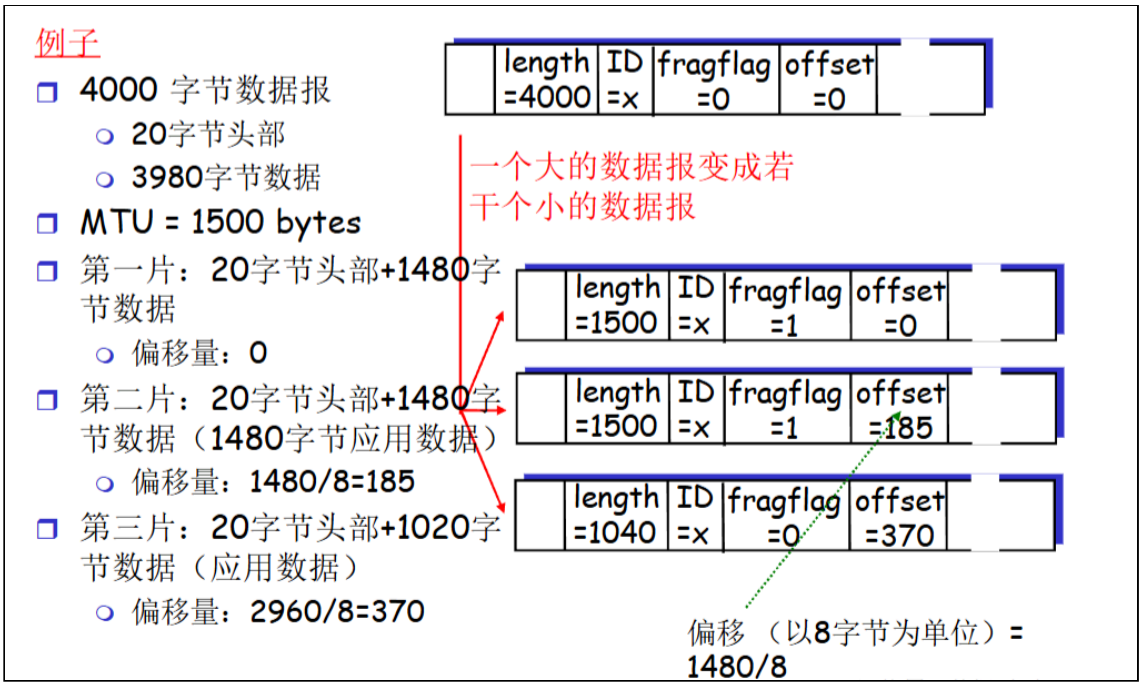
不同的链路会有不同的 MTU (Maximum Transmission Unit, 最大传输单元), 当一个 IP 数据报的长度超过了某条链路的 MTU 时, 就需要进行分片



不同的分片头部信息基本相同, 但也有些不同的部分用去**在目标主机重组**:

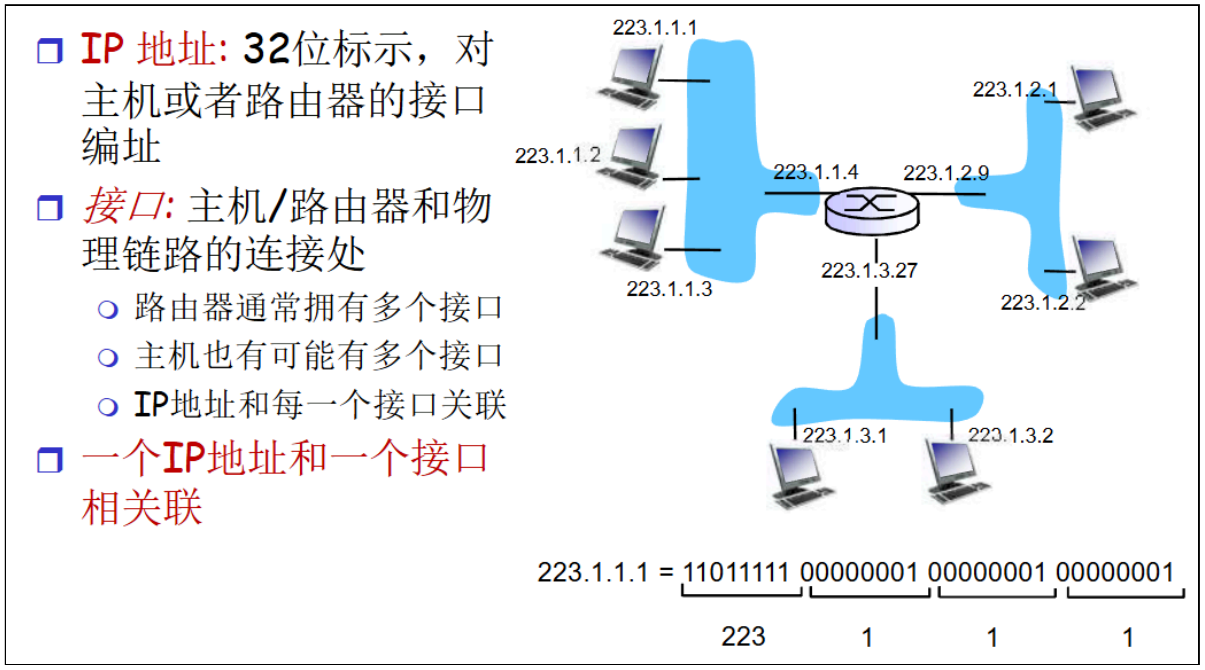
1. `identification` **字段**: 用于标识一个 IP 数据报的所有分片, 同一个数据报的所有分片具有相同的 `identification` 值
2. `flags` **字段**: 包含一个重要的标志位 `MF` (More Fragments), 当一个分片不是最后一个分片时, `MF` 位为 1; 当一个分片是最后一个分片时, `MF` 位为 0

3. `fragment offset` **字段**: 用于标识一个分片在原始数据报中的位置, 以 **8 字节为单位进行标识**, 因此它的值必须乘以 8 才能得到偏移量字节数 (这也决定了分片时**必须以 8 字节为最小单位进行分片**)



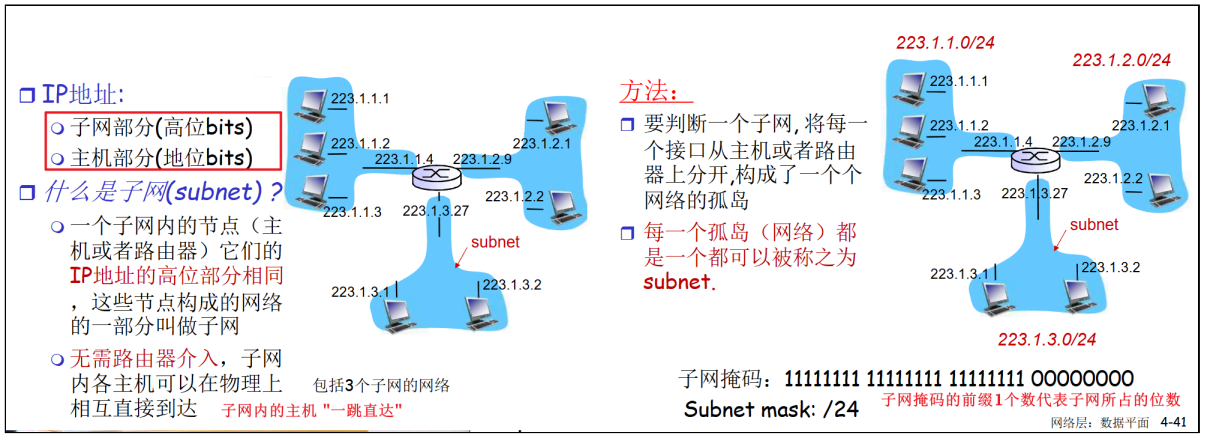
4.3.3 IP 编址

IP 编址是为了给网络中的每个主机分配一个唯一的地址, IP 地址不同部分划分**本质是通过对IP地址的划分来区分不同的网络和主机**



子网

观察可以发现, "同一区域" 的主机 IP 的前24比特位是相同的, 这里的 "区域" 就是一个**子网** (subnet), 也就是一个局域网 (LAN)



全球范围内路由器的转发实际上是以子网为单位进行的, 而不是以每个主机为单位, 这样的目的是减轻路由转发的负担

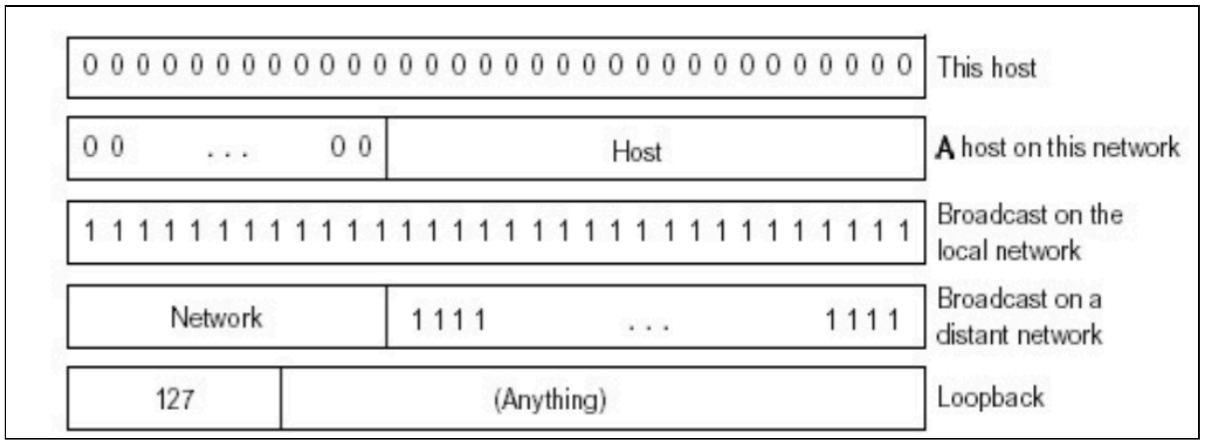
传统 IP 地址分类

传统上, IP 地址被划分为 A 类, B 类, C 类, D 类, E 类五类

类	最高位	网络号位数	主机号位数	默认掩码	用途
A	0	8	24	255.0.0.0	超大规模网络
B	10	16	16	255.255.0.0	大中型网络
C	110	24	8	255.255.255.0	小型网络
D	1110	—（组播组地址）	224.0.0.0 ~ 239.255.255.255	组播	
E	1111	—（保留研究）	240.0.0.0 ~ 255.255.255.255	实验	

> 现在无类别域间路由（CIDR）已取代传统分类，但这些范围仍用于特殊用途。

特殊的 IP 地址



特殊的 IP 地址 作用如下:

1. `0.0.0.0`: 表示“本网络上的本机”或“任意地址”。在服务器中代表监听所有可用接口。
2. `255.255.255.255`: 这个网络的所有主机, 本地网络有限广播，路由器不转发。
3. `127.0.0.0/8`: 本机回环地址, 数据不经过物理网卡

内网地址

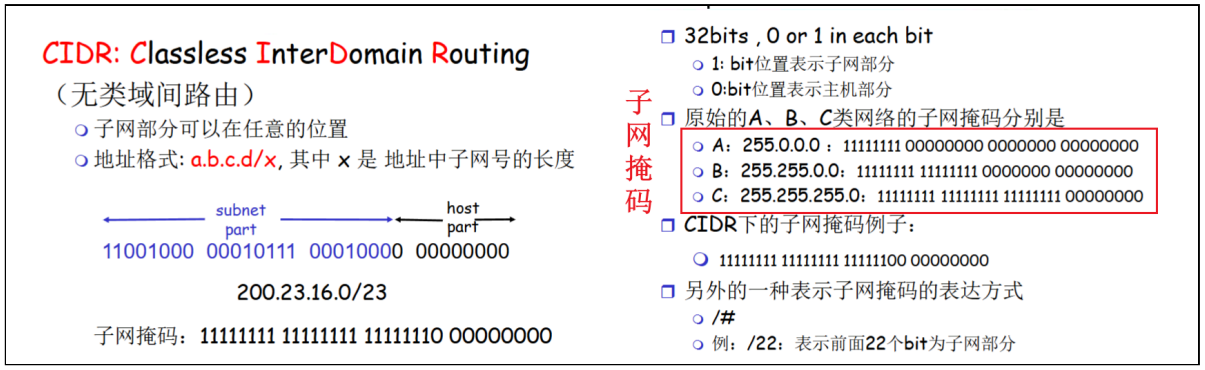
- 专用地址：地址空间的一部份供专用地址使用
- 永远不会被当做公用地址来分配，不会与公用地址重复
  - 只在局部网络中有意义，区分不同的设备
- 路由器不对目标地址是专用地址的分组进行转发
- 专用地址范围　　不会被路由器路由
  - Class A 10.0.0.0-10.255.255.255 MASK 255.0.0.0
  - Class B 172.16.0.0-172.31.255.255 MASK 255.255.0.0
  - Class C 192.168.0.0-192.168.255.255 MASK 255.255.255.0

内网地址在互联网范围内不具备意义，它只在内网范围内有效，也就是说，内网地址只能在内网范围内使用，不能在互联网范围内使用

无类域间路由选择 (CIDR)

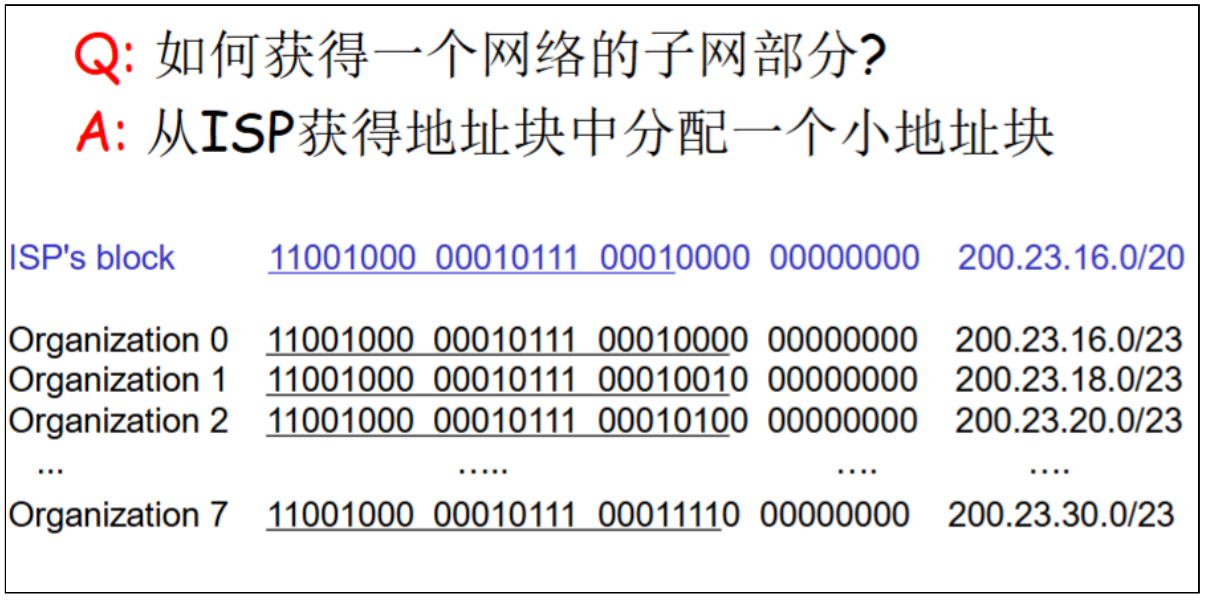
传统的 IP 地址分类方法，各个类的子网和主机数量是固定的，这样的话没法根据实际需求来分配 IP 地址，可能会造成 IP 地址资源的浪费

CIDR 是一种 IP 地址分配方法，它允许更灵活地划分 IP 地址空间(**通过子网掩码来实现子网部分和主机部分的任意划分**),从而更有效地利用 IP 地址资源



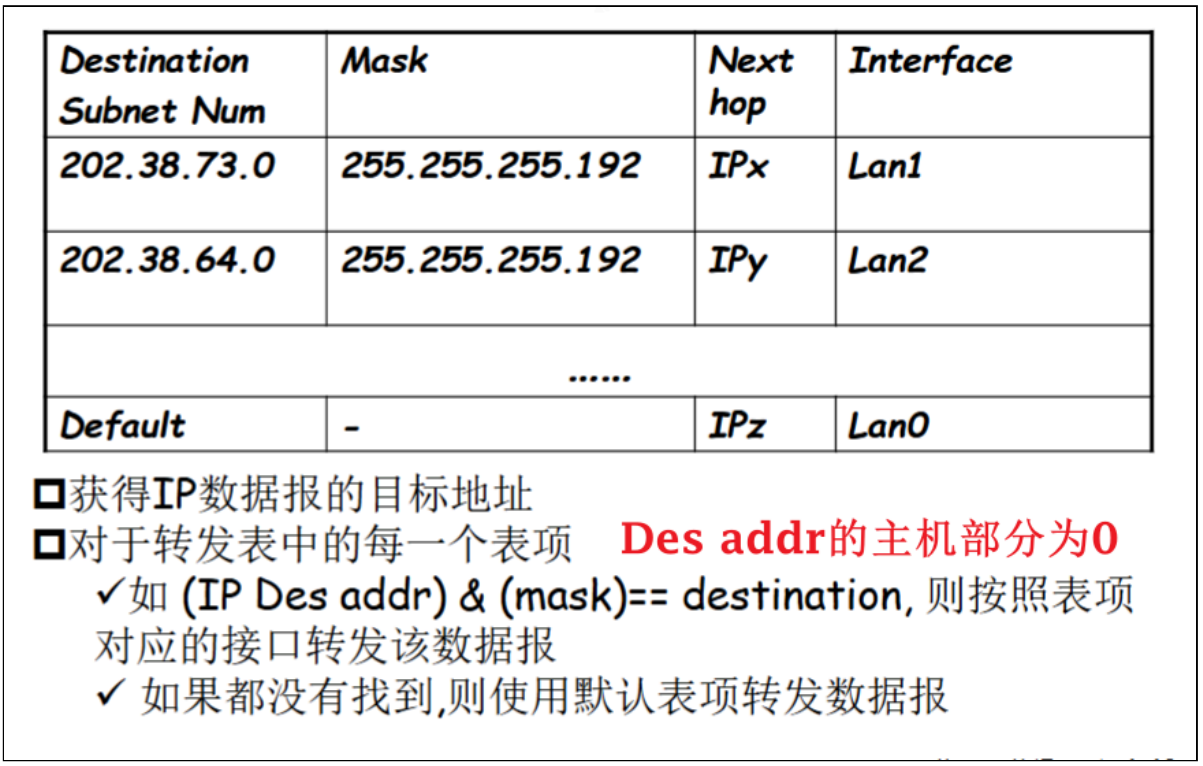
层次编址

一个 ISP 的地址块如何分配出不同的子网 分配给不同的部门



采取的就是 CIDR 的方式, 通过 **更长子网掩码** 来划分出不同的子网, 这样的代价是每个子网所能容纳的主机数量减少了.

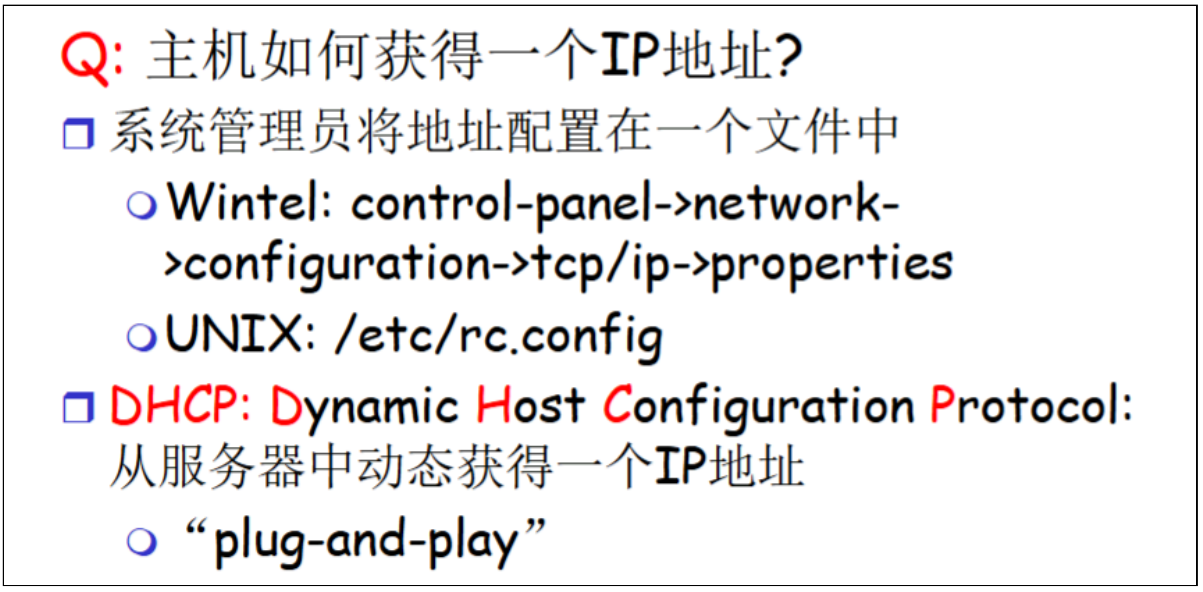
4.3.4 转发表和转发算法



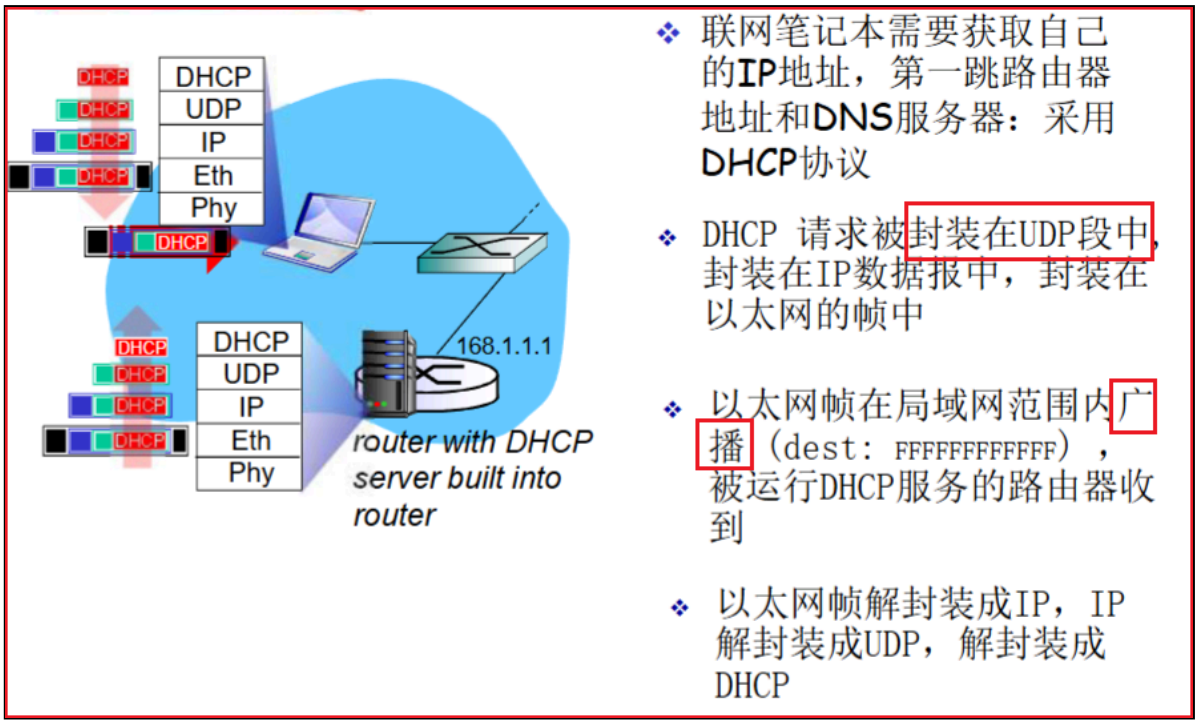
注意: 转发表的每一个 Destination 的主机部分都是0, 因为转发表是以子网为单位进行转发的, 而 IP 数据报中的 目标IP 的主机部分只在到达相应子网后才会被用来确定具体的主机, 在转发过程中, 目标IP 的主机部分并不参与转发决策

4.3.5 IP 地址的获取

当一个 计算机 开始联网时, 第一步要获得一个合适的 IP 地址, 有两种方式, 一种是**手动配置**, 一种是**DHCP** (Dynamic Host Configuration Protocol, 动态主机配置协议)

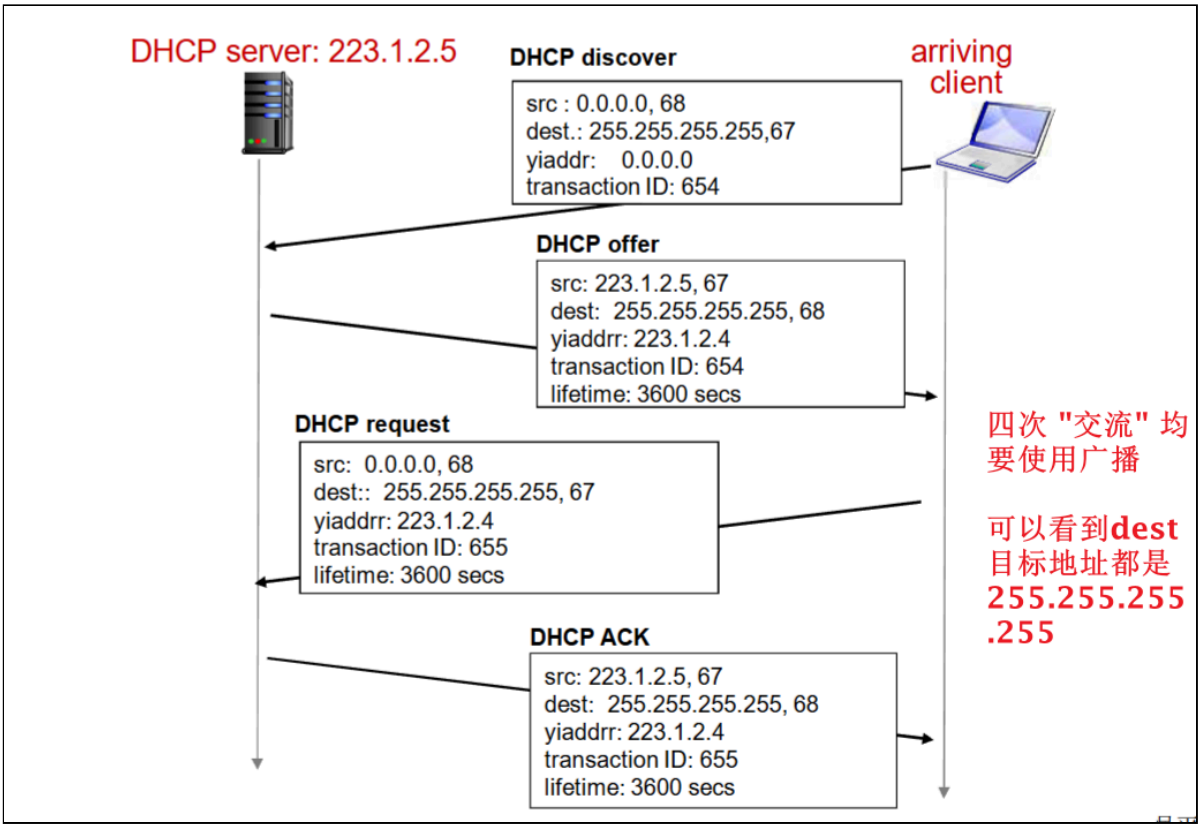


DHCP 自动分配



联网之前, 主机是没有 IP 地址的, 因此它无法直接与 DHCP 服务器通信, 只能通过**广播**的方式, 来建立联系, 最终获得 IP地址

广播交流 的过程如下:



ISP 获取地址块 -- ICANN 分配

全球范围内的 IP 地址分配由 ICANN (Internet Corporation for Assigned Names and Numbers, 互联网名称与数字地址分配机构) 负责, ICANN 将 IP 地址空间划分成多个地址块, 分配给不同的 ISP (Internet Service Provider, 互联网服务提供商), 然后 ISP 再将地址块划分成更小的子块, 分配给不同的客户

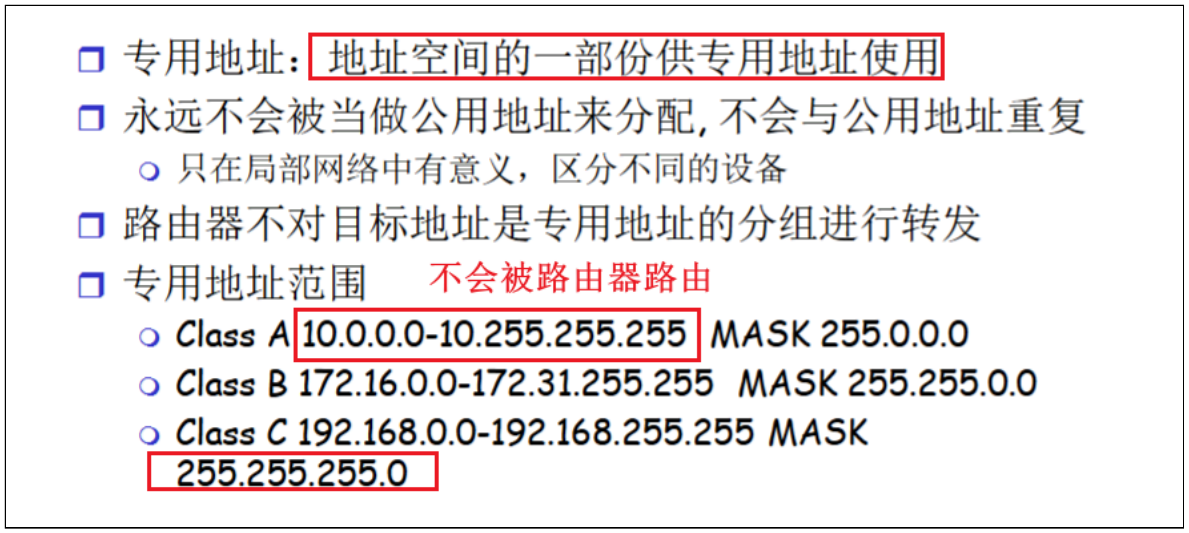
ICANN 负责的工作大致如下:

- **分配地址**

- **管理 DNS**
- **分配域名, 解决冲突**

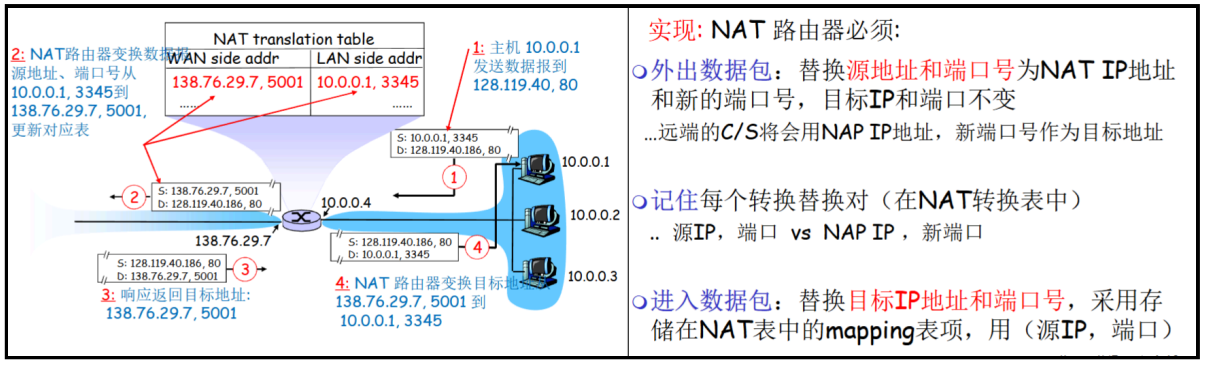
4.3.6 NAT -- 网络地址转换

前面有提到一些专用的 IP 地址, 内网地址.

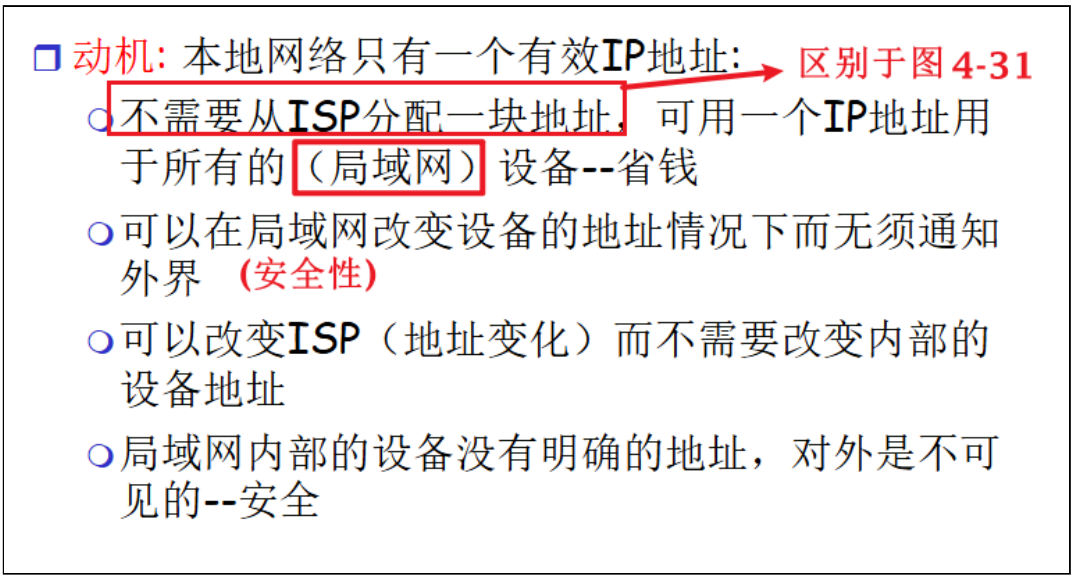


由于 IPv4 地址资源的紧张, 以及内网地址的使用, 导致了 NAT (Network Address Translation, 网络地址转换) 的出现

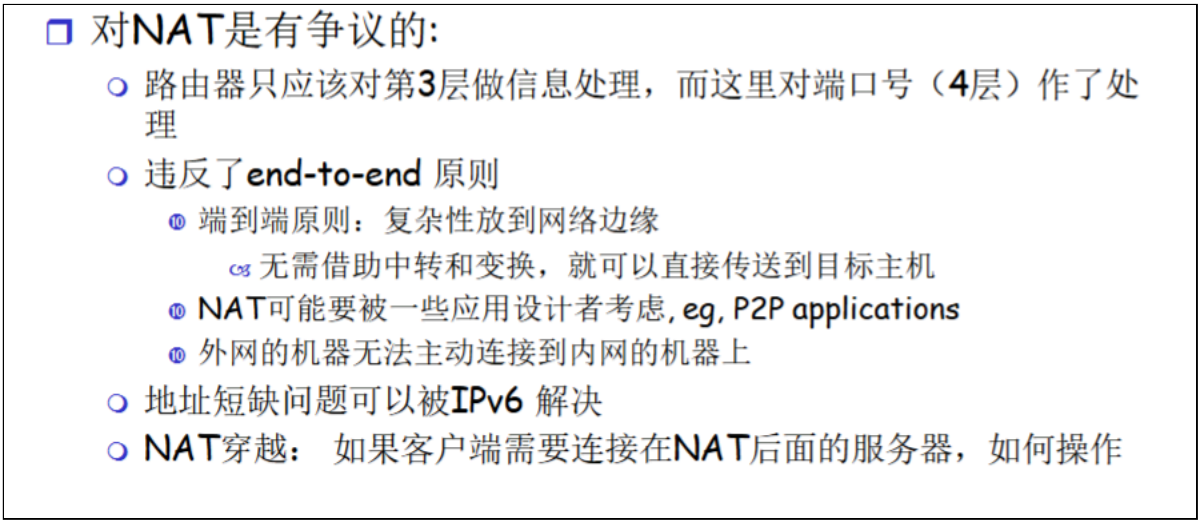
NAT 的工作流程



NAT 出现的原因/好处



NAT 的问题



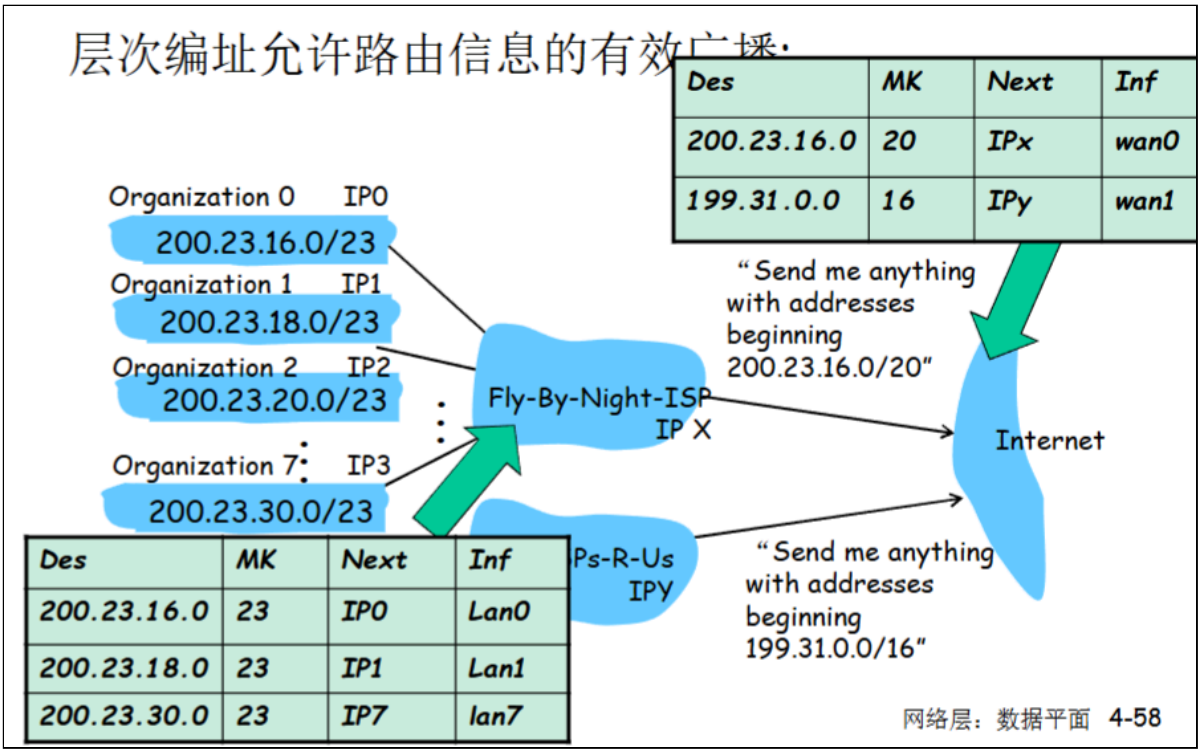
4.3.7 层次编址下的路由

前面提到了基于 CIDR 的层次编址, 那么这种编址下的路由是怎么样?

路由聚集

路由聚集就是 层次编址的反向过程

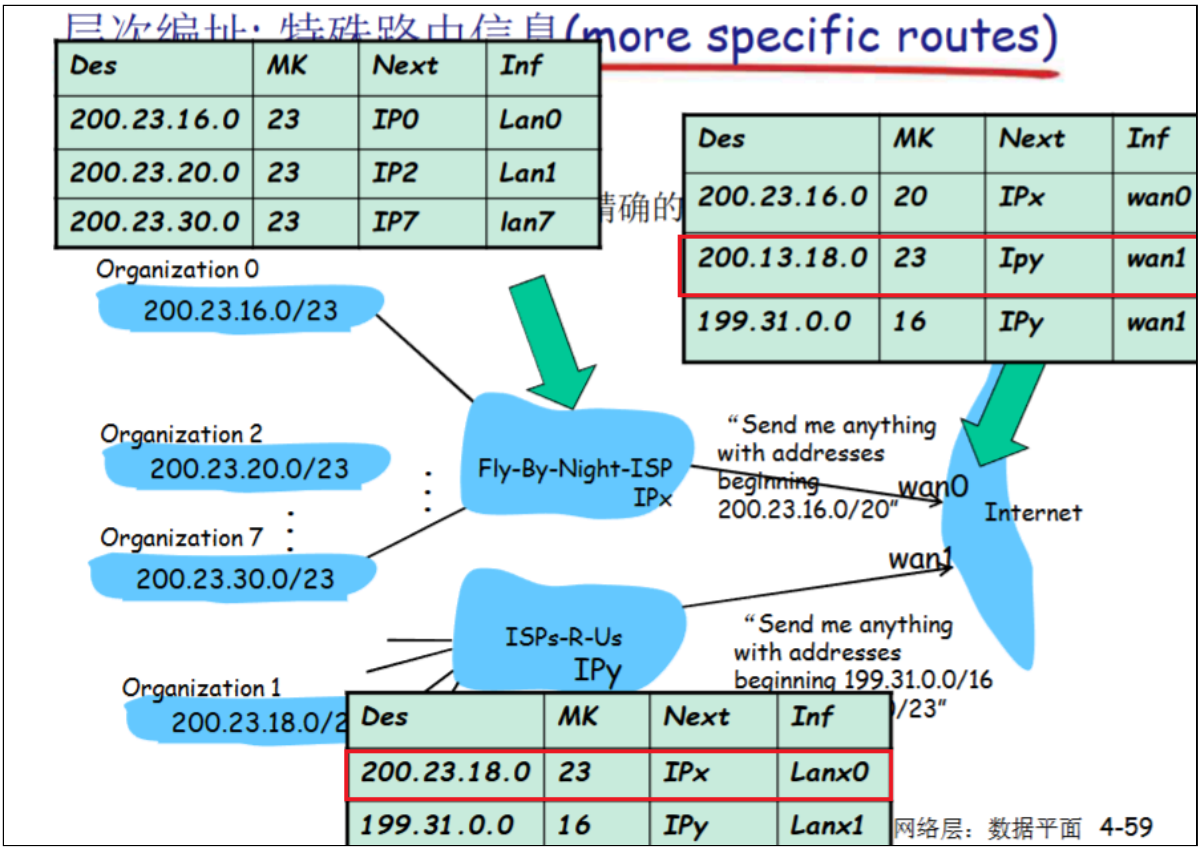
- **多个具有相同前缀**的子网可以聚集为一条**更短前缀**的路由条目。
- 路由器对外只宣告聚合后的前缀，内部子网细节被隐藏。
- 极大地减少了**路由表条目的数量。**



特殊路由信息

当出现**一个子网的 IP 被放到另一个 ISP 下**时, 路由聚集就会做出相应的信息更新, 此时也会充分利用 **最长前缀匹配** 的机制, 来保证数据报能够正确地被转发到目的地

下图的例子中, `200.23.18.0/23` 这一子网被分到了 ISPs 下, 因此该 ISP 的路由表以及层次更高的 **广域网** 的路由表中, 都会更新一条新的路由信息



4.4 ICMP

前面提到的网络层的服务时, 提到 IP 和 ICMP 分别负责不同的服务.

ICMP (Internet Control Message Protocol, 互联网控制消息协议) 是网络层的**控制与差错反馈协议** -- 它不传输用户数据, 而是让路由器或主机向源端报告网络异常, 诊断连通性, 典型应用就是 `ping`

ICMP 报文则**封装在 IP 数据报**中(协议号 Protocol = 1), 作为 IP 的载荷传输, 但承载的只是网络层的控制信息

1. 作用

ICMP 不是用来传输用户数据的，而是**用于网络层的控制和差错反馈。**它允许路由器或主机向源端报告网络异常、诊断连通性、传递控制信息。

> **典型场景**：当你 `ping` 一个不存在的 IP 时，你收到"目的不可达"的回应，这就是 ICMP 发的。

2. 报文封装

ICMP 报文封装在 IP 数据报中，**协议号（Protocol）为 1。**

```text
IP头部 + ICMP报文
```

Traceroute

Traceroute 用于探测源主机到目的主机之间**经过的每一跳路由器**的 IP 地址与往返时延(RTT), 原理是 **TTL 逐跳递减 + ICMP 超时报文**: TTL 从 1 开始逐次加 1, 第 n 跳路由器将 TTL 减到 0 后丢弃分组并回送 **ICMP 超时(类型 11)**, 源主机据此得到第 n 跳的地址与时延; 分组最终到达目的主机时, 因高位端口(如 33434)无服务监听, 目的主机回送 **ICMP 端口不可达(类型 3, 代码 3)**, 探测结束(Windows 的 tracert 改用 ICMP Echo 报文实现, 思路相同)

1. 目的

探测从源主机到目的主机之间**经过的每一跳路由器**的 IP 地址，并测量每一跳的往返时延（RTT）。

2. 基本原理

利用 **ICMP 时间超时（类型 11，代码 0）** 和 **IP 的 TTL 字段。**

- **TTL**：每经过一个路由器减 1，减到 0 时路由器丢弃该包，并向源发送 ICMP 超时报文。
- **逐步递增 TTL**：从 1 开始，每次加 1，让包依次在每跳"过期"，从而得到每个路由器的回应。

3. 具体工作流程（以 Linux/macOS 的 UDP traceroute 为例）

1. 源主机向目的主机发送一个 **UDP 数据报**（目标端口通常为 33434 起始的高位端口，期望不可达）。
2. 将 IP 头中的 **TTL 设为 1。**
3. **第 1 跳路由器**收到后 TTL 减为 0，丢弃 UDP 包，回送 **ICMP 时间超时（类型 11）** 给源主机。源主机得到第 1 跳的 IP 和时延。
4. 发送 TTL=2 的 UDP 包，第 1 跳正常转发，**第 2 跳路由器** TTL 减为 0，回送 ICMP 超时。得到第 2 跳。
5. 重复直到：
   - 达到最大跳数（默认 30）。
   - 或 UDP 包最终到达**目的主机。**目的主机发现端口不可达（UDP 没有服务监听），会回送 **ICMP 目**
     不可达（类型 3，代码 3）。源收到后得知已到达目的地，结束探测。

第五章：网络层 - 控制平面

- 5.1 路由的概念
- 5.2 层次路由

5.1 路由的概念

计算机网络中的路由是以**网络** (子网) 为单位进行的, 这样做的目的是使得**路由信息传输, 计算和匹配的代价低**

- "到了这个路由器, 就是到了这个网络"

5.2 层次路由

层次路由是将互联网分成一个一个的小AS, "自治系统", 每个AS有唯一的 ASN (AS number) 标识

- 同一个AS内路由器运行的相同的路由协议;
- 一个 ISP 可能包括1个或多个AS
- 一个 AS 内有通常有很多个子网和路由器

两个层次路由

1. AS 内部路由 - **IGP** - 内部网关协议
    - 如: RIP, OSPF, IGRP
    - *网关路由器*: AS边缘路由器, 可以连接到其他 AS
2. AS 间路由协议 - **EBGP** - 外部网关协议
    - 解决 AS 之间的路由问题, 完成 AS 之间的互联互通

第六章：链路层

- 6.1 概述与服务
  - 6.1.1 链路层提供的服务
  - 6.1.2 适配器通信
- 6.2 差错检测和纠错
  - 6.2.1 机制过程
  - 6.2.2 三种主要差错检测技术
- 6.3 多(址)路访问协议
  - 6.3.1 三种协议类型

6.1 概述与服务

6.1.1 链路层提供的服务

链路层协议

- 用来规范单段链路上的分组传输
- 定义了两个节点之间交互的**分组格式**
- 以及当发送和接收分组时节点采取的**动作**
  - 差错检测, 纠正(更复杂)
  - 重传/ **节点间的可靠传输**
  - 流量控制: 防止发送节点的发送速率过高
  - 随机访问
- **不同阶段采用不同的链路层协议**

链路层帧

- 首部包含若干字段: 编号, 物理地址等
- 不同的链路层协议, 帧格式可能不同

链路接入

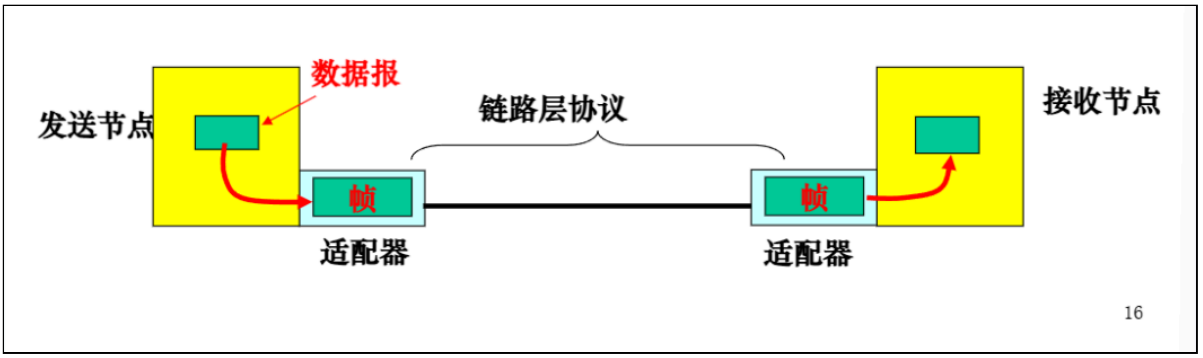
有**媒体访问控制 MAC 协议**规定帧在链路上传输的规则, 以下是两种网络链路

1. **点对点链路**: 只要链路空闲, 发送方都能够发送帧
2. **广播链路**: 多个节点共享一个链路(**多路访问**), 使用MAC协议协调多个节点的帧传输

6.1.2 适配器通信

适配器(adapter): 网络接口卡 (网卡, NIC, network interface card)

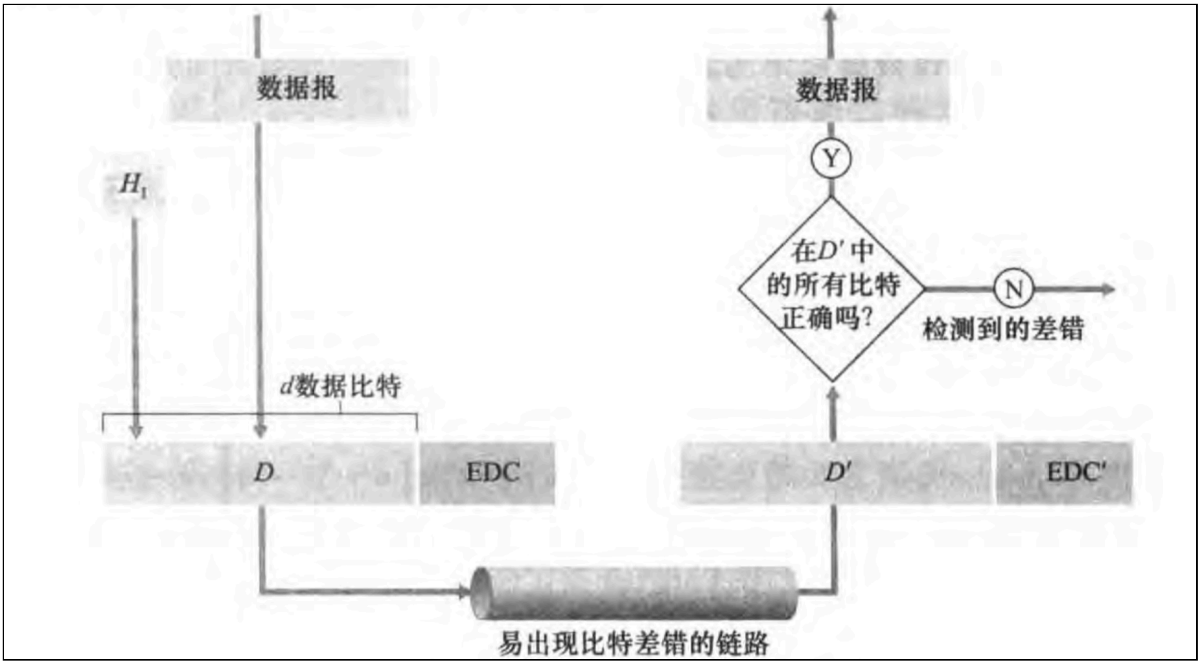
- 是一个电路板
- 实现物理层及链路层的主要功能



1. 数据报封装成帧
2. 帧传输
3. 适配器接收帧, 并
    - 差错检测: 发送适配器设置差错检测比特, 接收适配器完成差错检测
    - 可靠交付机制
4. 解封帧, 取出数据报, 递交网络层

6.2 差错检测和纠错

6.2.1 机制过程



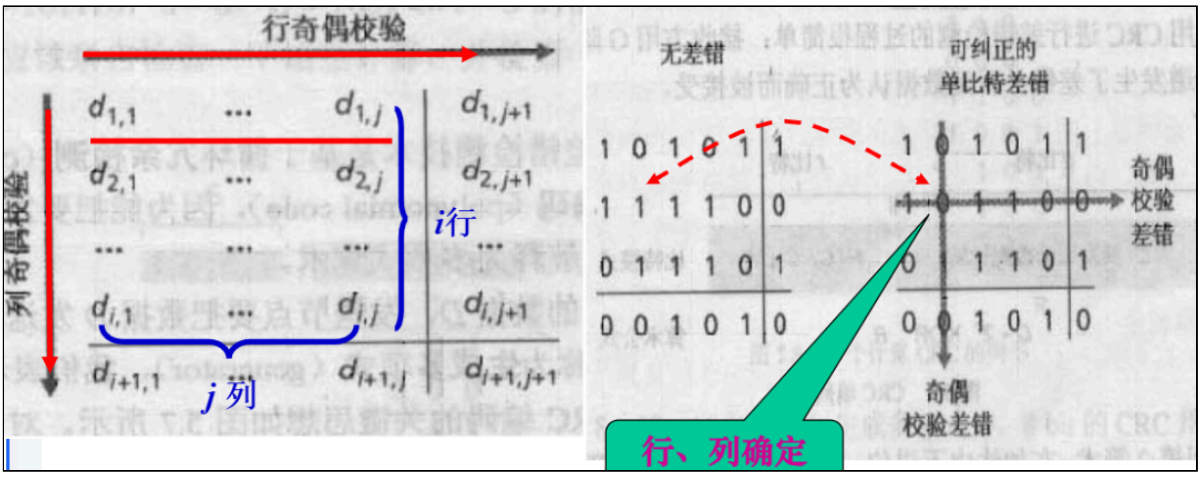
1. **发送节点**: 把数据D附加若干差错检测和纠错位EDC, 一起发送到链路.
    - 数据D包含网络层传来的数据报 + 链路级的寻址信息, 序列号等
2. **接受节点**: 接收到 `D' + EDC'`, 判断 `D'` 是否和初始的D相同
    - 正确: 解封, 提交
    - 出错: 差错处理

6.2.2 三种主要差错检测技术

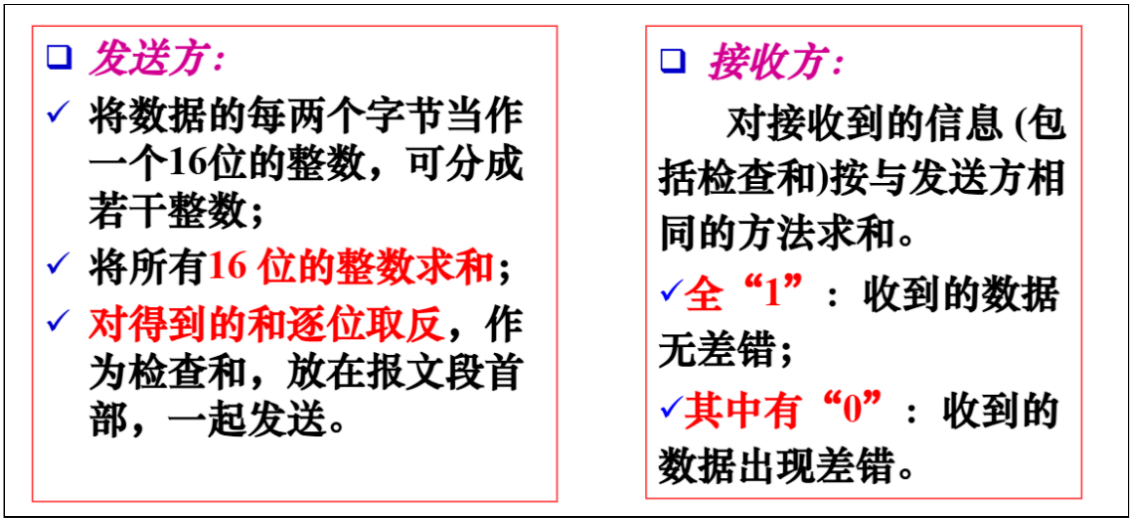
奇偶校验(一维), 基本方法

在d数据比特的后面加上一个校验位, 使得所有比特的'1'的个数为奇数或偶数; 双方约定**奇校验**还是**偶校验**

二维奇偶校验



校验和方法, 常用于 运输层



特点:

1. 分组开销小: 检查和尾数比较少
2. 差错检测能力**弱**
    1. 适用于运输层
    2. 链路层使用后面提到的更强的CRC方法, 有适配器硬件实现

循环冗余检测 CRC / 多项式编码

生成多项式 G:

- 双方共同选定
- r+1比特, 最高位必须是1, r+1通常是33
- G被发送方用来计算出一个 r 比特(余数)的附加比特R, 添加到 D 的后面 (**G 模2运算 D得到R**, G是除数)

接收方:

1. 用 G 去除 (模2运算, G是除数, DR是被除数) 接收到的DR (d+r比特)

- 余数为0, 传输正确, 去掉R, 得到D
- 余数不为零, 传输发生错误

6.3 多(址)路访问协议

两种网络链路, 点对点链路和广播链路, 本节学习**广播链路的信道共享技术**

如何**协调多个发送和接收节点对共享广播信道的访问**, 这就是本节需要学的 多路访问协议, 目的就是为了避免多个节点同时使用信道去传输帧, 使得接收方收不到正确的帧 (冲突导致所有帧受损)

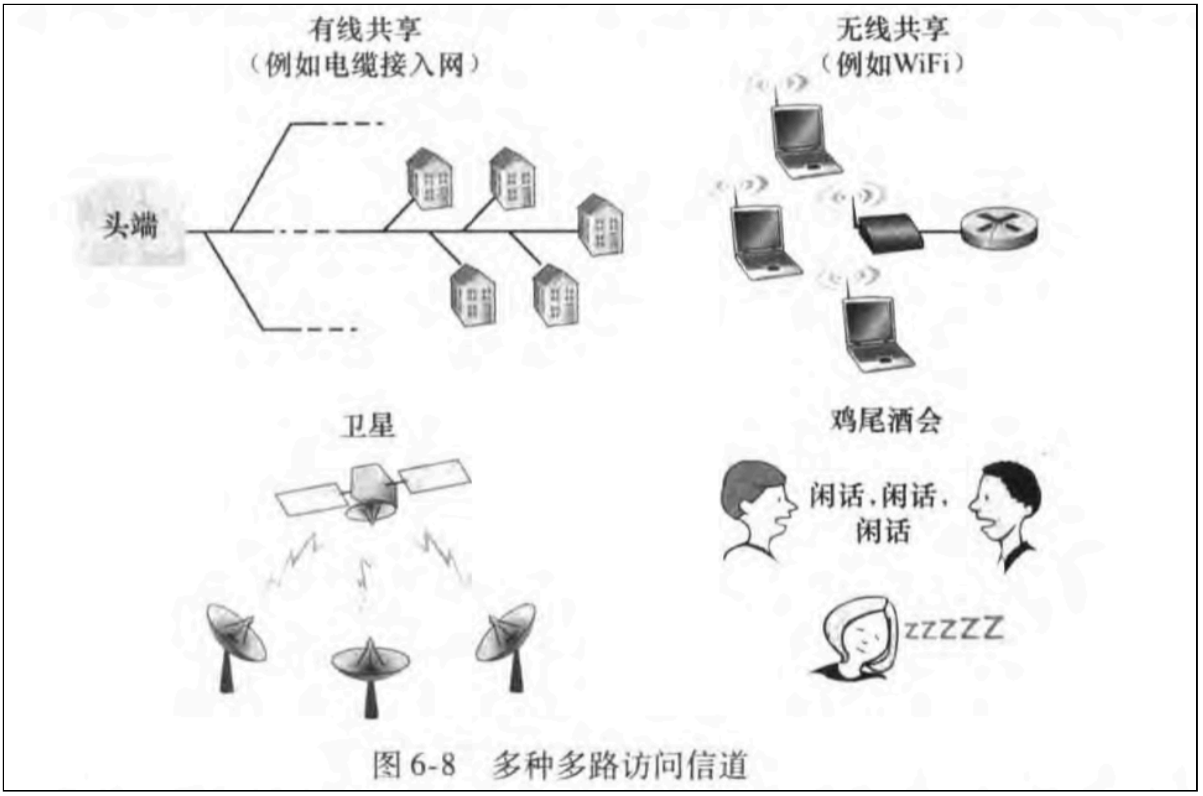


图 6-8　多种多路访问信道

多路访问的理想特性

设广播信道的速率为R b/s

- 只有一个节点传输时, 节点的吞吐量为 R b/s
- 有 M 个节点发送数据时: 每个节点吞吐量为 R/M b/s
- 协议是去中心化的 , 不需要主节点协调传输
- 协议成本: 实现方便, 价格适中

6.3.1 三种协议类型

1. 信道划分协议
    - 信道划分为小片(时隙)
    - 给节点分配**专用**小片
2. 随机访问协议
    - 不划分信道, 允许冲突
    - **能从冲突中恢复**
3. 轮流协议
    - 轮流访问信道来避免冲突

信道划分协议

主要有 **TDM, FDM, CDMA**三种

随机访问协议

基本思想:

- 发送节点以信道全部速率 (R b/s) 发送;
- 发生冲突时, 冲突的每个节点分别等待一个随机时间, 再重发, 直到帧(分组)发送成功.

典型随机访问协议:

- ALOHA 协议
  - 时隙ALOHA
  - 纯ALOHA
- 载波监听多路访问CSMA协议
- 带冲突检测载波监听多路访问CSMA/CD

轮流协议

轮流协议能够总结随机访问和信道划分协议的优点

- 轮流MAC协议
- 令牌传递协议